

Practical SRE Engineering

From protected journeys and error budgets to on-call, game days, and regional fail-over

Birol Tilki

Version 1.0 — 17 August 2026

Contents

1. Define What Reliability Must Protect
2. Choose Indicators Users Can Feel
3. Set SLOs and Error Budgets Across the Portfolio
4. Govern Change with Error-Budget Policy
5. Page on Burn Rate, Not on Every Symptom
6. Design On-Call as a System
7. Measure and Bound Toil Without Hiding the Work
8. Put Dependencies Inside the Reliability Contract
9. Degrade Deliberately Before Failure Cascades
10. Command Incidents Across Services and Tenants
11. Turn Incidents into a Reliability Learning Program
12. Design for the Loss of a Region
13. Run Game Days as a Recurring Program
14. Fail Over a Region Without Taking the Portfolio Down
15. Conclusion — A Governed Reliability Portfolio

Glossary and Abbreviations

References

How to Use This Book

Who this book is for

This book is for intermediate-to-advanced DevOps, cloud, infrastructure, platform, security, and **SRE (Site Reliability Engineering)** practitioners who need to govern reliability across a service portfolio that already has a delivery path, a security path, and an owned internal platform.

You should already be comfortable with Linux, Git, containers, **CI/CD (Continuous Integration and Continuous Delivery)**, cloud and Kubernetes fundamentals, infrastructure as code, networking, workload identity, observability, incident coordination for a single failed change, reconstruction of one environment, and the platform-product, tenancy, and fleet contracts taught in *Practical Platform Engineering*. The book does not teach those subjects from first principles. It uses them as the surface on which portfolio reliability decisions must operate.

You do not need to be an observability-stack author, chaos-engineering product specialist, public-cloud multi-region architect, or incident-command handbook author. Those disciplines contain important knowledge, but this book has a narrower promise: help you decide which user journeys reliability must protect, choose indicators users can feel, set portfolio **SLOs (Service Level Objectives)** and error budgets, write policy that can freeze change, page on burn rather than on every symptom, design on-call as a system, bound toil, put dependencies inside the contract, degrade before failure cascades, command multi-service incidents, turn incidents into learning, design for regional loss, run game days, and fail over a region without taking the portfolio down.

This is not a catalog of monitoring or paging tools. Products change. The durable skill is being able to explain:

- what user-visible journey is at risk, and for which services;
- what error budget remains, and what change it authorizes or freezes;
- what human system absorbs the failure without informal heroics; and
- what evidence proves the portfolio recovered—not only one service, one environment, or one control plane.

Relationship to earlier books

This book continues the Northwind Commerce system from *Practical DevOps Engineering*, *Practical DevSecOps Engineering*, and *Practical Platform Engineering*.

The DevOps book established one team's production delivery path: fast feedback, immutable artifact identity, reconciled infrastructure, a bounded Kubernetes runtime, workload identity, observable outcomes, progressive delivery, compatible data changes, safe asynchronous processing, GitOps reconciliation, cost and capacity, incident coordination, and reconstruction from durable evidence. Chapter 6 defined provisional **SLIs (Service Level Indicators)** and a burn policy for Storefront. Chapter 12 coordinated one failed production change. Chapter 13 reconstructed one environment. Those remain single-path capabilities.

The DevSecOps book made security of that same path explicit: assets and harms, threat and risk decisions, attributable identity, privileged delegation, supply-chain trust, vulnerability treatment, secrets, data protection, policy, runtime confinement, detection, investigation, eradication, bounded restored trust, and operational governance.

The Platform book turned repeated delivery and security capabilities into an owned internal product: users, jobs, tenancy, catalog, paved road, environments, contracts, shared control plane, guardrails, developer-experience measurement, quota, fleet lifecycle, support and escalation, and control-plane recovery that keeps tenant data planes bounded. Platform-product indicators such as time-to-first-environment remain a **job-time budget**. Catalog escalations storefront-oncall, fulfillment-oncall, and platform-oncall are contacts, not an on-call system. Chapter 14 restored a plane without taking tenants with it, and recorded not-regional-loss and not-portfolio-rto. Remaining owner reliability-program holds portfolio SLO governance.

Those capabilities are prerequisites here. This book does not repeat their implementation and present it as new SRE work. Instead, it asks the reliability questions that remain:

- Which user journeys must stay reliable, and what must not count as success?
- Which measurements are user-visible SLIs, and which are platform job-time or component uptime?
- What SLO and error budget govern Storefront and Fulfillment together?
- When remaining budget is exhausted, what change freezes—including a fleet step Platform already knew how to freeze for upgrades?
- Who is on-call, with what rotation, load, handoff, and authority?
- What happens when a region is lost, and what evidence proves the portfolio recovered?

The published DevOps, DevSecOps, and Platform v1.0 manuscripts remain unchanged. This book has a separate cumulative implementation under:

```
books/labs/sre/northwind/
```

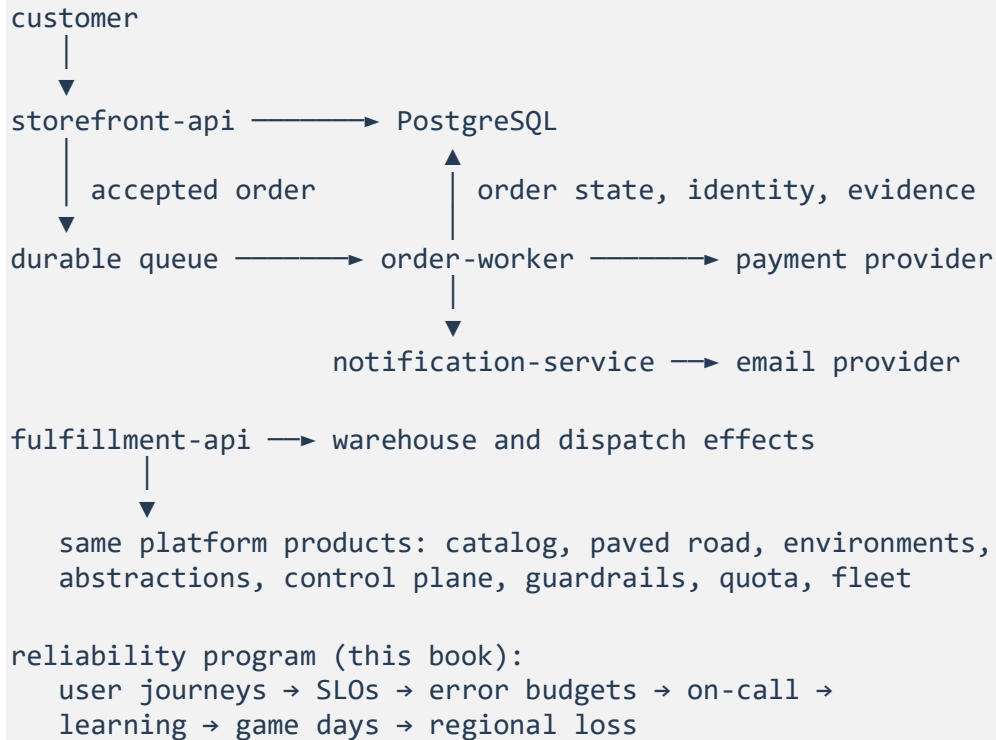
The new lab does not read earlier labs' working trees at runtime. It carries reduced interface fixtures whose manifests record source paths, source checksums, local checksums, and consumers:

```
inherited/devops-v1.1/  
inherited/devsecops-v1.0/  
inherited/platform-v1.0/
```

`inherited/devops-v1.1/` is a lab-snapshot identifier, not the published DevOps book version.

The Northwind system

Northwind Commerce remains deliberately small enough to understand and large enough to require real reliability decisions:



- storefront-api serves catalog reads and accepts orders.
- order-worker processes accepted orders asynchronously and can cause payment and inventory effects.
- notification-service sends non-critical confirmations. It is not a critical user journey.
- fulfillment-api is the second tenant's user-facing service. A Storefront SLO cannot stand in for dispatch.
- PostgreSQL stores order and processing state.
- A durable queue separates acceptance from asynchronous completion.
- Payment and warehouse providers are critical dependencies. Email is not.

The critical business outcome remains:

A valid order is durably accepted and reaches a correct terminal state without duplicate charge, invalid inventory state, or permanent disappearance.

This book adds a critical reliability outcome:

Storefront and Fulfillment keep user-visible SLOs; exhausted error budget can freeze change, including fleet change; on-call and learning are systems rather than heroics; a region can be lost and the portfolio recovered from a reviewed plan, with evidence that this is not reconstruction of one environment or restore of one control plane.

A green cluster is not that outcome. A launched dashboard of nines is not that outcome. A Platform upgrade freeze is not an error-budget freeze. Slack as the page destination is the opposite of an on-call system. Reconstructing one environment, or restoring a control plane, is not **Evidence of portfolio recovery**.

The cumulative reliability path

The chapters form one dependency chain:

```
protected journeys, refusals, and owners
→ SLI selection (accept / adjacent / reject)
→ portfolio SLOs, windows, and error budgets
→ error-budget policy that can freeze change
→ burn-rate paging versus symptom noise
→ on-call as a system
→ toil measurement and bounds
→ dependency contracts inside the journey
→ degradation and cascade control
→ multi-service incident command
→ learning with verified follow-through
→ regional-loss architecture
→ recurring game days
→ regional fail-over and portfolio recovery
```

Later chapters consume earlier decisions. The journey register determines which indicators Chapter 2 may accept. Accepted SLIs determine which SLOs Chapter 3 may publish. Remaining error budget determines which change Chapter 4 may freeze. Pages bind to the on-call system in Chapter 6, not to a catalog label. Game days in Chapter 13 must exercise error-budget freeze, on-call page path, dependency loss, and regional-loss tabletop or simulated fail-over. They are not a rehearsal of Chapter 14 fail-over.

The cumulative reliability failure is availability theater plus informal heroics: green graphs, exhausted budgets that do not freeze change, Slack-as-on-call, and a regional outage whose plan is invented during the event. It develops across SLOs, policy, paging, on-call, cascade, incidents, and fail-over.

Not every reliability failure comes from that story. Exercises are labeled according to their relationship with the main narrative:

- **Cumulative reliability failure:** advances availability theater and informal heroics.
- **Connected consequence:** demonstrates harm enabled by an earlier reliability decision.
- **Independent control failure:** proves that the control matters even without the cumulative story.

Four chapter forms

The book does not force every learning objective into the same structure.

Concept-led chapters

A concept-led chapter develops a mental model required for later production decisions. The reader may name protected journeys, refuse uptime theater, or design for the loss of a region. Its durable output can be a reviewed reasoning artifact rather than executable code.

Concept-led does not mean detached theory. Every substantial conceptual section must affect a chapter decision, durable artifact, interpretation of evidence, failure diagnosis, or later implementation dependency.

Decision-led chapters

A decision-led chapter compares approaches under explicit user journeys and operational constraints. It records the recommendation, trade-offs, owner, and review trigger.

A copied 99.9 percent target, a customer **SLA (Service Level Agreement)**, a dashboard of nines, or a survey score is an input—not the decision itself. An SLA is a customer or legal promise. An SLO is the internal reliability contract that error budget governs. Do not treat them as synonyms.

Implementation-led chapters

An implementation-led chapter starts from a red capability, establishes the necessary mental model, guides a contract or system change, exercises a controlled failure, diagnoses the evidence, and verifies containment or recovery where required.

Hybrid chapters

A hybrid chapter uses a concept or decision immediately to govern implementation. It still answers one primary production question.

There is no theory-to-practice percentage. **Decision 001** in `SERIES-DECISIONS.md` forbids adding an exercise merely to satisfy one. The practical requirement is consequential reader reasoning, meaningful change where appropriate, trustworthy evidence, and explicit limits on what has been proved.

Four kinds of evidence

Reliability mechanisms often produce impressive output while leaving the important claim unanswered. This book separates four evidence categories:

1. **Mechanism evidence** proves that a dashboard, alert, pager, runbook, or fail-over controller operated.
2. **Decision evidence** proves that an owner evaluated user journeys, error-budget consequences, and operational load.
3. **Outcome evidence** proves that a user-visible journey kept its SLO, or that remaining error budget was accounted.
4. **Recovery evidence** proves **Evidence of portfolio recovery**—not merely that one service returned, one environment was reconstructed, or one control plane was restored.

These categories are complementary. A published SLO catalog does not prove journeys hold. A healthy Storefront `order_success_ratio` does not prove Fulfillment dispatch succeeded, and it does not prove a platform job finished. Time-to-first-environment does not prove orders succeeded. A restored control-plane process does not prove a region failed over.

Do not call regional fail-over recovery **Evidence of restored trust**. That phrase belongs to DevSecOps compromise recovery. Do not call it **Evidence of restored isolation** or **Evidence of bounded platform-product recovery**. Those phrases belong to Platform. Use **Evidence of portfolio recovery**.

Do not call platform-product indicators portfolio SLOs. Platform job proofs time-to-first-environment, paved-road-completion, and catalog-freshness remain a job-time budget. Reserve **error budget** for SRE portfolio governance. A Platform fleet freeze for an upgrade is not an error-budget freeze. An error-budget freeze may halt that same fleet step for a reason Platform did not own.

Lab states and commands

Use Python 3.13. From the SRE lab root:

```
python3 -m venv .venv
source .venv/bin/activate
make bootstrap
make test
make lint
make audit
```

The lab uses these states when they support the chapter's learning objective:

- **Start:** cumulative prerequisites exist, but the chapter capability is incomplete.
- **Baseline:** the evaluator runs successfully and proves the declared weakness is present.
- **Complete:** the chapter decision or implementation satisfies independent expectations.
- **Challenge:** a concept or decision scenario exposes incomplete reasoning. Correction happens inside that write-up; there is no corrected lab state.
- **Failure:** an inert local input exercises burn, cascade, page-path, freeze, or regional-loss failure.
- **Contained:** modeled harm and change are bounded, but portfolio health is not yet assumed recovered.
- **Recovered:** required state is reconciled and journey plus portfolio-recovery evidence pass.

A successful baseline command means the evaluator correctly found the expected unsafe state. It does not mean the portfolio is already reliable.

Chapter commands follow a stable pattern where applicable:

```
make chapter-NN-baseline
make chapter-NN-checkpoint
make chapter-NN-challenge
make chapter-NN-failure
make chapter-NN-contain
make chapter-NN-recover
make chapter-NN-verify-recovery
```

Not every chapter uses every command. Concept-led and decision-led chapters do not manufacture a failure or recovery sequence when classification, modeling, or decision correction is the real work.

Chapter snapshots

Versioned start and complete tags are the exercise snapshots:

```
v1.0-chapter-NN-start
v1.0-chapter-NN-complete
chapter-NN-start
chapter-NN-complete
```

The versioned tags are the immutable release identity. The reader-facing aliases point to the same commits in this freeze and may advance in a later release. Tags are curated exercise snapshots, not merge milestones. Do not infer the teaching contract from Git ancestry.

Failure state is generated from the complete snapshot. An exhausted budget that did not freeze, a Slack-as-primary page, a retry cascade, or a mixed-region fail-over must never be preserved as if it were a trusted completed reference.

Create a working branch from the start snapshot and compare against the complete reference without merging it as a shortcut:

```
git switch -c my-chapter-NN chapter-NN-start
git diff chapter-NN-start chapter-NN-complete
```

Working-tree procedure

From the SRE lab root:

1. from the SRE lab root, create the Python virtual environment and run `make bootstrap`, `make test`, `make lint`, and `make audit`;
2. start from `chapter-NN-start` (or work in the completed tree if you already have later chapters);
3. run the chapter baseline and confirm that it detects the documented red state;
4. follow the chapter's guided work and preserve its decision or implementation evidence;
5. run the completed checkpoint;
6. run the declared challenge or failure where applicable;
7. verify correction, containment, or recovery where the chapter requires it; and
8. retain the chapter's durable outputs.

A concept-led chapter may change only reviewed YAML artifacts. The absence of application code does not make the reliability brief disposable.

Safe failure simulations

The lab's failures are deterministic, local, inert, and non-destructive:

- cross-tenant and cross-region access uses synthetic identifiers;
- burn, overload, and cascade cases are numeric fixtures, not live load tests;
- game days and fail-over mutate generated lab state only; and
- no real credential, cluster mutation, paging-vendor API, regional traffic shift, or destructive external action is included.

What local verification proves

The local evaluators verify structure, relationships, SLI class, remaining budget, freeze policy, page versus ticket, on-call rotations, toil bounds, dependency criticality, cascade denial, incident command, learning verification, game-day coverage, and bounded fail-over within the declared model.

They do not prove that a real telemetry backend, paging vendor, identity provider, multi-region fleet, or incident-management product enforces the same behavior.

Chapter 14 can prove that inventoried regional evidence was failed over and tenant isolation held in the model. It cannot prove that a production multi-region estate failed over. One-environment reconstruction and control-plane restore remain inherited recoveries. They are listed as insufficient.

Five principles and four reliability questions

The five principles established for the series remain active:

1. **Blast-radius control:** expose the smallest defensible scope to uncertain change, attack, or failure.
2. **Explicit contracts:** make identity, authority, trust, state, policy, outcomes, and failure behavior reviewable.
3. **Trustworthy evidence:** separate observations and expectations so a mechanism cannot approve itself.
4. **Reconciliation:** compare desired, recorded, external, and actual state, then make disagreement visible and owned.
5. **Recovery:** distinguish corrective action from proof that the protected outcome and required trust are healthy again.

SRE work repeatedly asks four reliability questions:

1. What user-visible journey is at risk, and for which services?
2. What error budget remains, and what change does it authorize or freeze?
3. What human system absorbs the failure without informal heroics?
4. What evidence proves the portfolio recovered—not only one service, one environment, or one control plane?

Dedicated failures name only the principles and questions that materially apply.

Best Practice and Production Practice

The book retains two labels from the series:

- **Best Practice:** a strong default that is broadly useful.
- **Production Practice:** how that default must be validated or adapted for the actual journeys, error budgets, on-call load, dependencies, failure modes, evidence quality, and recovery obligations.

For example, an SLO is a strong default. In production, its value depends on a user-visible SLI, a computed remaining error budget, a policy that can freeze change, an on-call system that can be paged, and evidence that recovery of the portfolio—not only one graph—has been proved.

How to judge your work

Do not ask only whether the dashboard is green. Ask:

What user-visible journey is at risk, what error budget remains and what change it freezes, who is on-call with authority rather than heroics, and what evidence proves the portfolio recovered—not only one service, one environment, or one control plane?

That question is the reading contract for the chapters ahead.

1. Define What Reliability Must Protect

Northwind already has a delivery path, a security path, and an owned internal platform. Storefront can promote a digest, bind workload identity, and stop a bad release. Fulfillment can obtain a bounded environment on a paved road. Those mechanisms do not answer the first reliability question:

Which user journeys must stay reliable, what does Northwind refuse to count as reliability, and who owns each decision?

Without that answer, reliability work becomes a dashboard of green graphs. The cluster is idle. The apiserver is Ready. The developer portal loads. Leadership asks whether Northwind has five nines. Each signal may be true, and a customer can still fail to complete an order while a warehouse dispatch never happens.

This chapter establishes that foundation. You will name the journeys reliability must protect, the measurements it refuses to count as success, the owners of those decisions, and the evidence that would prove the portfolio is reliable. The result is not a slide. Later chapters use it to choose **SLIs (Service Level Indicators)**, set **SLOs (Service Level Objectives)**, freeze change when budget is exhausted, and prove **Evidence of portfolio recovery**.

1. An unsafe reliability definition

The inherited DevOps, DevSecOps, and Platform systems protect a critical business outcome:

A valid order is durably accepted and reaches a correct terminal state without duplicate charge, invalid inventory state, or permanent disappearance.

That statement is strong for one path and one platform product. It does not describe a reliability program across Storefront and Fulfillment.

Consider four examples:

- Cluster and API uptime are green while `accept-and-complete-order` has no owner.
- Time-to-first-environment is treated as the portfolio SLO because leadership can see it.
- Fulfillment dispatch is assumed covered because Storefront orders look healthy.
- Reconstructing one environment, or restoring a control plane, is filed as regional recovery.

The Storefront order path can remain operable through all four. Northwind still does not have a reliability program.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 1 baseline:

```
make chapter-01-baseline
```

The command succeeds when it detects the intended unsafe definition:

```
chapter 01 baseline: uptime-theater success correctly detected
```

The fixture treats cluster and API uptime as success, leaves `accept-and-complete-order` without an accountable owner, and uses `cluster-uptime` as the later proof that reliability works. Graphs can be green while no journey has an owner.

That distinction drives the chapter.

2. The production model: journeys, refusals, promise, and owners

Theory — User-journey reliability model

This model enables Northwind to select reliability work according to user-visible journeys rather than according to component uptime, portal availability, or whoever owns the loudest dashboard.

A journey is a user-visible outcome that can fail

In this book, a protected journey is a customer or operator outcome that must remain reliable. `accept-and-complete-order` is a journey. `dispatch-fulfillment` is a journey. Cluster Ready is not a journey. A platform job such as `obtain-bounded-environment` is already named by Platform; it remains a **job-time budget**, not a user-facing journey.

A journey is not “the storefront-api Deployment.” A journey is not “Kubernetes.” Naming a component without a failed user outcome produces availability theater.

A failed outcome is what the user experiences, not what the graph shows

The journey must name:

- who is harmed when it fails;
- what failed looks like in user terms;
- who owns the reliability decision; and
- what later proof would show it keeps working.

If the later proof is cluster uptime, kubelet Ready, portal availability, or five nines of an apiserver, the journey has been replaced by theater. Chapter 2 will choose SLIs. Chapter 1 must not encode theater as the program’s definition of success.

A promise is what the reliability program commits to protect

The promise is the program’s public contract. For Northwind:

Storefront and Fulfillment keep user-visible journeys; exhausted error budget can later freeze change; component uptime is not success.

The promise is not “we will run **SRE (Site Reliability Engineering)**.” That sentence names a category. It does not name a journey or a refusal.

A refusal is a measurement that must not count as success, with a remaining owner

A program that will not refuse measurements becomes a dashboard with extra YAML. Every refusal must name who still owns that measurement. Otherwise the refusal is a vacuum, and the green graph returns as the SLO.

This book's first refusals are structural, not taste:

- cluster, API, kubelet, and portal uptime must not count as reliability success;
- platform job-time proofs remain a job-time budget owned by the platform team, not a portfolio SLO;
- one-environment reconstruction and control-plane restore remain inherited recoveries; they are not **Evidence of portfolio recovery**.

Best Practice: Write refusals in the same register as journeys, not in a slide appendix.

Production Practice: A refusal is only real when a later SLI candidate for that measurement is rejected or kept adjacent. Chapter 2 will test that.

User-journey evidence is not platform job-time and not component uptime

Time-to-first-environment can be healthy while orders fail. `order_success_ratio` can be healthy while Fulfillment cannot dispatch. Platform-product indicators describe platform jobs. User-journey later proofs describe customer outcomes. Neither is a customer **SLA (Service Level Agreement)**. An SLA is a legal or commercial promise. An SLO is the internal contract Chapter 3 will publish.

3. Build Northwind's reliability foundation

The completed Chapter 1 model uses four files:

```
reliability/brief.yaml
reliability/journeys.yaml
reliability/refusals.yaml
reliability/owners.yaml
```

The separation is deliberate. Owners describe who may be accountable. Journeys describe failed user outcomes and later proof. The brief binds promise, owner, and success evidence. Refusals name measurements that must not count as success and remaining owners.

Practice — Name the owners of the reliability program

Identify who owns the program before choosing indicators or tools.

Open `reliability/owners.yaml`. The register begins with four owners:

```
owners:
- id: reliability-program
  role: reliability-program-owner
  journeys: [accept-and-complete-order, dispatch-fulfillment]
- id: storefront-team
  role: application-team
  journeys: [accept-and-complete-order]
- id: fulfillment-team
  role: application-team
  journeys: [dispatch-fulfillment]
- id: platform-team
  role: platform-product-owner
  journeys: []
```

`reliability-program` is the remaining owner Platform already recorded on non-goal `portfolio-slo-governance`. If the program names only Storefront, Fulfillment's dispatch will be assumed covered. If it names "operations," no journey has an owner. The platform team remains owner of job-time proofs; it is not the owner of user-facing journeys. An empty `journeys` list on `platform-team` is therefore required, not an omission.

Practice — State falsifiable journeys

Express failed user outcomes the program can later prove without counting cluster uptime.

Open `reliability/journeys.yaml`:

```
- id: accept-and-complete-order
  user: storefront-team
  failed_outcome: A valid order is not durably accepted or does not reach
  a correct terminal state.
  owner: reliability-program
  later_proof: order_success_ratio
```

Inspect each journey with three questions:

1. Can you tell when the journey has failed without looking at cluster Ready?
2. Would the journey remain important if Northwind replaced the dashboard product?
3. Is the named owner authorized to change how the journey is protected?

`order_success_ratio` is an inherited DevOps outcome indicator. Chapter 1 names it as later proof. Chapter 2 decides whether it is an accepted user-journey SLI. Do not put time-to-first-environment on a user journey.

Do not expand the register into every component. Later chapters need a reviewable set of journeys, not an inventory of Deployments. `notification-service` is non-critical; it is not a third protected journey.

Practice — Bind promise, owner, and journey-completion evidence

Make success evidence match the journeys, and make ownership reciprocal.

Open `reliability/brief.yaml` and `reliability/refusals.yaml`. The brief's `success_evidence` values must be journey proofs, not `cluster-uptime`. Each refusal names a `remaining_owner`. The checkpoint verifies that owners exist, journeys have owners, theater proofs are rejected, job-time proofs are not used as journey success, and required refusals are present.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-01-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 01 checkpoint: reliability owners, journeys, refusals, and
journey-completion evidence verified
```

The audit validates the four Chapter 1 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- all independently required owners, journeys, and refusals exist;
- the brief names `reliability-program` as owner;
- success evidence and later proofs are not theater labels and not platform job-time ids;
- every journey names a known user, owner, failed outcome, and later proof; and
- every refusal names a remaining owner.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not prove that two journeys are sufficient for a real company. It does not prove that `reliability-program` has organizational authority, or that `warehouse dispatch` is the right second journey. Those are judgment claims.

This distinction is the chapter's evidence model:

Evidence category	Chapter 1 evidence
Mechanism evidence	Schemas and the relationship evaluator operated successfully.
Decision evidence	Owners, journeys, promise, refusals, and later proofs are explicit and reviewed.
Outcome evidence	Later SLI, SLO, and recovery chapters can be evaluated against these journeys.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 1 primarily creates decision evidence. Pretending that the local checkpoint proves journeys already keep their SLOs would weaken every later chapter.

4. Test the model under failure

Independent control failure — Cluster and API uptime counted as success

Practice — Diagnose uptime theater

Replace cluster-uptime evidence with owned journey-completion evidence.

The baseline fixture contains this model:

```
promise: Keep the graphs green.
owner: unassigned
success_evidence:
  - cluster-uptime
  - api-uptime
```

The problem is not merely missing fields. The model classifies component uptime as the program.

Severity: high; every later SLO, alert, and freeze conversation will optimize graphs instead of journeys.

Plausible harm: orders fail while Ready stays green; Fulfillment dispatch is unowned; a region can be lost with no named journey to recover.

Potential blast radius: every service asked to “be reliable”; Storefront’s provisional indicators remain the only real signal.

Bounded by: later SLI decisions, error-budget policy, and regional architecture. None repairs a program definition that counts uptime theater.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** The journeys are accept-and-complete-order and dispatch-fulfillment; cluster and API uptime are not those journeys.
- **Error budget:** Not yet applicable as a freeze decision. Chapter-local implication: a program with no journey cannot later compute remaining budget.
- **Human system:** Not yet applicable as on-call design. Chapter-local implication: an unowned program invites Slack-as-primary when graphs turn red.
- **Portfolio recovery:** Not yet produced. Cluster uptime cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling the program “keep the graphs green” encourages graph controls: scrape kubelet, count Ready replicas, publish an uptime tile. Those may reduce orientation noise, but they do not answer which user fails, who owns the wait, or what evidence would falsify success.

The missing owner makes SLI selection and on-call ambiguous. The missing refusals make every green tile in-scope. The missing journey proof leaves Chapter 3 nothing honest to target.

Correction

The completed model does not elevate uptime into the program. It defines owners, journeys, a promise about user-visible outcomes, and success evidence that later chapters can collect without a cluster graph.

That correction changes later decisions:

- Chapter 2 must accept `order_success_ratio` as a user-journey candidate and keep `time-to-first-environment` adjacent.
- Chapter 3 must give Fulfillment its own SLO; Storefront cannot cover dispatch by copy.
- Chapter 4 must freeze change against journey budget, not against cluster Ready.
- Chapter 12 must list one-environment reconstruction and plane restore as insufficient for regional loss.

The concept is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common modeling errors

Naming the dashboard as the program

A dashboard is a distribution surface. If it disappeared, the journeys should still be nameable. If they are not, there was no reliability program.

Treating Storefront's provisional indicators as the portfolio

One team's DevOps Chapter 6 burn policy is not a portfolio. Fulfillment's missing journey is the demand signal.

Refusing nothing

A program that accepts cluster uptime, platform job-time, and inherited restores as success will not have time to protect orders. Record remaining owners.

Using platform job-time as user-journey success

Healthy time-to-first-environment can hide failed orders. Journey evidence must describe user outcomes.

Assigning ownership without authority

A reliability program cannot own `order_success_ratio` if it cannot change how that journey is measured or frozen. Record the real decision path. Inherited `self_approval_forbidden` will bind on-call authority in Chapter 6; it does not replace an owner here.

6. What changed

Before	After
Success was cluster and API uptime.	Success evidence is <code>order_success_ratio</code> and <code>dispatch_success_ratio</code> .
Owners were "operations."	reliability-program, Storefront, Fulfillment, and the platform team are named, with journeys.
Journeys were component names.	Journeys are failed user outcomes with owners and later proofs.
Refusals lived in conversation.	Refusals name remaining owners for uptime theater, job-time-as-SLO, and inherited restores.
A valid schema could appear to prove a sound program.	Structural, decision, outcome, and recovery evidence remain distinct.
Later chapters had no stable journey reference.	SLI selection, SLOs, freeze policy, and fail-over inherit this brief.

What changed was not merely four YAML files. Northwind now has a falsifiable reliability contract that later chapters can measure, freeze, page, and recover.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Reliability brief	reliability/brief.yaml	It retains the promise, owner, and journey-completion evidence later SLOs must not replace with theater.
Journey-and-refusal register	reliability/journeys.yaml and reliability/refusals.yaml	They retain the failed outcomes and the measurements the program will not count as success.

These artifacts should change when Northwind's journeys or organizational authority materially change—not whenever a dashboard theme is redesigned.

What You Learned

A reliability program is defined by protected journeys, a promise, explicit refusals, and accountable owners. A dashboard, cluster, or portal is a resource that may serve those journeys. Theater evidence cannot prove the program. Schema checks can prove structural completeness within declared scope. They cannot prove that human judgment is correct. A concept earns its place when it changes later production decisions, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Model catalog browse without copying Fulfillment

Decide whether Northwind should protect customer catalog reads as a journey or refuse them as a supporting measurement.

Extend the Chapter 1 model without adding implementation policy yet:

1. Decide whether catalog browse is a new journey or a supporting signal of accept-and-complete-order.
2. If it is a journey, name a failed outcome that is not “the Deployment is not Ready.”
3. Choose later proof that is not cluster-uptime, api-uptime, time-to-first-environment, or csat.
4. Assign an accountable owner and a refusal if the team asks to count kubelet Ready as success.
5. Identify one observation that would falsify the decision.
6. Explain which material change would trigger review of your decision.

Do not copy the Fulfillment entry and rename it. Catalog reads have different criticality, dependencies, and freeze consequences. Your durable output is the decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 1 capability when you can explain why cluster uptime is not a finished journey, trace every journey to a known user and owner, describe evidence that would falsify the promise, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Northwind now knows which journeys reliability must protect and what it refuses to count as success. It still does not know which measurements may represent those journeys, which inherited indicators stay adjacent as platform job-time, and which component graphs must be rejected.

Chapter 2 turns the reliability brief into an SLI method with accept, adjacent, and reject decisions.

2. Choose Indicators Users Can Feel

Chapter 1 named the journeys, refusals, promise, and owners. That brief does not decide which inherited graph becomes a user-visible **SLI (Service Level Indicator)**.

The production question is now:

Which measurements are user-visible SLIs, and which measurements must stay non-indicators for the portfolio?

Without a selection method, every green tile looks like “the SLO.” Leadership can see time-to-first-environment. Storefront already emits `order_success_ratio`. CPU is idle. Replica Ready is true. Fulfillment still has no user-visible indicator. Promoting platform job-time or component uptime hides user harm. Measuring nothing leaves dispatch unowned.

This chapter records accept, adjacent, and reject decisions against the Chapter 1 journeys. Later chapters publish targets; they must not reopen job-time as a portfolio **SLO (Service Level Objective)** because the dashboard already shows it.

1. An unsafe SLI decision

A weak record says:

```
candidate: time-to-first-environment
treatment: accept
class: portfolio-slo
justification: leadership-can-see-it
```

It does not identify a Chapter 1 journey, a good-event definition, a remaining owner for job-time, or a review trigger. “Leadership can see it” may justify a conversation. It cannot complete an SLI decision.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 2 baseline:

```
make chapter-02-baseline
```

The command succeeds when it detects the intended unsafe selection:

```
chapter 02 baseline: job-time classed as portfolio-slo correctly detected
```

The fixture accepts `time-to-first-environment` as a portfolio SLO because leadership can see it, leaves `order_success_ratio` as a tenant-workload non-metric, and never accepts a Fulfillment dispatch indicator. Visibility has been treated as user impact. A Chapter 1 refusal has been absorbed. Dispatch still has no signal.

That inversion drives the chapter.

2. The production model: good events, classes, and Goodhart's nines

Theory — SLI selection

This model enables Northwind to choose measurements according to user-visible journeys rather than according to whichever graph leadership can already see.

An SLI is a good-event ratio a user can feel

An SLI answers whether a named journey succeeded for a user in a window. The numerator is good events. The denominator is valid events. A request SLI counts HTTP responses that the user needed. An event SLI counts business outcomes such as a correctly completed order. A windowed SLI counts time buckets in which the journey stayed healthy.

CPU idle is not a good event. Replica Ready is not a good event. A portal page load is not a good event for `accept-and-complete-order`. `order_success_ratio` can be a good-event ratio for that journey because Chapter 1 already named it as later proof. `dispatch_success_ratio` can be a good-event ratio for `dispatch-fulfillment`. Chapter 2 decides the class. Chapter 3 will set the target.

Class is user-journey, adjacent job-time, or rejected uptime

Northwind can:

- **Accept** a measurement as a user-journey SLI when it traces to a Chapter 1 journey and a user can feel the failed event;
- **Keep adjacent** a platform-product indicator that remains a **job-time budget**, with the remaining owner already recorded in `reliability/refusals.yaml`; or
- **Reject** component uptime, kubelet Ready, portal availability, and other theater the brief already refused.

“Backlog,” “maybe later,” and “the dashboard already has it” are not treatments. `portfolio-slo` is not a class this book may assign to a platform job proof. Platform Chapter 10 already recorded `time-to-first-environment` as `platform-product-sli`. This chapter must not promote it.

Adjacent is not reject, and it is not accept

Job-time proofs still matter. Fulfillment still needs a bounded environment. The reliability program must not spend those proofs as evidence that orders succeeded, and must not discard them as noise. Adjacent means: reference the inherited id, keep class `platform-product-sli`, name `platform-team` as remaining owner, and leave **error budget** for Chapter 3's user-journey SLOs.

Platform classified `order_success_ratio` as a tenant-workload non-metric so it could not prove a platform job. That classification is not a reliability reject. The same id is an inherited DevOps outcome indicator. Accepting it as a user-journey SLI does not rename the Platform fixture and does not spend it as `time-to-first-environment`.

Goodhart's law applies to nines

A measurement that becomes the SLO will be optimized. If the SLO is cluster Ready, teams will keep Ready green while orders fail. If the SLO is time-to-first-environment, teams will provision faster while dispatch disappears. If the SLO is a copied Storefront 99.9 percent, Fulfillment will inherit a number with no journey.

Best Practice: Require the same fields on every treatment: class, journey or remaining owner, owner, and review trigger.

Production Practice: An adjacent decision is only real when it cites the remaining owner already recorded on platform-job-time-as-slo. A reject is only real when it cites the remaining owner on cluster-api-uptime. Chapter 1's refusals are tested here.

Targets are not yet the SLI

Selection does not publish a window or an error budget. It decides which measurements may later be targeted. A customer **SLA (Service Level Agreement)** sentence is not an SLI and not an SLO. Chapter 3 owns targets. The decision record is the first reviewable promise about what the portfolio will and will not count.

3. Make the SLI decisions

The completed Chapter 2 model uses three files:

```
slis/method.yaml
slis/candidates.yaml
slis/decisions.yaml
```

The separation is deliberate. The method names allowed treatments, allowed classes, and forbidden justifications. Candidates are proposed measurements, not decisions. Decisions bind treatment to a Chapter 1 journey or remaining owner.

Practice — Adopt a method that cannot treat visibility as user impact

Name the three treatments and the justifications that must never accept a measurement.

Open `slis/method.yaml`. The method forbids leadership-can-see-it, dashboard-already-has-it, and copied-from-storefront. It requires class, journey or remaining owner, owner, and review trigger on every row. Allowed classes are user-journey-sli, platform-product-sli, and component-uptime. If portfolio-slo appears as a class, later chapters inherit a slogan instead of a decision.

Practice — Accept the journey proofs Chapter 1 already named

Keep `order_success_ratio` and `dispatch_success_ratio` on the accept path, and bind each to a Chapter 1 journey.

Open `slis/decisions.yaml`. The required accept decisions are:

```
- id: decide-order-success-ratio
  candidate: order_success_ratio
  treatment: accept
  class: user-journey-sli
  journey: accept-and-complete-order
  justification: inherited-outcome-matches-named-journey
  owner: reliability-program
  review_trigger: order-path-changes-good-event-definition
- id: decide-dispatch-success-ratio
  candidate: dispatch_success_ratio
  treatment: accept
  class: user-journey-sli
  journey: dispatch-fulfillment
  justification: chapter-1-later-proof-for-dispatch
  owner: reliability-program
  review_trigger: warehouse-contract-changes-good-event-definition
```

Inspect each accept row with three questions:

1. Does the journey exist in `reliability/journeys.yaml`, or was a new journey invented to justify the graph?
2. Would the measurement still matter if Northwind replaced the dashboard product?
3. Is the named owner authorized to change how the journey is measured?

`order_latency` is accepted for the same Storefront journey. It is an inherited DevOps outcome indicator, not a Platform job proof. Chapter 3 may give it a different window from success ratio. Chapter 2 only decides that users can feel latency on `accept-and-complete-order`.

Practice — Keep job-time adjacent and reject component uptime

Do not promote Platform job proofs, and do not accept CPU or replica Ready as user-journey SLIs.

The environment job-time row must be adjacent, not accepted:

```
- id: decide-time-to-first-environment
  candidate: time-to-first-environment
  treatment: adjacent
  class: platform-product-sli
  remaining_owner: platform-team
  justification: inherited-platform-job-proof
  owner: reliability-program
  review_trigger: platform-product-jobs-change
```

remaining_owner: platform-team must match the Chapter 1 refusal platform-job-time-as-slo. paved-road-completion and catalog-freshness stay adjacent for the same reason. The forbidden label is leadership-can-see-it used as an accept justification, and the forbidden class is portfolio-slo on a job-time id.

CPU utilization, replica Ready, and portal availability are rejected with remaining owner platform-team, matching cluster-api-uptime. They are theater. They are not Fulfillment's missing dispatch signal.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-02-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 02 checkpoint: accept, adjacent, and reject SLI decisions verified
```

The audit validates the three Chapter 2 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- the method names accept, adjacent, and reject;
- order_success_ratio and dispatch_success_ratio are accepted against known journeys;
- time-to-first-environment is adjacent with the job-time remaining owner;
- CPU and replica Ready are rejected;
- accept rows do not use forbidden justifications or class portfolio-slo;
- every decision names class, owner, and review trigger; and
- adjacent and reject rows name a remaining owner.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not prove that these candidates are the right set for a real company. It does not prove that order_success_ratio is the correct good-event definition under live traffic. Those are judgment claims. The review triggers exist so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 2 evidence
Mechanism evidence	Schemas and the SLI evaluator operated successfully.
Decision evidence	Accept, adjacent, and reject records trace to journeys, refusals, classes, owners, and review triggers.
Outcome evidence	Later SLO and burn chapters can be evaluated against these accepted indicators.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 2 primarily creates decision evidence. Pretending that the local checkpoint proves journeys already keep their SLOs would weaken every later chapter.

4. Test the decision under failure

Independent control failure — Time-to-first-environment is classed as a portfolio SLO because leadership can see it

Practice — Keep job-time adjacent and accept the named journey proofs

Reverse a visibility-based accept and restore `order_success_ratio` as a user-journey SLI.

The baseline fixture contains this model:

```
- id: decide-time-to-first-environment
  candidate: time-to-first-environment
  treatment: accept
  class: portfolio-slo
  journey: accept-and-complete-order
  justification: leadership-can-see-it
- id: decide-order-success-ratio
  candidate: order_success_ratio
  treatment: reject
  class: tenant-workload
  remaining_owner: storefront-team
```

The problem is not merely missing fields. The model classifies visibility as user impact and a Chapter 1 refusal as a portfolio SLO. `order_success_ratio`, the later proof Storefront actually named, is left as a tenant-workload non-metric. Dispatch is still absent.

Severity: high; every later target, burn page, and freeze conversation will optimize environment wait time instead of orders and dispatch.

Plausible harm: environments provision quickly while orders fail; Fulfillment has no SLI; CPU pages become “the reliability program.”

Potential blast radius: every service asked to “put it on the SLO dashboard”; Storefront’s provisional indicators remain the only real signal.

Bounded by: Chapter 1 refusals, later SLO catalogs, and burn-alert policy. None repairs a method that accepts whichever graph leadership can see.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** The journeys are accept-and-complete-order and dispatch-fulfillment. Time-to-first-environment is a platform job proof, not those journeys. Rejecting `order_success_ratio` removes the Storefront later proof Chapter 1 already named.
- **Error budget:** Not yet applicable as a freeze decision. Chapter-local implication: a program that targets job-time cannot later freeze Storefront releases for order burn.
- **Human system:** Not yet applicable as on-call design. Chapter-local implication: paging on job-time or CPU trains heroes to ignore dispatch.
- **Portfolio recovery:** Not yet produced. A green time-to-first-environment tile cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling justification “leadership can see it” encourages selection by dashboard: accept job-time, accept CPU, copy Storefront’s number onto Fulfillment. Those graphs may be sincere, but they do not answer which user event is good, who owns the remaining measurement, or what class the inherited Platform fixture already had.

The missing adjacent decision makes Chapter 1’s job-time refusal ornamental. The reject on `order_success_ratio` makes Chapter 3’s Storefront SLO a surprise rather than an inherited later proof. Mapping time-to-first-environment onto accept-and-complete-order invents a journey binding the brief never named. Class `portfolio-slo` on a platform-product SLI collapses two books’ evidence.

Correction

The completed model does not accept job-time as a portfolio SLO. It keeps time-to-first-environment adjacent with platform-team as remaining owner, accepts `order_success_ratio` and `dispatch_success_ratio` against the Chapter 1 journeys, rejects CPU and replica Ready, and forbids visibility labels as accept justification.

That correction changes later decisions:

- Chapter 3 must publish windows and remaining budget on accepted user-journey SLIs, not on job-time.
- Chapter 4 must freeze change against order and dispatch burn, not against time-to-first-environment.
- Chapter 5 must page on journey burn and ticket platform job-time to `platform-oncall`.
- Chapter 8 must put payment inside `order_success_ratio`, not inside cluster Ready.

The decision is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common selection errors

Treating a visible graph as a user-journey SLI

Leadership can see many true graphs. Visibility is not a good-event definition.

Leaving Platform's tenant-workload label on a user outcome

`order_success_ratio` was a non-metric *for the platform product*. It is a candidate *for the reliability program*. Do not copy the Platform exclusion into an SRE reject.

Accepting job-time “for now”

Promoting time-to-first-environment creates a target that never expires. Keep it adjacent with a remaining owner and a review trigger instead.

Copying Storefront's indicator onto Fulfillment

Dispatch is not order success. If the journey is not in `reliability/journeys.yaml`, either the brief is wrong or the measurement is out of scope. Do not mint `accept-and-complete-order` bindings for environment wait time.

Measuring selection success as dashboard coverage

A thick catalog of CPU tiles is not a thin SLI set. Later measurement must count journey good events, not accepted graphs.

Assigning class without authority

The reliability program cannot accept `order_success_ratio` if it cannot change how that ratio is computed. Record the real decision path.

6. What changed

Before	After
Time-to-first-environment was accepted as a portfolio SLO.	Job-time proofs are adjacent with the platform team as remaining owner.
<code>order_success_ratio</code> stayed a tenant-workload non-metric.	<code>order_success_ratio</code> is accepted for <code>accept-and-complete-order</code>.
Fulfillment had no user-visible indicator.	<code>dispatch_success_ratio</code> is accepted for <code>dispatch-fulfillment</code>.

Before	After
Justification meant dashboard visibility.	Justification cannot be leadership-can-see-it; class cannot be portfolio-slo.
CPU and replica Ready looked like reliability.	Component uptime is rejected with a remaining owner.
A valid schema could appear to prove a sound SLI.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable SLI contract that later chapters can target, page, freeze, and recover without promoting job-time or theater.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
SLI selection method	slis/method.yaml	It retains the treatments, classes, and forbidden justifications later chapters must not reopen as visibility.
First indicator decisions	slis/decisions.yaml	They retain which measurements the program accepts, keeps adjacent, or rejects, with owners and review triggers.

These artifacts should change when Northwind's journeys or good-event definitions materially change—not whenever a new graph is added to the dashboard.

What You Learned

A reliability program accepts measurements users can feel on named journeys. It keeps platform job-time adjacent. It rejects component uptime even when the graph is green. Visibility is not user impact. Schema checks can prove structural completeness within declared scope. They cannot prove that an indicator is objectively correct. A decision earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Decide a catalog-browse ratio without copying the dispatch row

A storefront engineer proposes `catalog_read_success_ratio` because “customers hit that page more than they order.”

Extend the Chapter 2 model without adding implementation policy yet:

1. Decide whether catalog reads are a user-visible journey event or a supporting signal of accept-and-complete-order.
2. Choose accept, adjacent, or reject without using leadership-can-see-it, dashboard-already-has-it, or copied-from-storefront.
3. Name the Chapter 1 journey or the remaining owner if the measurement is not a user-journey SLI.
4. State the good-event definition as it would exist if the ratio became an SLO.
5. Identify one observation that would falsify your treatment.
6. Explain which material change would trigger review of your decision.

Do not copy the dispatch accept and rename it. Catalog reads have different criticality, dependencies, and freeze consequences from warehouse dispatch. Your durable output is the decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 2 capability when you can explain why a visible job-time graph is not a user-journey SLI, trace every accept row to a known journey, keep a Chapter 1 job-time refusal adjacent with the recorded remaining owner, describe evidence that would falsify the treatment, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Northwind now knows which measurements may represent the journeys and which must not. There are still no portfolio targets, windows, or remaining error budgets. Fulfillment can still inherit Storefront’s 99.9 percent by copy.

Chapter 3 sets SLOs and error budgets across the portfolio so accepted SLIs become governed contracts rather than dashboard tiles.

3. Set SLOs and Error Budgets Across the Portfolio

Chapter 2 accepted the measurements users can feel. Storefront has `order_success_ratio` and `order_latency`. Fulfillment has `dispatch_success_ratio`. Job-time stays adjacent. Component uptime is rejected.

Those decisions do not answer the next production question:

What **SLO (Service Level Objective)** target, window, and error budget does each protected journey get, and how is the portfolio cataloged?

Without that catalog, Northwind still has a dashboard of nines. Storefront carries a provisional single-service objective from DevOps Chapter 6. Fulfillment has none. Leadership pastes 99.9 percent onto both services. Legal quotes a customer **SLA (Service Level Agreement)** as if it could freeze a release. `notification-service` looks like a third critical journey because email is visible. A number that was never computed cannot later stop change.

This chapter publishes windows, targets, and remaining unreliability for the Chapter 1 journeys. Chapter 4 will freeze change against those remaining budgets. It must not inherit a copied Storefront tile, an SLA sentence, or a catalog that emitted its own remaining budget.

1. An unsafe SLO catalog

A weak record says:

```
id: portfolio-nines
journey: accept-and-complete-order
sli: order_success_ratio
window: rolling-30d
target: 0.999
remaining_budget: 1.0
sla: "99.9 percent availability per customer contract"
```

It does not give `dispatch-fulfillment` a window. It treats an SLA sentence as a target. It emits remaining budget instead of computing it from observations. 99.9 percent on Storefront is not a portfolio.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 3 baseline:

```
make chapter-03-baseline
```

The command succeeds when it detects the intended unsafe catalog:

```
chapter 03 baseline: portfolio 99.9 from storefront alone correctly
detected
```

The fixture publishes 99.9 percent on the Storefront order path, stores remaining budget as a constant, quotes the customer SLA, and never gives dispatch a window or a computed budget. One service's nines have been treated as the portfolio. A legal sentence has been treated as an SLO. Remaining unreliability has been declared rather than observed. That inversion drives the chapter.

2. The production model: targets, windows, and remaining unreliability

Theory — Portfolio SLO catalog

This model enables Northwind to publish a reviewable internal contract per journey rather than a copied nine, a customer SLA, or a dashboard that approves itself.

An SLO is an internal contract, not an SLA and not an SLI

An **SLI (Service Level Indicator)** is the measurement Chapter 2 accepted. An SLO is the target that measurement must meet in a named window. An SLA is a customer or legal promise. Error budget is the remaining unreliability the SLO permits. Those four words are not synonyms.

DevOps Chapter 6 already recorded a provisional 99.5 percent order-success objective over 30 days for Storefront. That is an input. It is not a portfolio catalog. It does not cover dispatch. It cannot freeze Fulfillment. It is not an SLA.

If the catalog's target field is a sentence, later chapters inherit a brochure. If the catalog copies Storefront's target and window onto Fulfillment, later freeze policy will stop the wrong service for the wrong reason.

A window is a compliance period, not a graph zoom

A rolling 30-day window and a rolling 28-day window are different contracts. Copying rolling-30d onto dispatch because Storefront already had it is the same failure as copying 99.9 percent. The window must be named, owned, and referenced by identifier. "Last month on the dashboard" is not a window.

Northwind's Storefront success SLO keeps the inherited 30-day period and the provisional 99.5 percent target, now as a portfolio record rather than a single-service comment. Fulfillment gets its own 28-day window and a 99.0 percent target. Those numbers are teaching values. They are not industry constants. They exist so later chapters can compute remaining budget without a copied signature.

Error budget is remaining unreliability, computed from observations

If the target is 99.5 percent, the error budget is 0.5 percent of valid events in the window. Remaining budget is:

$$\text{remaining} = 1 - (\text{observed bad events} / \text{allowed bad events})$$

Allowed bad events are $\text{valid events} \times (1 - \text{target})$. Observed bad events are $\text{valid events} - \text{good events}$. The catalog must not emit `remaining_budget`. The budget register must not store a constant remaining. Both would let a dashboard approve itself.

This book's **error budget** is SRE portfolio governance. It is not Platform job-time. time-to-first-environment still has no row in this catalog.

Critical and supporting are not the same catalog row

accept-and-complete-order and dispatch-fulfillment are critical journeys. They receive SLOs. notification-service sends non-critical confirmations. Email can fail while an order still reaches a correct terminal state. Recording notification as a critical SLO would freeze Storefront releases for a confirmation template. Chapter 8 will put email in the dependency contract as non-critical. Chapter 3 must already refuse it as a protected journey SLO.

Best Practice: Give every Chapter 1 journey a window, a numeric target, an accepted SLI, and a budget whose remaining value is computed.

Production Practice: A copied target is only real when the evaluator can see the same target and window on Fulfillment as on Storefront. An SLA is only refused when a legal sentence cannot occupy the target field.

A catalog that emits remaining budget is not outcome evidence

Schema validity can prove that a row has a number. Outcome evidence for this chapter is remaining unreliability computed from named observations. Pretending the catalog's remaining_budget: 1.0 proves journeys are healthy would make Chapter 4 freeze against fiction.

3. Publish the portfolio catalog

The completed Chapter 3 model uses three files, plus named observations the evaluator consumes:

```
slos/catalog.yaml
slos/windows.yaml
slos/budgets.yaml
fixtures/observations/chapter-03.yaml
```

The separation is deliberate. The catalog names journeys, accepted SLIs, windows, numeric targets, and criticality. Windows name duration. The budget register joins each SLO to an observation. Remaining budget is computed at evaluation time. Observations are not a self-improving dashboard.

Practice — Publish a catalog that cannot treat an SLA as a target

Give each Chapter 1 journey a numeric target and record the customer SLA as out of scope.

Open `slos/catalog.yaml`. The required critical rows are:

```
- id: slo-accept-and-complete-order
  journey: accept-and-complete-order
  sli: order_success_ratio
  window: rolling-30d
  target: 0.995
  criticality: critical
  owner: reliability-program
  review_trigger: order-path-changes-good-event-definition
- id: slo-dispatch-fulfillment
  journey: dispatch-fulfillment
  sli: dispatch_success_ratio
  window: rolling-28d
  target: 0.99
  criticality: critical
  owner: reliability-program
  review_trigger: warehouse-contract-changes-good-event-definition
```

Inspect each row with three questions:

1. Is the SLI an accepted user-journey indicator from Chapter 2, or a job-time proof the method kept adjacent?
2. Would the target still be a number if the customer contract were rewritten?
3. Is Fulfillment's (target, window) different from Storefront's, or was 99.9 percent pasted twice?

`order_latency` receives its own Storefront SLO on the same journey, with a 99.0 percent good-event target in the 30-day window. Chapter 2 allowed that second contract. It is not a copy of `dispatch.notification-service` is listed under `non_critical`, not under `slos`. The customer availability SLA is recorded as `out-of-scope`. It is not a target.

Practice — Name windows so Fulfillment cannot inherit Storefront's period by copy

Keep Storefront on the inherited 30-day window and give dispatch a distinct compliance period.

Open `slos/windows.yaml`. `rolling-30d` is 30 days. `rolling-28d` is 28 days. If dispatch pointed at `rolling-30d` and also used 0.995, the evaluator would treat that as a copied Storefront SLO even though the SLI name differed.

Practice — Compute remaining budget from observations

Join each critical SLO to good and valid event counts. Do not store remaining as a constant.

Open `slos/budgets.yaml` and `fixtures/observations/chapter-03.yaml`. Each budget names an SLO and an observation. The observation names `good_events` and `valid_events`. The evaluator computes remaining unreliability. If `remaining_budget` appears on the catalog or the register, the checkpoint fails.

Storefront's teaching observation is 19,940 good events in 20,000 valid events against a 99.5 percent target: 60 bad events of 100 allowed, remaining 0.4. Fulfillment's teaching observation is 9,930 good events in 10,000 valid events against a 99.0 percent target: 70 bad events of 100 allowed, remaining 0.3. Those fixtures are local. They do not prove a production telemetry backend.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-03-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 03 checkpoint: portfolio SLOs, windows, and computed remaining
budgets verified
```

The audit validates the three Chapter 3 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- every Chapter 1 journey has a critical SLO;
- each SLO uses an accepted user-journey SLI, a named window, and a numeric target in (0, 1);
- Fulfillment's target and window are not a copy of Storefront's order-success SLO;
- job-time proofs and theater indicators are not cataloged as SLOs;
- notification-service is not a critical SLO;
- SLA text is not a target;
- remaining budget is computed from observations, not emitted by the catalog or the register; and
- every critical SLO has a budget row.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not prove that 99.5 percent or 99.0 percent is the right commercial target. It does not prove that 20,000 events represent live traffic. Those are judgment claims. The review triggers exist so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 3 evidence
Mechanism evidence	Schemas and the SLO evaluator operated successfully.
Decision evidence	Windows, numeric targets, criticality, and the SLA-out-of-scope record are explicit.
Outcome evidence	Remaining error budget is computed from named observations for each critical journey.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 3 creates decision evidence and the first computed outcome evidence. Pretending that the local checkpoint proves journeys already keep their SLOs in production would weaken every later freeze, page, and fail-over.

4. Test the design under failure

Independent control failure — Portfolio 99.9 percent from Storefront alone

Practice — Add the Fulfillment SLO and recompute remaining budget

Remove the SLA sentence, stop emitting remaining budget, and give dispatch its own window and target.

The baseline fixture contains this model:

```
- id: slo-accept-and-complete-order
  journey: accept-and-complete-order
  sli: order_success_ratio
  window: rolling-30d
  target: 0.999
  remaining_budget: 1.0
  sla: "99.9 percent availability per customer contract"
```

The problem is not merely a missing Fulfillment row. The model treats Storefront's nines as the portfolio, a legal sentence as a target, and a constant as remaining unreliability. Dispatch still has no window. Notification could be added next as another 99.9 percent tile.

Severity: high; every later freeze, page, and fail-over conversation will optimize a copied Storefront number instead of dispatch.

Plausible harm: Storefront looks healthy at 99.9 percent while warehouse dispatch has no budget; an SLA dispute is mistaken for remaining unreliability; a dashboard reports remaining 1.0 while events are uncounted.

Potential blast radius: every service asked to “take the portfolio SLO”; Fulfillment inherits Storefront's target by paste.

Bounded by: later error-budget policy, burn alerts, and dependency contracts. None repairs a catalog that has only one journey and an SLA sentence.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** accept-and-complete-order has a row. dispatch-fulfillment does not. A portfolio 99.9 percent is not a journey.
- **Error budget:** Applicable as remaining unreliability, not yet as a freeze decision. Chapter-local implication: remaining cannot be 1.0 because the catalog said so; Fulfillment has no remaining value to freeze against.
- **Human system:** Not yet applicable as on-call design. Chapter-local implication: paging on a copied nine trains heroes to ignore dispatch.
- **Portfolio recovery:** Not yet produced. A green Storefront tile cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling the catalog “99.9 percent portfolio availability” encourages copy controls: paste Storefront’s target onto Fulfillment, quote the customer contract, set remaining to 1.0 so the slide is green. Those moves may be sincere, but they do not answer which events are valid, how many are allowed to fail, or whether dispatch has a contract at all.

The missing Fulfillment window makes Chapter 4’s freeze ornamental for warehouse change. The SLA field makes a legal product look like an internal target. The emitted remaining budget makes outcome evidence a constant. Mapping notification into the same 99.9 percent would freeze the order path for email.

Correction

The completed model does not publish a portfolio nine. It gives Storefront a 99.5 percent / 30-day success SLO and a separate latency SLO, gives Fulfillment a 99.0 percent / 28-day dispatch SLO, records notification-service as non-critical, records the customer SLA as out of scope, and computes remaining budget from observations.

That correction changes later decisions:

- Chapter 4 must freeze change against remaining journey budget, not against an SLA sentence or a copied 99.9 percent.
- Chapter 5 must page on burn of these windows, not on a single Storefront tile.
- Chapter 8 must put payment inside the Storefront success SLO and warehouse inside dispatch, not inside notification.
- Chapter 9 must account degraded success as burn against these remaining budgets.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common catalog errors

Copying Storefront's target onto Fulfillment

Two services can share a percentage by coincidence. They cannot share a (target, window) signature because someone pasted the row. Dispatch is not order success.

Treating an SLA sentence as an SLO target

A customer contract can motivate a conversation. It cannot occupy the target field. Error budget cannot freeze a legal clause.

Emitting remaining budget from the catalog

A constant remaining of 1.0 is a slide. Remaining unreliability is $1 - (\text{observed bad} / \text{allowed bad})$ from named events.

Cataloging job-time as a portfolio SLO

Chapter 2 already kept time-to-first-environment adjacent. Chapter 3 must not reopen it as a target.

Making notification-service critical because email is visible

Visibility is the Chapter 2 failure again. Confirmation mail is not accept-and-complete-order.

Measuring catalog success as coverage of every graph

A thick catalog of CPU SLOs is not a thin portfolio. Later freeze policy will stop the wrong change.

6. What changed

Before	After
The portfolio was Storefront's 99.9 percent.	Each Chapter 1 journey has its own window, numeric target, and computed remaining budget.
Fulfillment had no SLO.	dispatch-fulfillment has a 99.0 percent target over 28 days, not a copied Storefront signature.
An SLA sentence sat in the target field.	The customer SLA is out of scope; targets are numbers in (0, 1).
Remaining budget was a constant.	Remaining unreliability is computed from good and valid events.
Notification looked like a third critical journey.	notification-service is recorded as non-critical.
A valid schema could appear to prove a sound SLO.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable portfolio contract that later chapters can freeze, page, and recover without a copied nine or a legal sentence.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Portfolio SLO catalog	slos/catalog.yaml	It retains journey, accepted SLI, window, numeric target, and criticality later policy must not replace with an SLA or a copied nine.
Error-budget register	slos/budgets.yaml	It retains the join from SLO to observation so remaining unreliability stays computed.

These artifacts should change when Northwind's journeys, good-event definitions, or commercial risk materially change—not whenever a dashboard theme is redesigned.

What You Learned

A portfolio SLO catalog gives each protected journey a numeric target, a named window, and remaining unreliability computed from observations. An SLA is not that catalog. A copied Storefront nine is not a Fulfillment SLO. notification-service is not a critical journey. Schema checks can prove structural completeness within declared scope. They cannot prove that 99.5 percent is the right commercial target. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Set a catalog-browse SLO without copying the dispatch row

A storefront engineer proposes a 99.0 percent / 28-day SLO for `catalog_read_success_ratio` “so it matches Fulfillment.”

Extend the Chapter 3 model without adding freeze policy yet:

1. Decide whether catalog reads are a critical journey SLO, a supporting SLO of accept-and-complete-order, or out of scope.
2. If you publish a target, choose a window that is not a paste of rolling-28d plus 0.99 unless you can defend that signature on its own facts.
3. State the good and valid event definition as it would exist in an observation file.
4. Compute remaining budget from one teaching observation; do not emit `remaining_budget`.
5. Identify one observation that would falsify your target or criticality.
6. Explain which material change would trigger review of your SLO.

Do not copy the dispatch row and rename it. Catalog reads have different criticality, dependencies, and freeze consequences from warehouse dispatch. Your durable output is the contract and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 3 capability when you can explain why a copied 99.9 percent is not a portfolio, trace every critical SLO to a Chapter 1 journey and a Chapter 2 accepted SLI, compute remaining budget from observations rather than from a catalog constant, refuse an SLA sentence as a target, keep notification-service non-critical, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Northwind now has windows, targets, and remaining error budgets for Storefront and Fulfillment. Those numbers still cannot stop a release, a fleet upgrade, or an exception. Exhausted budget is still a report.

Chapter 4 governs change with error-budget policy so remaining unreliability can continue, slow, or freeze work—including fleet change—without relabeling a Platform upgrade freeze.

4. Govern Change with Error-Budget Policy

Chapter 3 published windows, numeric targets, and remaining unreliability. Storefront and Fulfillment now have **SLOs (Service Level Objectives)**. Remaining budget is computed from observations. Those numbers still cannot stop a digest promotion, a fleet step, or an exception that never expires.

The production question is now:

When remaining **error budget** is healthy, slowing, or exhausted, what change is authorized, slowed, or frozen?

Without that policy, the catalog is theater. Releases ship on the calendar. Platform already knows how to freeze fleet upgrade storage-1-0-to-2-0 for an upgrade. Fulfillment's cohort is still pending. Storefront's order-success budget can sit at zero while that step proceeds because "the freeze window was for upgrades." Freezing everything on first burn is the opposite failure: a change freeze with no bands, no owner, and no expiry.

This chapter maps remaining budget to continue, slow, and freeze. It may halt the same fleet step Platform knows how to freeze, for a reliability reason Platform did not own. It must not copy freeze window, cohort, or rollback, and must not relabel a Platform upgrade freeze as this policy.

1. An unsafe policy: exhausted budget, fleet still proceeds

A weak record says:

```
target: storage-1-0-to-2-0
action: continue
freeze_reason: platform-upgrade-freeze
freeze:
  start: 2026-08-16T00:00:00Z
  end: 2026-08-23T00:00:00Z
rollback: "1.0"
```

It notices Storefront is red. It still lets the fleet step run because the inherited freeze was scheduled for the contract bump. Copying freeze, cohorts, and rollback onto this policy pretends Platform already answered the reliability question. Platform answered a different one.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 4 baseline:

```
make chapter-04-baseline
```

The command succeeds when it detects the intended unsafe policy:

```
chapter 04 baseline: unfrozen fleet under exhausted storefront budget
correctly detected
```

The fixture computes Storefront remaining at zero, may freeze Storefront releases, and still continues storage-1-0-to-2-0 with reason platform-upgrade-freeze and copied upgrade fields. Exhausted budget has been treated as a dashboard. A Platform upgrade freeze has been treated as an error-budget freeze. This is the book's cumulative reliability failure: availability theater plus informal heroics, now wearing a fleet calendar. That residual drives the chapter.

2. The production model: bands, freeze by reference, exceptions that expire

Theory — Error-budget policy

This model enables Northwind to continue, slow, or freeze change from remaining unreliability rather than from a calendar, a ticket, or a Platform upgrade window.

Remaining budget selects an action; it does not emit one

Chapter 3 already defined remaining as $1 - (\text{observed bad} / \text{allowed bad})$. Policy does not store remaining as a constant. It names bands:

Remaining	Action	Meaning
≥ 0.5	continue	Ordinary change proceeds
≥ 0.1 and < 0.5	slow	Change proceeds with extra review and reduced rate
< 0.1	freeze	Named change stops until budget recovers or an exception expires

Those thresholds are teaching values. They are not industry constants. Remaining below zero is still freeze. A band of “freeze everything” with no target is not a policy. A band of “continue until someone pages” is not a policy.

Best Practice: Bind every action to a Chapter 3 SLO or to an explicit non-critical exception. A freeze that cannot name the journey is a holiday freeze.

Release freeze and fleet freeze are different change kinds

Exhausted Storefront order-success budget freezes Storefront releases. The inherited DevOps promotion identity is `artifact_digest`. This chapter references that identity. It does not redesign digest promotion.

The same exhausted budget may freeze fleet upgrade `storage-1-0-to-2-0` while Fulfillment's cohort is still pending. Storefront and Fulfillment share the storage contract. A fleet step that proceeds under zero Storefront budget spends reliability the catalog already refused. Fulfillment still having remaining budget does not authorize that step. Slow Fulfillment releases. Freeze the shared fleet change.

This freeze **references** the inherited upgrade id. It adds freeze reason, remaining-budget source, owner, and expiry. It must not copy Platform freeze window, cohort list, rollback, last known good, or from/to versions. Those fields already live on the Platform record. Copying them here collapses two freeze reasons into one YAML document.

Production Practice: A Platform upgrade freeze is not an error-budget freeze. An error-budget freeze may halt that same fleet step for a reason Platform did not own. `freeze_reason: platform-upgrade-freeze` is the collapse. `freeze_reason: exhausted-storefront-error-budget` with `owner: reliability-program` is this book's act.

Exceptions expire and name remaining journey risk

A freeze without exceptions recreates a change freeze. An exception without owner, scope, expiry, and remaining journey risk recreates a ticket. The exception is not a second SLO. It is a time-bounded permission to change a named scope while the freeze remains in force elsewhere.

If the scope is `storage-1-0-to-2-0`, the remaining journey risk must be explicit: which Chapter 1 journey still burns if the step proceeds. If the exception has no `expires_at`, it is a permanent unfreeze. If it has no `removal_path`, nobody knows how it ends.

`notification-service` is still non-critical. Confirmation templates may continue under a Storefront freeze. That continue is a policy decision, not proof that orders succeeded.

Expiry is part of the freeze

A freeze that never expires becomes folklore. Record `expires_at` on freeze actions. Review when remaining budget recovers or the window elapses. Chapter 5 will page on burn. Chapter 4 must already stop the digest and the fleet step without waiting for a page.

3. Write the policy

The completed Chapter 4 model uses three files, plus Chapter 4 observations the evaluator consumes:

```
policy/error-budget.yaml
policy/actions.yaml
policy/exceptions.yaml
fixtures/observations/chapter-04.yaml
```

The separation is deliberate. The policy names bands. Actions bind `continue`, `slow`, and `freeze` to change kinds. Exceptions are the only legal unfreeze. Remaining budget is computed from Chapter 4 observations against the Chapter 3 catalog. The policy files must not emit `remaining` as a constant.

Practice — Name bands that cannot freeze everything or freeze nothing

Require `continue`, `slow`, and `freeze`, each with a remaining threshold.

Open `policy/error-budget.yaml`. `Continue` starts at remaining 0.5. `Slow` starts at 0.1. `Freeze` is everything below that, including zero and negative remaining. If `freeze` is missing, exhausted budget cannot stop change. If `continue` is missing, every blip is a freeze.

Practice — Freeze Storefront releases and the shared fleet step by reference

Halt digest promotion and `storage-1-0-to-2-0` for exhausted Storefront budget without copying Platform upgrade fields.

Open `policy/actions.yaml`. The required freeze rows are:

```
- id: freeze-storefront-releases
  slo: slo-accept-and-complete-order
  action: freeze
  change_kind: release
  target: storefront-releases
  promotion_identity: artifact_digest
  owner: reliability-program
  expires_at: 2026-08-23T00:00:00Z
  review_trigger: remaining-budget-recovers-or-window-elapses
- id: freeze-fleet-storage-upgrade
  slo: slo-accept-and-complete-order
  action: freeze
  change_kind: fleet
  target: storage-1-0-to-2-0
  freeze_reason: exhausted-storefront-error-budget
  owner: reliability-program
  expires_at: 2026-08-23T00:00:00Z
  review_trigger: remaining-budget-recovers-or-fulfillment-cohort-replanned
```

Inspect the fleet row with three questions:

1. Is target the inherited upgrade id, or a renamed local copy?
2. Does the row contain freeze, cohorts, rollback, or last_known_good? If it does, this policy has absorbed Platform Chapter 12.
3. Is freeze_reason an SRE-owned remaining-budget reason, or platform-upgrade-freeze?

Fulfillment releases are slowed, not frozen: dispatch remaining is still 0.3. Notification templates continue: they are not a critical journey. Order-latency remaining is still healthy; it does not unfreeze Storefront releases. Exhausted order success is enough.

Practice — Record an exception that expires

Name owner, scope, remaining journey risk, expiry, and removal path. Do not unfreeze the fleet step.

Open `policy/exceptions.yaml`. A valid exception can cover a non-critical confirmation template. It cannot omit `expires_at`. It cannot take `storage-1-0-to-2-0` as scope without stating which journey still burns.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-04-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 04 checkpoint: continue, slow, and freeze policy including SRE-
owned fleet freeze verified
```

The audit validates the three Chapter 4 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- remaining budget is computed from Chapter 4 observations, not emitted by the policy;
- Storefront order-success remaining is at or below the freeze band;
- Storefront releases are frozen against inherited artifact_digest;
- fleet upgrade storage-1-0-to-2-0 is frozen with an SRE-owned reason;
- the fleet freeze does not copy Platform freeze window, cohort, or rollback fields;
- freeze_reason is not platform-upgrade-freeze;
- Fulfillment releases are slowed while dispatch still has remaining budget;
- continue, slow, and freeze are all used; and
- every exception names owner, scope, remaining journey risk, expiry, and removal path.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not halt a live release or a live fleet. It does not prove that 0.5 / 0.1 are the right commercial bands. Those are judgment claims. The review triggers exist so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 4 evidence
Mechanism evidence	Schemas and the policy evaluator operated successfully.
Decision evidence	Bands, freeze targets, SRE-owned fleet reason, and expiring exceptions are explicit.
Outcome evidence	Remaining budget computed from observations selects freeze rather than continue for Storefront and the shared fleet step.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 4 creates decision evidence and uses Chapter 3's computation as outcome input. Pretending that the local checkpoint froze a production cluster would weaken every later page, incident, and fail-over.

4. Test the design under failure

Cumulative reliability failure — Storefront budget exhausted, fleet upgrade still proceeds

Practice — Apply the error-budget freeze and halt the fleet step

Keep Storefront releases frozen, freeze storage-1-0-to-2-0 by reference, and drop copied Platform upgrade fields.

The baseline fixture contains this model:

```
- id: proceed-fleet-storage-upgrade
  slo: slo-accept-and-complete-order
  action: continue
  change_kind: fleet
  target: storage-1-0-to-2-0
  freeze_reason: platform-upgrade-freeze
  freeze:
    start: 2026-08-16T00:00:00Z
    end: 2026-08-23T00:00:00Z
  rollback: "1.0"
```

The problem is not merely a missing freeze row. The model treats a Platform upgrade calendar as the reliability decision. Storefront remaining is zero. Fulfillment's cohort is still pending. The shared storage step proceeds. Copied freeze fields make the two reasons look like one record.

Severity: high; exhausted journey budget cannot stop the change that still couples Storefront and Fulfillment.

Plausible harm: Storefront releases pause while the fleet step still moves storage; Fulfillment absorbs 2.0 under a Storefront outage; heroes ship around the dashboard.

Potential blast radius: every tenant on tenant-storage; Storefront order path and the pending Fulfillment cohort.

Bounded by: later burn pages, on-call, and incident freeze joins. None repairs a policy that continues fleet change because the upgrade freeze was "already on the calendar."

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** accept-and-complete-order is at remaining zero. dispatch-fulfillment still has remaining budget. The fleet step is shared; it is not a Fulfillment-only change.
- **Error budget:** Applicable as a freeze decision. Remaining Storefront unreliability is exhausted. Continuing storage-1-0-to-2-0 spends that budget. A Platform upgrade freeze does not compute remaining.
- **Human system:** Not yet applicable as on-call design. Chapter-local implication: an unfrozen fleet step under a red Storefront tile trains heroes to “just finish the upgrade.”
- **Portfolio recovery:** Not yet produced. Completing a fleet cohort cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling the continue “the freeze window is for upgrades” encourages calendar controls: keep Platform’s dates, copy rollback to 1.0, mark the reliability program aligned. Those fields are true for Platform Chapter 12. They do not answer whether Storefront may still change, who owns a reliability freeze, or when that freeze expires.

The missing SRE freeze reason makes Chapter 3’s remaining budget ornamental. The copied upgrade fields make Chapter 12’s freeze look like this policy. An exception without expiry would finish the collapse: a permanent unfreeze by ticket.

Correction

The completed model does not continue the fleet step. It freezes Storefront releases against artifact_digest, freezes storage-1-0-to-2-0 with reason exhausted-storefront-error-budget and owner reliability-program, slows Fulfillment releases, lets non-critical notification continue, and records an exception that expires. Platform freeze window, cohort, and rollback stay on the inherited record.

That correction changes later decisions:

- Chapter 5 must page on journey burn that this policy already froze, not on CPU that cannot freeze a fleet.
- Chapter 6 must page a rotation that can act on this freeze, not a catalog label.
- Chapter 10 must join incident close to whether this freeze still applies.
- Chapter 13 must exercise an error-budget freeze, not only a Platform upgrade freeze.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common policy errors

Relabeling a Platform upgrade freeze as error-budget policy

The same upgrade id can be frozen twice for two reasons. Copying the reason collapses them.

Freezing Storefront releases but letting the shared fleet step run

Fulfillment's remaining budget does not authorize a storage bump Storefront cannot currently survive.

Copying freeze window, cohort, or rollback into this policy

Those fields are Platform's. Reference the upgrade id. Add reason, owner, and expiry.

Freezing everything on first burn

Without a slow band, every dip is a holiday freeze. Heroes will file exceptions that never expire.

Exceptions without expiry

A freeze with a permanent ticket is not a freeze. Record `expires_at` and `removal_path`.

Emitting remaining budget on the policy

Chapter 3 already refused catalog self-approval. Policy self-approval is the same failure.

6. What changed

Before	After
Storefront remaining was zero and the fleet step still proceeded.	storage-1-0-to-2-0 is frozen for exhausted Storefront error budget.
The freeze reason was platform-upgrade-freeze.	The freeze reason is SRE-owned remaining-budget exhaustion.
Platform freeze window, cohort, and rollback were copied here.	The policy references the upgrade id and leaves those fields on the inherited record.
Bands were "ship unless someone yells."	Continue, slow, and freeze bind remaining unreliability to named change kinds.
Exceptions were tickets.	Exceptions name owner, scope, remaining journey risk, expiry, and removal path.
A valid schema could appear to prove a freeze.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable change-control contract that later chapters can page, command, and exercise without relabeling a Platform upgrade freeze.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Error-budget policy	policy/error-budget.yaml	It retains the continue / slow / freeze bands later pages and incidents must not replace with a calendar.
Freeze-and-exception contract	policy/actions.yaml and policy/exceptions.yaml	They retain which releases and which fleet step freeze, by reference, with expiry.

These artifacts should change when Northwind's journeys, remaining-budget bands, or fleet upgrade identity materially change—not whenever a change-calendar theme is restyled.

What You Learned

Error-budget policy maps remaining unreliability to continue, slow, or freeze. Exhausted Storefront budget freezes Storefront releases and may freeze a shared fleet step Platform already knows how to freeze for upgrades. That freeze references the inherited upgrade id. It does not copy Platform freeze window, cohort, or rollback, and it does not relabel an upgrade freeze as this policy. Schema checks can prove structural completeness within declared scope. They cannot halt a live fleet. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Exception a catalog experiment without copying the fleet freeze

A storefront engineer wants to ship a catalog-browse experiment while order-success remaining is zero “because Fulfillment’s cohort is pending anyway.”

Extend the Chapter 4 model without adding paging policy yet:

1. Decide whether the experiment is a Storefront release (frozen), a non-critical continue, or an exception.
2. If you grant an exception, name remaining journey risk that is not “the fleet freeze window is still open.”
3. Choose a scope that is not a paste of storage-1-0-to-2-0 unless the experiment actually mutates that upgrade.
4. Set `expires_at` and a `removal_path`.
5. Identify one observation that would falsify the exception—for example remaining still zero and catalog reads now in the order path.
6. Explain which material change would trigger review of the exception, not just the experiment ticket.

Do not copy the fleet freeze row and rename it. Catalog reads have different freeze consequences from a shared storage upgrade. Your durable output is the policy decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 4 capability when you can explain why a Platform upgrade freeze is not an error-budget freeze, freeze Storefront releases and storage-1-0-to-2-0 from computed remaining budget, refuse copied upgrade fields, keep Fulfillment slowed rather than frozen while dispatch still has budget, require exceptions to expire, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Northwind can now stop a digest and a fleet step when remaining budget is exhausted. Humans are still woken by every red graph. CPU pages. Replica Ready pages. Order-success burn is a dashboard panel.

Chapter 5 pages on burn rate rather than on every symptom, so this policy has a signal that matches the journey rather than a hero roster chasing tiles.

5. Page on Burn Rate, Not on Every Symptom

Chapter 4 can freeze a digest and a fleet step when remaining budget is exhausted. Humans are still woken by every red graph. CPU pages. Replica Ready pages. Portal errors page.

order_success_ratio burn is a dashboard panel. Inherited DevOps Chapter 6 already defined a provisional fast and slow burn for Storefront. That rule is a single-service interface. It is not a portfolio page policy.

The production question is now:

Which burn rates page a human, which create tickets, and which symptoms must not wake anyone?

Without that map, symptom pages train people to ignore journeys. Missing burn pages leave exhausted budget silent until a customer complains. Platform job-time burn pages Storefront because leadership can see time-to-first-environment. Chapter 2 already kept that proof adjacent. Chapter 5 must not page it as if it were accept-and-complete-order.

This chapter defines multi-window burn alerts that page on user-journey burn and ticket the rest. It does not reteach logs, metrics, or traces. Catalog destinations storefront-oncall, fulfillment-oncall, and platform-oncall are contacts. Chapter 6 will turn them into an on-call system. A page that emits user_impact: true is not evidence that a user was harmed.

1. An unsafe page map

A weak record says:

```
- id: page-cpu
  burn: cpu-utilization
  destination: storefront-oncall
  user_impact: true
- id: panel-order-success
  burn: order_success_ratio
  disposition: dashboard
```

It pages a component symptom at a catalog contact and treats journey burn as a tile. “The CPU is high” may justify a ticket. It cannot complete a page. user_impact: true written by the page is a slogan.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 5 baseline:

```
make chapter-05-baseline
```

The command succeeds when it detects the intended unsafe mapping:

```
chapter 05 baseline: cpu paging storefront while order burn is a panel
correctly detected
```

The fixture pages `cpu-utilization` at `storefront-ocall`, stamps `user_impact: true`, leaves `order_success_ratio` as a dashboard panel, and may page job-time at Storefront. Symptom noise has been treated as user impact. Inherited burn windows have been ignored. Exhausted Chapter 4 budget can freeze a fleet while nobody is paged for the journey.

That inversion drives the chapter.

2. The production model: paired windows, page versus ticket, minimum volume

Theory — Burn-alert disposition

This model enables Northwind to wake a human for user-journey burn and to ticket or record the rest, without reteaching observability fundamentals.

A burn alert is a paired window, not a red tile

DevOps Chapter 6 already recorded fast burn (5m and 1h, threshold 14.4) and slow burn (30m and 6h, threshold 6) for `order_success_ratio`. Those numbers are teaching values. This chapter **consumes** them. It does not rewrite them into a new observability stack, and it does not pretend they are industry constants.

Fast burn catches urgent consumption of error budget. Slow burn catches sustained consumption that a short spike would miss. A single CPU threshold is not a paired window. A dashboard panel is not a burn alert.

Fulfillment reuses the same inherited window pairs on `dispatch_success_ratio`. That is extending the interface to a second journey. It is not copying Storefront's **SLO (Service Level Objective)** target. Chapter 3 already forbade that copy.

Page, ticket, and record are different dispositions

Disposition	When	Destination
page	Fast or slow burn of an accepted user-journey SLI (Service Level Indicator)	Catalog contact for that journey
ticket	Adjacent job-time burn, or a component symptom that still needs an owner	platform-ocall for job-time and theater
record	A graph that must not wake anyone and need not open a ticket	No page, no ticket

`order_success_ratio` pages `storefront-ocall`. `dispatch_success_ratio` pages `fulfillment-ocall`. `time-to-first-environment` tickets `platform-ocall` and must not page Storefront. CPU and replica Ready do not page. Replica Ready may ticket. CPU may record.

Best Practice: Require a minimum evidence volume on every page. One failed request is not a burn.

Production Practice: A page is only real when it names a Chapter 2 accepted user-journey SLI, a paired window, a catalog contact, and a volume floor. A page that emits `user_impact` has forged outcome evidence.

Inherited burn is an interface, not a rewrite

Do not replace `5m / 1h / 14.4` with a vendor-specific “anomaly score” and call it maturity. Do not drop slow burn because fast burn already pages. Do not point Storefront’s inherited windows at `time-to-first-environment` and call that portfolio coverage.

Job-time proofs remain a **job-time budget**. Their alerts are tickets to the platform contact. They are not pages on the order path.

A catalog contact is not yet an on-call system

`storefront-oncall` is who the catalog says to call. It is not a rotation, a load limit, a handoff, or authority. Chapter 6 will bind these pages to living primaries. Chapter 5 must already refuse to page CPU at that contact. Otherwise Chapter 6 inherits a hero roster whose first job is silencing symptoms.

3. Define the page-versus-ticket map

The completed Chapter 5 model uses three files:

```
alerting/burns.yaml
alerting/pages.yaml
alerting/tickets.yaml
```

The separation is deliberate. Burns name SLI, window pair, class, volume floor, and disposition. Pages bind journey burns to catalog contacts. Tickets bind job-time and remaining symptoms. Pages must not carry a `user_impact` field.

Practice — Consume the inherited fast and slow windows

Keep `5m/1h/14.4` and `30m/6h/6` on user-journey burns. Do not rewrite them.

Open `alerting/burns.yaml`. The method records those pairs once and applies them to `order_success_ratio` and `dispatch_success_ratio`. `minimum_evidence_volume` is 50 valid events. That number is a teaching floor. It exists so a single error cannot page.

User-journey burns have `disposition: page`. Job-time has `disposition: ticket`. CPU has `disposition: record`. Replica Ready has `disposition: ticket`.

Practice — Page journey burn at the matching catalog contact

Fast and slow order_success_ratio page storefront-oncall. Fast and slow dispatch_success_ratio page fulfillment-oncall.

Open alerting/pages.yaml. Each page names a burn id, a destination contact from the inherited catalog, and destination_kind: catalog-contact. If destination_kind is on-call-system, Chapter 6 has been skipped. If user_impact appears, the page has approved itself.

Inspect each page with three questions:

1. Is the SLI an accepted user-journey indicator from Chapter 2?
2. Would this still page if the CPU graph were deleted?
3. Is the destination a catalog contact for that journey, or whoever answered last time?

Practice — Ticket job-time and symptoms; do not page Storefront for them

time-to-first-environment tickets platform-oncall. CPU does not page. Replica Ready does not page.

Open alerting/tickets.yaml. The job-time ticket destination is platform-oncall. A row that pages Storefront for environment wait time reopens Chapter 2. Replica Ready may open a platform ticket. CPU is recorded, not paged.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-05-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 05 checkpoint: journey burn pages and symptom tickets verified
```

The audit validates the three Chapter 5 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- fast and slow burns of order_success_ratio page storefront-oncall;
- fast and slow burns of dispatch_success_ratio page fulfillment-oncall;
- inherited window pairs are consumed, not rewritten;
- CPU and replica Ready do not page;
- time-to-first-environment tickets platform-oncall and does not page Storefront;
- every page names destination_kind: catalog-contact and has no user_impact field;
- minimum evidence volume is greater than one; and
- page targets are accepted user-journey SLIs.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not send a real page. It does not prove that 14.4 or 50 events are the right commercial thresholds. Those are judgment claims. The review triggers exist so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 5 evidence
Mechanism evidence	Schemas and the alerting evaluator operated successfully.
Decision evidence	Page versus ticket versus record, catalog contacts, and inherited windows are explicit.
Outcome evidence	Not yet produced as live user impact; a page must not emit user_impact.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 5 creates decision evidence about who is woken. Pretending that the local checkpoint proved a customer felt the burn would weaken every later on-call, incident, and fail-over.

4. Test the design under failure

Independent control failure — CPU pages Storefront while order burn is a panel

Practice — Invert the mapping

Page journey burn; ticket or record component symptoms; drop forged user_impact.

The baseline fixture contains this model:

```
pages:
- id: page-cpu
  burn: cpu-utilization
  destination: storefront-oncall
  user_impact: true
burns:
- id: panel-order-success
  sli: order_success_ratio
  disposition: dashboard
```

The problem is not merely a missing dispatch page. The model treats a symptom as the reason to wake Storefront and treats the Chapter 2 accepted SLI as decoration. Forged user_impact makes the page look like outcome evidence. Job-time can follow the same path next.

Severity: high; the humans who must act on Chapter 4's freeze are woken by CPU and sleep through order burn.

Plausible harm: Storefront silences CPU while orders fail; Fulfillment is never paged; platform wait time pages the wrong contact.

Potential blast radius: every catalog contact asked to “just add the graph”; inherited burn windows unused.

Bounded by: later on-call systems, incident command, and dependency contracts. None repairs a map that pages symptoms and panels journeys.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** accept-and-complete-order is the journey CPU does not measure. dispatch-fulfillment is still silent if only Storefront CPU pages.
- **Error budget:** Applicable as the thing burn consumes. Chapter-local implication: a CPU page cannot freeze Storefront releases; an unpaged order_success_ratio burn can sit at the Chapter 4 freeze band with no human.
- **Human system:** Not yet applicable as rotation design. Chapter-local implication: paging CPU at storefront-oncall trains whoever answers that contact to hunt nodes instead of orders.
- **Portfolio recovery:** Not yet produced. A silenced CPU page cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling the page “CPU is user-visible” encourages symptom controls: scrape kubelet, count Restarts, stamp user_impact: true so the ticket looks serious. Those graphs may be true. They do not name a good-event ratio, a paired window, or a minimum volume. Leaving order_success_ratio on a panel makes Chapter 3’s SLO and Chapter 4’s freeze ornamental at 2 a.m.

Paging time-to-first-environment at Storefront would finish the collapse: Chapter 2’s adjacent decision undone by the loudest tile.

Correction

The completed model does not page CPU. It pages fast and slow burns of order_success_ratio and dispatch_success_ratio at the matching catalog contacts, tickets job-time to platform-oncall, tickets replica Ready, records CPU, consumes inherited window pairs, requires a volume floor, and forbids user_impact on the page.

That correction changes later decisions:

- Chapter 6 must bind these pages to rotations, not to a new CPU roster.
- Chapter 8 must page payment burn as Storefront journey burn, not as a provider tile equal to email.
- Chapter 9 must not page Fulfillment for Storefront payment slowness.
- Chapter 10 must not close an incident because CPU recovered.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common alerting errors

Paging the loudest graph

CPU, replica Ready, and portal errors are easy to scrape. They are not user-journey burn.

Leaving inherited burn on a dashboard

DevOps Chapter 6 already named the windows. A panel is not a page.

Paging Storefront for job-time

Time-to-first-environment is adjacent. Ticket platform-oncall.

Forging user impact on the page

Outcome evidence is computed remaining budget and later recovery. A boolean on the alert is not that evidence.

Paging on one event

Minimum volume exists so a single error cannot wake the contact. Choose a floor and review it.

Treating the catalog contact as the on-call system

Chapter 5 may destination a contact. Chapter 6 must still add rotation, load, and handoff. Do not skip that by naming the contact on-call-system here.

6. What changed

Before	After
CPU paged storefront-oncall.	CPU is recorded; it does not page.
order_success_ratio burn was a dashboard panel.	Fast and slow order-success burns page storefront-oncall.
Dispatch had no page.	Fast and slow dispatch burns page fulfillment-oncall.
Job-time could page Storefront.	time-to-first-environment tickets platform-oncall.
Pages stamped user_impact: true.	Pages cannot emit user impact.
Inherited windows were ignored or rewritten.	5m/1h/14.4 and 30m/6h/6 are consumed as interface.
A valid schema could appear to prove a sound page.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable page-versus-ticket contract that later on-call, incidents, and game days can bind without waking people for theater.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Burn-alert contract	alerting/burns.yaml	It retains paired windows, dispositions, and the volume floor later pages must not replace with a symptom threshold.
Page-versus-ticket map	alerting/pages.yaml and alerting/tickets.yaml	They retain which burns wake a catalog contact and which must not.

These artifacts should change when Northwind's journeys, accepted SLIs, or inherited window policy materially change—not whenever a new graph is added to the dashboard.

What You Learned

A reliability program pages on user-journey burn and tickets or records the rest. Inherited DevOps burn windows are an interface, not a rewrite. CPU and replica Ready must not page. Job-time tickets the platform contact. A page cannot emit user impact. Catalog contacts are destinations, not yet an on-call system. Schema checks can prove structural completeness within declared scope. They cannot send a real page. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Alert catalog-browse burn without copying the dispatch page

A storefront engineer wants a fast-burn page for `catalog_read_success_ratio` at `storefront-oncall` “because customers hit that page more than they order.”

Extend the Chapter 5 model without adding rotation policy yet:

1. Decide whether catalog-browse burn pages, tickets, or records, given Chapter 2 and Chapter 3 treatments of catalog reads.
2. If you page, consume the inherited window pairs rather than inventing a one-window CPU-style threshold.
3. Name the catalog contact and keep `destination_kind: catalog-contact`.
4. Set a minimum evidence volume that is not 1.
5. Identify one observation that would falsify the disposition—for example CPU still pages while this burn is a panel.
6. Explain which material change would trigger review of the alert, not just the dashboard widget.

Do not copy the dispatch page and rename it. Catalog reads have different criticality and freeze consequences from warehouse dispatch. Your durable output is the disposition and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 5 capability when you can explain why a CPU page is not journey burn, trace every page to an accepted user-journey SLI and a catalog contact, ticket job-time to platform-oncall, refuse user_impact on the page, consume inherited window pairs, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Pages now have a destination in the catalog. On-call is still whoever answered Slack. storefront-oncall is a label. It has no rotation, load, handoff, or authority.

Chapter 6 designs on-call as a system so these pages bind to a living primary rather than to a contact treated as the roster.

6. Design On-Call as a System

Chapter 5 pages journey burn at catalog contacts. `storefront-oncall`, `fulfillment-oncall`, and `platform-oncall` are who the catalog says to call. They are not a rotation, a load limit, a handoff, or authority. On-call is still whoever answered Slack.

The production question is now:

Who is paged, with what load, escalation, handoff, and authority, and how is that different from a catalog contact?

Without that system, a contact is a hero roster. A rotation without load, handoff, or authority is Slack with a nicer name. Platform job-time tickets land on Storefront because “they are already up.” Inherited DevSecOps `self_approval_forbidden` already forbids a responder from approving their own break-glass. An on-call label that ignores that field is informal heroics wearing a catalog id.

This chapter designs on-call systems for Storefront, Fulfillment, and the platform product. It **references** catalog escalation ids. It adds rotation, load, handoff, and authority. It binds Chapter 5 pages to a living primary. It must not treat the contact label as the system.

1. An unsafe on-call label

A weak record says:

```
id: storefront-oncall
primary: slack
handoff: none
```

It copies the catalog contact as the system id. It pages chat history. It has no secondary, no load limit, no recorded handoff, and no authority that consumes `self_approval_forbidden`. “Someone is in Slack” may describe last night. It cannot complete an on-call system.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 6 baseline:

```
make chapter-06-baseline
```

The command succeeds when it detects the intended unsafe design:

```
chapter 06 baseline: slack-as-primary without rotation correctly detected
```

The fixture treats `storefront-oncall` as the system, sets primary to Slack, omits handoff, lets platform destinations land on Storefront, and leaves `self_approval_forbidden` false. Pages still go to chat history. This is the book’s cumulative reliability failure continuing under a friendlier name: availability theater plus informal heroics, now wearing an escalation label.

That residual drives the chapter.

2. The production model: contact, rotation, load, handoff, authority

Theory — On-call as a system

This model enables Northwind to bind Chapter 5 pages to a living primary with load, handoff, and attributable authority rather than to a catalog label or a Slack channel.

A catalog contact is an input, not the system

The inherited Platform catalog already names `storefront-oncall`, `fulfillment-oncall`, and `platform-oncall`. This chapter must not replace those ids and must not reuse them as the system id. A system **references** the contact. If `id: storefront-oncall`, the contact has been treated as the roster.

Three systems exist because three destinations exist. Storefront pages bind to the Storefront system. Fulfillment pages bind to the Fulfillment system. Platform tickets bind to the platform system. A platform destination that lands on Storefront's rotation is the unofficial path Chapter 5 already refused for job-time pages.

A rotation has a living primary and a secondary

A living primary is a named responder who is current as of a recorded time. `slack`, `chat-history`, and `whoever-answered` are not living primaries. A secondary exists so the primary is not a single hero. The evaluator checks that the primary is named and that `living_primary` is true. It cannot prove that the named person is awake. That limitation is honest.

Best Practice: Bind every Chapter 5 page destination to exactly one system, and every system to a rotation with a living primary and a secondary.

Production Practice: Slack-as-primary is the heroics path this book interrupts. A rotation that lists Slack as primary fails even if the catalog contact is correctly referenced.

Load, compensation, and training are operational constraints

Page load is a numeric limit the rotation can absorb. Compensation and training are constraints on whether that limit is survivable. They are not an HR policy chapter. If the system cannot name a load limit, later toil bounds have nothing to protect. If compensation and training are "someone else's problem," the rotation will be staffed by whoever feels guilty.

Handoff is a record, not a chat message

A rotation without a handoff is a shift that never ended. The handoff names `from-primary`, `to-primary`, and when it was recorded. Missing handoff is how Slack becomes the real primary at 17:01.

Authority consumes DevSecOps; it does not rewrite it

Break-glass remains DevSecOps-attributable. The inherited authorization interface requires `self_approval_forbidden: true`. The inherited privilege interface requires requester, independent approver, expiry, and after-action review. This chapter consumes those fields. A primary who can approve their own freeze or break-glass is the unofficial path again.

The Storefront primary may act on Chapter 4's freeze of Storefront releases. They may not approve that action as themselves. Reliability-program or a named independent approver remains in the record.

3. Design the on-call systems

The completed Chapter 6 model uses four files:

```
oncall/system.yaml
oncall/rotations.yaml
oncall/handoffs.yaml
oncall/authority.yaml
```

The separation is deliberate. Systems reference catalog contacts and name load. Rotations name living primaries. Handoffs are the shift record. Authority consumes `self_approval_forbidden` and break-glass shape. Chapter 5 pages stay `destination_kind: catalog-contact`. This chapter binds those contacts to systems.

Practice — Reference the catalog contact; do not become it

Give Storefront, Fulfillment, and the platform product distinct system ids that point at the inherited escalation ids.

Open `oncall/system.yaml`. Storefront is recorded as:

```
- id: storefront-oncall-system
  catalog_contact: storefront-oncall
  owner: storefront-team
  journeys: [accept-and-complete-order]
  page_load_limit: 8
  compensation: operational-constraint
  training: operational-constraint
```

Inspect each system with three questions:

1. Is `id` different from `catalog_contact`, or was the label copied as the roster?
2. Do Chapter 5 pages to that contact bind here, or to Slack?
3. Would a platform ticket still land on Storefront if this row were deleted?

`page_load_limit: 8` is a teaching value: eight pages per week before the rotation is overloaded. It is not an industry constant. Compensation and training are marked operational constraints so Chapter 7 can measure toil against a staffed rotation, not against unpaid heroics.

Practice — Name a living primary and record a handoff

Slack is not a primary. A shift without a handoff record is still Slack.

Open `oncall/rotations.yaml` and `oncall/handoffs.yaml`. Each system has a primary and a secondary. `living_primary` is `true`. `as_of` is `2026-08-16T00:00:00Z`. Each rotation has a handoff from one named primary to the next. If `primary: slack`, the checkpoint fails. If handoffs are empty, the checkpoint fails.

Practice — Bind authority to `self_approval_forbidden`

A responder may act on a freeze. They may not approve that action as themselves.

Open `oncall/authority.yaml`. Storefront freeze authority sets `self_approval_forbidden: true` to match the inherited authorization field. Break-glass names requester, independent approver, expiry, and `after_action_review: true`. If the approver equals the requester, the unofficial path is back.

Chapter 5 pages to `storefront-oncall` now resolve to `storefront-oncall-system` and its living primary. Pages to `fulfillment-oncall` resolve to `Fulfillment`. Tickets to `platform-oncall` resolve to the platform system. They must not resolve to Storefront.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-06-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 06 checkpoint: living rotations, handoffs, and attributable
authority verified
```

The audit validates the four Chapter 6 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- every Chapter 5 page and platform ticket destination binds to a system whose `catalog_contact` matches;
- no system id equals its catalog contact;
- every system has a rotation with a living primary and a secondary;
- `slack`, `chat-history`, and `whoever-answered` are not primaries;
- every rotation has a handoff record;
- platform destinations do not land on Storefront's rotation;
- `self_approval_forbidden` remains `true`; and
- break-glass names an independent approver, expiry, and after-action review.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not operate a real paging or calendar product. It does not prove that the named primary is trained or compensated. Those are judgment claims. The operational-constraint fields exist so later toil measurement cannot pretend staffing was free.

This distinction is the chapter's evidence model:

Evidence category	Chapter 6 evidence
Mechanism evidence	Schemas and the on-call evaluator operated successfully.
Decision evidence	Systems, rotations, handoffs, load limits, and attributable authority are explicit.
Outcome evidence	Not yet produced as a survived night; a living-primary flag is not proof someone woke.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 6 creates decision evidence about the human system. Pretending that the local checkpoint proved a real responder answered would weaken every later incident, game day, and fail-over.

4. Test the design under failure

Cumulative reliability failure — Pages still go to chat history

Practice — Bind pages to rotations and interrupt Slack-as-primary

Give each catalog contact a system, a living primary, a handoff, and authority that cannot self-approve.

The baseline fixture contains this model:

```
- id: storefront-oncall
  catalog_contact: storefront-oncall
  primary: slack
  handoff: none
  self_approval_forbidden: false
```

The problem is not merely missing YAML. The model treats the catalog label as the roster and Slack as the primary. Platform tickets can follow Storefront because that is who is in the channel. A freeze can be self-approved because authority was never designed.

Severity: high; Chapter 5's journey pages have no human system, so Chapter 4's freeze is a file nobody is obligated to enact.

Plausible harm: order-success burn pages Slack; the named engineer is not on shift; platform wait time wakes Storefront; a self-approved break-glass has no after-action review.

Potential blast radius: every catalog contact used as a chat mention; Fulfillment and platform inherit Storefront's channel.

Bounded by: later toil bounds, incident command, and game-day page-path exercises. None repairs a roster that is a Slack channel.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** accept-and-complete-order pages a contact with no living primary. dispatch-fulfillment is in the same channel if only one label exists.
- **Error budget:** Applicable as the freeze the primary must be able to enact. Chapter-local implication: Slack cannot freeze storage-1-0-to-2-0 with attributable authority.
- **Human system:** Applicable as the chapter's subject. Slack-as-primary is informal heroics. Load, handoff, and independent approval are the system.
- **Portfolio recovery:** Not yet produced. A Slack mention cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling the roster storefront-oncall encourages label controls: paste the catalog id, mention the channel, skip handoff because "we all saw the message." Those moves may find a human once. They do not answer who is primary as of now, who is secondary, what load is allowed, or who can approve break-glass without self-approval.

The missing rotation makes Chapter 5's page map a destination without a person. The missing handoff makes the last shift the permanent primary. `self_approval_forbidden: false` makes DevSecOps ornamental. Routing platform destinations to Storefront finishes the collapse: job-time again wakes the order path.

Correction

The completed model does not page Slack. It references each catalog contact from a distinct system, names living primaries and secondaries, records handoffs, limits page load, keeps compensation and training as operational constraints, consumes `self_approval_forbidden`, and keeps platform destinations on the platform rotation.

That correction changes later decisions:

- Chapter 7 must measure toil against these rotations, not against "we were in Slack."
- Chapter 10 must page these systems, not a one-path hero.
- Chapter 13 must exercise the on-call page path, not a catalog mention.
- Chapter 14 must not invent a commander from chat history.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common on-call errors

Treating the catalog contact as the system

The contact is how the catalog points at a team. The system is rotation, load, handoff, and authority.

Slack-as-primary

A channel is a distribution surface. It is not a living primary.

Skipping handoff because the same person stays on

A record is how the next primary exists. Chat is how the last primary never ends.

Landing platform pages on Storefront

Job-time and plane issues belong on the platform rotation. Storefront cannot absorb them as courtesy.

Self-approved break-glass

Inherited `self_approval_forbidden` is the named field. Do not invert it to ship faster.

Calling compensation and training “HR”

If the rotation cannot be staffed, later **SLOs (Service Level Objectives)** are theater. Name the constraints here; do not write the payroll policy.

6. What changed

Before	After
storefront-oncall was the system.	Each system references a catalog contact and has a different id.
Primary was Slack.	Each rotation has a living primary and a secondary.
Handoff was a chat message.	Each rotation has a recorded handoff.
Platform tickets woke Storefront.	Platform destinations bind to the platform system.
Break-glass could self-approve.	<code>self_approval_forbidden</code> remains true; approver is independent.
Load, compensation, and training were implied.	Load is numeric; compensation and training are operational constraints.
A valid schema could appear to prove a roster.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely four YAML files. Northwind now has a reviewable human system that later toil, incidents, and game days can page without treating Slack as the roster.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
On-call system contract	oncall/system.yaml	It retains the reference from catalog contact to system, load, and operational constraints later chapters must not replace with a channel name.
Rotation-and-handoff record	oncall/rotations.yaml and oncall/handoffs.yaml	They retain living primaries and the shift record a page actually binds to.

These artifacts should change when Northwind’s catalog contacts, staffing, or authority model materially change—not whenever a chat tool is renamed.

What You Learned

An on-call system references a catalog contact and adds rotation, load, handoff, and authority. The contact label is not the system. Slack is not a living primary. Platform destinations must not land on Storefront. Break-glass remains DevSecOps-attributable through `self_approval_forbidden`. Schema checks can prove structural completeness within declared scope. They cannot operate a real paging calendar. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Staff catalog-browse pages without copying the Fulfillment rotation

A storefront engineer wants catalog-browse burn to page storefront-oncall “because Fulfillment already has a rotation we can paste.”

Extend the Chapter 6 model without adding toil policy yet:

1. Decide whether catalog-browse pages share the Storefront system or need a distinct rotation.
2. If they share Storefront, show how page load stays inside the numeric limit.
3. Name a living primary that is not Slack and not a pasted Fulfillment id.
4. Record a handoff; keep `self_approval_forbidden` true.
5. Identify one observation that would falsify the design—for example platform tickets landing on this rotation.
6. Explain which material change would trigger review of the system, not just the chat mention.

Do not copy the Fulfillment rotation and rename it. Catalog reads have different load and freeze consequences from warehouse dispatch. Your durable output is the system decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 6 capability when you can explain why a catalog contact is not an on-call system, bind every Chapter 5 page to a living primary, refuse Slack-as-primary, keep platform destinations off Storefront, consume `self_approval_forbidden`, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Humans can be paged with a rotation, a handoff, and attributable authority. Most of their week is still unmeasured tickets, silences, and copy-paste runbooks. “We will automate later” has no bound.

Chapter 7 measures and bounds toil so reliability engineering remains possible, and so a new critical SLO cannot be added because on-call “already watches email.”

7. Measure and Bound Toil Without Hiding the Work

Chapter 6 gave Storefront, Fulfillment, and the platform product living primaries, handoffs, and attributable authority. Those humans still spend the week on tickets, manual silences, and copy-paste runbooks. “We will automate later” has no number. “We are busy” is not a measurement. On-call “already watches email,” so someone proposes a new critical **SLO (Service Level Objective)** for notification-service.

The production question is now:

What work is toil, how is it measured, and what bound keeps reliability engineering possible?

Without that decision, unbounded toil makes SLO work theater. Pretending tickets are engineering hides the missing automation. Automating the wrong toil creates a new hero path: a bot that silences pages so nobody sees the journey burn. Chapter 3 already recorded notification as non-critical. Chapter 7 must not promote it because the rotation is drowning.

This chapter adopts a toil definition, an inventory, and a numeric bound. Remaining toil fraction is computed from inventoried hours, not emitted as “plenty of capacity.” A breach blocks adding a new critical SLO.

1. An unsafe toil decision

A weak record says:

```
bound: we-are-busy
proposed_slo: notification-service
criticality: critical
justification: on-call-already-watches-email
```

It does not classify the week's work, name a fraction, name an owner, or compute hours. “Already watches email” may describe last night's inbox. It cannot complete a toil decision. It reopens a Chapter 3 non-critical system as a critical SLO so the heroics look like scope.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 7 baseline:

```
make chapter-07-baseline
```

The command succeeds when it detects the intended unsafe decision:

```
chapter 07 baseline: unmeasured toil adding notification as critical slo
correctly detected
```

The fixture leaves inventory unclassified, stores the bound as we-are-busy, emits a comforting capacity constant, and allows notification-service as a new critical SLO because on-call already watches email. Toil has been treated as a mood. Scope has been treated as a reward for being overloaded.

That inversion drives the chapter.

2. The production model: classification, fraction, and a gate on scope

Theory — Toil bound

This model enables Northwind to keep reliability engineering possible by measuring interrupt work against a numeric bound, rather than by adding SLO scope because the rotation is already busy.

Toil is classified work, not a feeling

Toil is repetitive, manual, and scales with production: silences, ticket triage, copy-paste runbooks. Engineering reduces toil or advances SLO governance: writing the Chapter 4 freeze, fixing the good-event definition, removing a class of tickets. Interrupt work arrives unplanned. Project work is planned.

“We are busy” classifies nothing. A ticket volume graph without classes is a dashboard of heroics. An automation that files more tickets than it closes is engineering that hides toil.

Best Practice: Give every inventoried item a class (toil or engineering) and a kind (interrupt or project), with hours per week.

Production Practice: Notification ticket triage is toil and non-critical. It is not a new critical journey. Chapter 3 already refused that promotion.

The bound is a fraction computed from hours

Available hours are the reliability time the rotation can spend—not calendar myth, not Slack presence. Toil fraction is:

```
toil fraction = (sum of toil hours per week) / available hours per week
```

The bound is a number in (0, 1], owned by reliability-program. Teaching values: 20 available hours, bound 0.5. If inventoried toil is 14 hours, the fraction is 0.7 and the bound is breached. The bound file must not emit `toil_fraction` or `remaining_capacity`. Those would let the inventory approve itself.

The lab cannot measure real engineer hours. The inventory hours are local teaching fixtures, like Chapter 3’s event counts.

A breach blocks new critical SLO scope

When the fraction meets or exceeds the bound, Northwind may not add a new critical SLO. The first attempted add in this chapter is `notification-service`. It must be denied twice: it is already non-critical in the Chapter 3 catalog, and toil is over the bound.

Allowing it because “on-call already watches email” spends overload as permission. That is informal heroics applied to scope.

Automation that removes toil shrinks inventoried hours. Automation that hides toil—auto-silences that never reach a living primary—must stay classified as toil, not as engineering, until the hours actually fall.

Load from Chapter 6 is a constraint, not the bound

`page_load_limit: 8` is how many pages a rotation can absorb. Toil hours are how much of the week is interrupt work. They are related and not the same number. Do not replace the bound with “we only got six pages.” Six pages can still consume fourteen hours of silences and runbooks.

3. Adopt the definition, inventory, and bound

The completed Chapter 7 model uses three files:

```
toil/definition.yaml
toil/inventory.yaml
toil/bounds.yaml
```

The separation is deliberate. The definition names classes, kinds, and forbidden measurements. The inventory is the classified week. The bound names the fraction, available hours, owner, and denied scope proposals. Toil fraction is computed at evaluation time.

Practice — Forbid mood as a measurement

Name `we-are-busy` and `on-call-already-watches-email` as labels that cannot be a bound or a scope justification.

Open `toil/definition.yaml`. Allowed classes are `toil` and `engineering`. Allowed kinds are `interrupt` and `project`. Forbidden measurements include `we-are-busy` and `on-call-already-watches-email`. If those strings appear as the bound, later chapters inherit a slogan.

Practice — Classify the week, including notification triage

Keep notification work as non-critical toil. Do not relabel it as `engineering` so it can become an SLO.

Open `toil/inventory.yaml`. Teaching rows include manual alert silences, copy-paste runbooks, notification ticket triage, and error-budget policy engineering. Each row has `class`, `kind`, and `hours_per_week`. Notification triage is `toil`, `interrupt`, and `non-critical`. It is not a Chapter 1 journey.

Practice — Deny the notification critical SLO while the bound is breached

Compute the fraction from hours. Do not store remaining capacity. Do not allow the proposal.

Open `toil/bounds.yaml`. `bound_fraction` is 0.5. `available_hours_per_week` is 20. Owner is `reliability-program`. The scope proposal for `notification-service` as `critical` is `deny`, with reason that it is a non-critical journey and that toil exceeds the bound. If `decision: allow` appears, the checkpoint fails. If `toil_fraction` or `remaining_capacity` appears on the bound, the checkpoint fails.

Fourteen toil hours in a twenty-hour week is remaining engineering time of six hours, not permission to add email as a critical SLO.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-07-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 07 checkpoint: classified inventory, numeric bound, and blocked
notification SLO verified
```

The audit validates the three Chapter 7 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- every inventory item has class and kind;
- the bound is a number, not we-are-busy;
- toil fraction is computed from hours and available hours;
- the fraction meets or exceeds the bound in the teaching fixture;
- notification-service cannot be allowed as a new critical SLO;
- the proposal is denied against the Chapter 3 non-critical record; and
- forbidden justifications are not used as allow reasons.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not measure real engineer hours. It does not prove that 0.5 is the right commercial bound. Those are judgment claims. The review trigger exists so the bound can be reopened without pretending it was never set.

This distinction is the chapter's evidence model:

Evidence category	Chapter 7 evidence
Mechanism evidence	Schemas and the toil evaluator operated successfully.
Decision evidence	Definition, classified inventory, numeric bound, and denied notification scope are explicit.
Outcome evidence	Toil fraction is computed from inventoried hours, not emitted as capacity.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 7 creates decision evidence and a computed fraction. Pretending that the local checkpoint proved a real week's hours would weaken later learning and game days.

4. Test the decision under failure

Independent control failure — Unmeasured toil adds notification as a critical SLO

Practice — Classify notification as non-critical toil and enforce the bound

Replace we-are-busy with a fraction. Deny the critical SLO.

The baseline fixture contains this model:

```
bound: we-are-busy
proposed_slo: notification-service
criticality: critical
justification: on-call-already-watches-email
toil_fraction: 0.1
```

The problem is not merely missing hours. The model treats overload as permission and a Chapter 3 non-critical system as the next SLO. A constant `toil_fraction: 0.1` is the same self-approval Chapter 3 refused when remaining budget was emitted.

Severity: high; the humans Chapter 6 just designed will spend the next SLO on email while order and dispatch toil stay unbounded.

Plausible harm: notification becomes a critical page; order silences stay manual; engineering time disappears; a silence bot hides journey burn.

Potential blast radius: every inbox someone “already watches”; every busy rotation rewarded with more scope.

Bounded by: later dependency contracts (email stays non-critical) and learning-program action bounds. None repairs a bound that is a mood.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** notification-service is not accept-and-complete-order or dispatch-fulfillment. Adding it as critical spends the wrong human system on the wrong outcome.
- **Error budget:** Not a freeze decision. Chapter-local implication: a new critical SLO would consume Chapter 4 freeze attention and Chapter 5 pages the rotation cannot absorb.
- **Human system:** Applicable as load on the Chapter 6 rotations. Unmeasured toil makes `page_load_limit` ornamental.
- **Portfolio recovery:** Not yet produced. A quieter inbox cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling the bound “we are busy” encourages scope controls: add the SLO that matches the inbox, stamp a low fraction so the slide is green, skip classification because tickets “feel like engineering.” Those moves may be sincere. They do not answer which hours are toil, whether notification is a protected journey, or what would have to shrink before scope grows.

The missing classes make automation that hides toil look like progress. The allow on notification makes Chapter 3's non-critical row ornamental. The emitted fraction makes outcome evidence a constant.

Correction

The completed model does not allow the notification SLO. It classifies silences, runbooks, and notification triage as toil, classifies freeze-policy work as engineering, computes a fraction above 0.5 from 14 / 20 hours, denies the proposal, and forbids mood labels as the bound.

That correction changes later decisions:

- Chapter 8 must keep email non-critical in the dependency contract, not promote it because toil was high.
- Chapter 10 must not page Storefront as if notification were a critical journey.
- Chapter 11 must bound learning actions so they do not recreate this inventory as homework.
- Chapter 13 must not treat "we were busy" as a skipped game day.

The decision is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common toil errors

Using mood as the bound

"We are busy" cannot gate scope. A fraction can.

Rewarding overload with a new SLO

The inbox that already consumes the week is usually the thing that should shrink, not the thing that should become critical.

Calling tickets engineering

Classification is the decision. Volume is not.

Automating silence instead of removing the class of work

A bot that hides pages is toil with a nicer name until inventoried hours fall.

Emitting remaining capacity on the bound file

Computed fraction is outcome evidence. A stored 0.1 is a slide.

Ignoring Chapter 3 non-critical rows

notification-service was already refused as a protected journey. Toil does not reverse that.

6. What changed

Before	After
The bound was we-are-busy.	The bound is 0.5 of 20 available hours, owned by reliability-program.
Inventory was unclassified tickets.	Each item has class, kind, and hours per week.
Notification became a proposed critical SLO.	The proposal is denied; triage stays non-critical toil.
Capacity was a stored 0.1.	Toil fraction is computed from 14 / 20 hours.
“Already watches email” justified scope.	Forbidden justifications cannot allow a new critical SLO.
A valid schema could appear to prove a bound.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable work-allocation contract that later dependencies, incidents, and learning cannot expand by rewarding overload.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Toil definition and bound	toil/definition.yaml and toil/bounds.yaml	They retain the classes, forbidden measurements, fraction, and scope gate later chapters must not replace with mood.
Toil inventory	toil/inventory.yaml	It retains classified hours the fraction is computed from.

These artifacts should change when Northwind’s rotations, interrupt classes, or SLO scope materially change—not whenever the ticket tool is renamed.

What You Learned

Toil is classified, measured as a fraction of available hours, and bounded. A breach blocks new critical SLO scope. notification-service stays non-critical. “We are busy” is not a measurement. Automation that hides toil is still toil. Schema checks can prove structural completeness within declared scope. They cannot measure real engineer hours. A decision earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Bound catalog-browse toil without copying the notification row

A storefront engineer wants a critical catalog-browse SLO “because on-call already answers those tickets.”

Extend the Chapter 7 model without adding dependency policy yet:

1. Classify catalog-browse tickets as toil or engineering, interrupt or project.
2. Decide whether they are in-scope for a new critical SLO given Chapters 1–3.
3. Recompute toil fraction with one teaching hour count; do not emit `toil_fraction`.
4. Allow or deny the proposal against the numeric bound.
5. Identify one observation that would falsify the classification—for example hours falling after a real removal of the ticket class.
6. Explain which material change would trigger review of the bound, not just the ticket queue.

Do not copy the notification deny and rename it. Catalog reads have different criticality and freeze consequences from confirmation email. Your durable output is the classification and gate, not the number of YAML lines changed.

You can demonstrate the Chapter 7 capability when you can explain why “we are busy” is not a bound, classify every inventoried item, compute toil fraction from hours, deny `notification-service` as a new critical SLO, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Engineering time is protected by a numeric gate. Third parties can still burn the budget with no contract. Payment, warehouse, and email can fail while Northwind’s own graphs stay green, or every provider page is treated as a Storefront page.

Chapter 8 puts dependencies inside the reliability contract so payment burns Storefront, warehouse burns Fulfillment, and email stays non-critical.

8. Put Dependencies Inside the Reliability Contract

Chapter 7 classified toil, computed a fraction, and denied a new critical **SLO (Service Level Objective)** for notification-service. Engineering time is gated. Payment, warehouse, and email can still fail while Northwind's own graphs stay green, or every provider page is treated as a Storefront page.

The production question is now:

Which dependencies are inside a journey's SLO, which are degraded or non-critical, and what evidence does each provider owe?

Without that contract, absorbing a provider's unbounded failure makes the SLO dishonest. Ignoring a critical provider makes the SLO theater. Chapter 3 already recorded notification as non-critical. Chapter 5 already pages `order_success_ratio` at `storefront-oncall` and `dispatch_success_ratio` at `fulfillment-oncall`. Chapter 8 must not reopen email as a critical Storefront page, and it must not let a payment timeout stamp "no user impact."

This chapter publishes dependency contracts that join Chapter 3 journeys to providers. Payment failure burns `accept-and-complete-order`. Warehouse failure burns `dispatch-fulfillment`. Email failure is recorded, not paged as a critical SLO. A dependency row cannot emit its own "no user impact." Timeout and retry budget are named here. How Northwind sheds load when those bounds are crossed is Chapter 9.

1. An unsafe dependency map

A weak record says:

```
- id: payment
  no_user_impact: true
- id: notification-service
  criticality: critical
  destination: storefront-oncall
```

It treats a payment timeout as someone else's graph. It pages confirmation email as if it were the order journey. Warehouse may be missing, or copied onto Storefront so Fulfillment never burns. "The provider status page is green" may describe a vendor dashboard. It cannot complete a Northwind contract.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 8 baseline:

```
make chapter-08-baseline
```

The command succeeds when it detects the intended unsafe mapping:

```
chapter 08 baseline: payment no-impact while email pages storefront
correctly detected
```

The fixture stamps `no_user_impact` on payment, pages `notification-service` at `storefront-oncall` as critical, and attributes warehouse to Storefront. Payment timeouts can exhaust the order **SLI (Service Level Indicator)** while the dependency file claims nobody was harmed. Email pages the Storefront rotation for a journey Chapter 3 refused. The wrong budget and the wrong humans absorb the failure.

That inversion drives the chapter.

2. The production model: criticality, attribution, and named bounds

Theory — Dependency contract

This model enables Northwind to put payment, warehouse, and email inside journey SLOs, rather than treating every provider page as Storefront or claiming a timeout has no user impact.

A provider SLO is not a Northwind SLO

Payment may advertise 99.95 percent availability. That number is the provider's product. It is not `accept-and-complete-order`. If a payment timeout prevents a valid order from reaching a correct terminal state, Storefront remaining budget burns. A green vendor status page is mechanism evidence. It is not remaining error budget.

Warehouse is the same join on the other journey. A warehouse outage that blocks dispatch is `dispatch-fulfillment` burn, not a Storefront tile and not a copied Storefront target.

Email is not a third critical journey. Chapter 3 already recorded `notification-service` as non-critical. Chapter 7 already denied promoting it because on-call watched the inbox. A confirmation that never sends can annoy a customer. That is not permission to page Storefront as if the order had failed, and it is not permission to write `no_user_impact: true`.

Best Practice: Name each provider, the journey it can burn or must not burn, a timeout, a retry budget, and a fallback.

Production Practice: Payment is fail-closed on `accept-and-complete-order`. Warehouse is fail-closed on `dispatch-fulfillment`. Email is skip-and-record. None of those rows may emit user impact or its absence.

Critical versus non-critical is attribution, not a feeling

Provider	Consumer	Journey	Failure effect
payment	Storefront	<code>accept-and-complete-order</code>	journey-burn on <code>order_success_ratio</code>
warehouse	Fulfillment	<code>dispatch-fulfillment</code>	journey-burn on <code>dispatch_success_ratio</code>
<code>notification-service</code>	Storefront	none; Chapter 3 non-critical	record; do not page as a critical SLO

Copying Storefront's payment row onto warehouse does not protect dispatch. Copying warehouse onto Storefront exhausts the order budget for a warehouse the Storefront rotation does not own. Treating every provider as equally critical pages email with payment.

Chapter 5 already pages those two journey **SLIs**. This chapter does not add a third critical page for email. It says which provider failures *are* those burns.

Timeout and retry budget are named; cascade is not yet the chapter

A contract without a timeout absorbs the provider until the worker pile grows. Teaching values: payment 2s with retry budget 1, warehouse 5s with retry budget 1, email 10s with retry budget 0. Those numbers are local fixtures, not industry constants.

Retry budget 1 is not permission to retry until Fulfillment is pulled into Storefront's storm. Chapter 9 owns shedding and cascade denial. Chapter 8 must already name the bound, or Chapter 9 inherits "try harder" as the contract.

Fallback fail-closed means the journey is not counted good when the provider fails. Fallback skip-and-record means the supporting step is skipped and the protected journey is not automatically burned. Skip is not "no user impact." It is an explicit, reviewable choice that confirmation email is outside the protected outcome.

A row cannot approve its own harmlessness

`no_user_impact: true` is the same self-approval Chapter 3 refused when remaining budget was emitted, Chapter 5 refused when a page stamped `user_impact`, and Chapter 7 refused when the bound emitted `toil_fraction`. Outcome evidence is computed from the journey, not stored on the provider row.

The lab does not query a real provider status page. Attribution is a local contract.

3. Publish the catalog, criticality, and contracts

The completed Chapter 8 model uses three files:

```
dependencies/catalog.yaml
dependencies/criticality.yaml
dependencies/contracts.yaml
```

The separation is deliberate. The catalog names providers, consumers, and forbidden claims. Criticality joins each provider to a journey, **SLI**, and failure effect. Contracts name timeout, retry budget, fallback, and the evidence token that says whether failure is Northwind burn. None of the files may carry `user_impact` or `no_user_impact`.

Practice — Forbid “no user impact” as a self-claim

Keep no-user-impact on the forbidden-claim list. Do not store harmlessness on the provider.

Open dependencies/catalog.yaml. Providers are payment (Storefront), warehouse (Fulfillment), and notification-service (Storefront, supporting). Forbidden claims include no-user-impact. If that string is missing, later chapters inherit a slogan. Owner is reliability-program.

Practice — Attribute payment to Storefront burn and warehouse to Fulfillment burn

Do not copy the Storefront payment row onto warehouse. Do not put warehouse on accept-and-complete-order.

Open dependencies/criticality.yaml. Payment is critical, journey accept-and-complete-order, **SLI** order_success_ratio, failure effect journey-burn. Warehouse is critical, journey dispatch-fulfillment, **SLI** dispatch_success_ratio, failure effect journey-burn. Notification is non-critical, failure effect record, remaining owner storefront-team from Chapter 3. If destination: storefront-oncall appears on email as a critical page, the checkpoint fails.

Inspect each assignment with three questions:

1. If this provider times out, which Chapter 1 journey fails—or which Chapter 3 non-critical row already refused that promotion?
2. Would this still be true if the provider’s own status page stayed green?
3. Is the consumer the team that owns that journey, or whoever got the first ticket?

Practice — Name timeout, retry budget, and fallback without pretending cascade is solved

Payment and warehouse fail-closed. Email skip-and-record. Do not emit remaining budget or user impact.

Open dependencies/contracts.yaml. Payment timeout 2s, retry budget 1, fallback fail-closed, evidence payment-timeout-is-storefront-burn. Warehouse timeout 5s, retry budget 1, fallback fail-closed, evidence warehouse-timeout-is-fulfillment-burn. Email timeout 10s, retry budget 0, fallback skip-and-record, evidence email-failure-is-not-order-burn. If payment fallback is skip-and-record, orders can look successful without payment. If email fallback is fail-closed, confirmation has been promoted onto the critical path.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-08-checkpoint
```

Expected output includes:

```

inherited interface verification: passed
artifact validation: passed
chapter 08 checkpoint: attributed burns and non-critical email verified

```

The audit validates the three Chapter 8 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- payment, warehouse, and notification-service are cataloged;
- payment is critical journey-burn on accept-and-complete-order / order_success_ratio;
- warehouse is critical journey-burn on dispatch-fulfillment / dispatch_success_ratio;
- email is non-critical, recorded, and not paged as a Storefront critical SLO;
- the Chapter 3 non-critical row is consumed, not reversed;
- payment fallback is fail-closed and email fallback is skip-and-record;
- timeout and retry budget are present and numeric;
- no-user-impact is forbidden; and
- no row emits user_impact or no_user_impact.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not query a real provider status page. It does not prove that 2 seconds or one retry is the right commercial bound. Those are judgment claims. The review triggers exist so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 8 evidence
Mechanism evidence	Schemas and the dependency evaluator operated successfully.
Decision evidence	Catalog, criticality, timeout, fallback, and forbidden claims are explicit.
Outcome evidence	Payment and warehouse burns are attributed to journeys; harmlessness is not stored on the row.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 8 creates decision evidence about which provider failures are whose burn. Pretending that the local checkpoint proved a vendor outage would weaken every later degradation, incident, and game day.

4. Test the design under failure

Connected consequence — Payment timeouts burn Storefront while email is paged as equally critical

Practice — Reclassify email; attribute payment to Storefront; keep warehouse on Fulfillment

Drop `no_user_impact`. Stop paging confirmation as the order journey.

The baseline fixture contains this model:

```
payment:
  no_user_impact: true
  failure_effect: none
notification-service:
  criticality: critical
  destination: storefront-oncall
warehouse:
  journey: accept-and-complete-order
```

The problem is not merely a missing timeout. Payment can take the order **SLI** while the contract claims nobody was harmed. Email exhausts Storefront pages and Chapter 4 freeze attention for a system Chapter 3 called non-critical. Warehouse copied onto Storefront leaves dispatch without a provider contract and spends the wrong remaining budget.

Severity: high; exhausted Storefront budget and a Storefront rotation now absorb email, while payment harm is denied in the file that should have attributed it.

Plausible harm: orders fail at payment while graphs stay green; confirmation pages wake the order primary; Fulfillment dispatch has no warehouse burn.

Potential blast radius: every vendor treated as equal to payment; every supporting tool promoted because it already has a ticket queue.

Bounded by: later degradation (retry amplification) and incident command. None repairs a contract that denies payment harm and pages email.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** Payment is inside `accept-and-complete-order`. Warehouse is inside `dispatch-fulfillment`. `notification-service` is not a protected journey.
- **Error budget:** Payment timeouts consume Storefront remaining budget. Email pages would spend freeze attention on the wrong SLO. Not yet a new freeze rule.
- **Human system:** Applicable as load on Chapter 6 rotations. Email as a critical Storefront page reopens Slack-as-heroics under a provider name.
- **Portfolio recovery:** Not yet produced. A green payment status page cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling payment harmless encourages two controls: keep Storefront graphs green by excluding the timeout, and page whatever already has an inbox so someone is “covering dependencies.” Those moves may be sincere. They do not answer which journey the customer lost, which remaining budget burns, or which rotation Chapter 6 designed for that page.

The `no_user_impact` stamp makes outcome evidence a constant. The email page makes Chapter 3’s non-critical row and Chapter 7’s deny ornamental. Warehouse on Storefront copies the wrong consumer, the same class of copy Chapter 3 forbade for SLO targets.

Correction

The completed model does not let payment approve its own harmlessness. It attributes payment timeout as Storefront journey-burn, warehouse timeout as Fulfillment journey-burn, and email as non-critical record with skip-and-record. It forbids `no-user-impact` as a claim. It names timeouts and retry budgets without pretending cascade is solved.

That correction changes later decisions:

- Chapter 9 must shed and bound retries against these contracts, not invent a new payment path that is “always success.”
- Chapter 10 must not close a portfolio incident because email was paged and Storefront graphs were green.
- Chapter 13 must exercise dependency loss as payment or warehouse, not as a mixed email drill that stands in for both journeys.
- Chapter 14 must not treat a restored notification template as **Evidence of portfolio recovery**.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary vendor API call would not make it more practical.

5. Production reality

Common dependency errors

Storing “no user impact” on the provider row

Harmlessness is not a field the dependency gets to emit. Attribution is.

Paging every provider as Storefront

Email is not payment. A contact that already exists is not a criticality decision.

Copying Storefront’s payment contract onto warehouse

Fulfillment has a different journey, **SLI**, window, and rotation. Copying the consumer copies the lie.

Treating a provider status page as remaining budget

Mechanism evidence is not outcome evidence.

Skipping timeout so the worker can “wait it out”

An unnamed bound is not a bound. Chapter 9 cannot deny a cascade that Chapter 8 never limited.

Fail-closing email, or skip-recording payment

Fallback is criticality in operational form. Mixing them reverses Chapters 3 and 1.

6. What changed

Before	After
Payment stamped no_user_impact.	Payment timeout is Storefront journey-burn on order_success_ratio.
Email paged storefront-oncall as critical.	Email is non-critical, recorded, skip-and-record.
Warehouse sat on the order journey.	Warehouse burns dispatch-fulfillment on Fulfillment.
Provider status was treated as the SLO.	Provider availability is not the Northwind journey SLO.
Timeout and retry were “try harder.”	Timeouts and retry budgets are named; cascade remains Chapter 9.
A valid schema could appear to prove harmlessness.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable provider contract that later degradation, incidents, and game days can consume without paging email as the order path or hiding payment timeouts behind a slogan.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Dependency reliability catalog	dependencies/catalog.yaml	It retains providers, consumers, and the forbidden harmlessness claim later chapters must not restore.
Criticality-and-fallback contract	dependencies/criticality.yaml and dependencies/contracts.yaml	They retain which provider burns which journey, with timeout, retry budget, and fallback.

These artifacts should change when Northwind’s providers, protected journeys, or fallback policy materially change—not whenever a vendor renames a status page.

What You Learned

Dependencies sit inside journey SLOs or they are explicitly non-critical. Payment burns Storefront. Warehouse burns Fulfillment. Email stays non-critical. A provider’s advertised availability is not Northwind’s SLO. A row cannot emit “no user impact.” Timeout and retry budget are named here; cascade control is the next chapter. Schema checks can prove structural completeness within declared scope. They cannot query a real provider. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Contract catalog-browse search without copying the payment row

A storefront engineer wants the catalog search index marked critical “because browse already has tickets, and users see search more than they pay.”

Extend the Chapter 8 model without adding shedding policy yet:

1. Decide whether catalog-search is critical journey-burn, non-critical record, or out of scope, given Chapters 1–3 treatments of catalog reads.
2. Name a consumer, timeout, retry budget, and fallback that are not a renamed payment `fail-closed`.
3. Join or refuse a Chapter 1 journey; do not invent `accept-and-complete-order` coverage for search.
4. Do not emit `no_user_impact`, even if you skip search.
5. Identify one observation that would falsify the criticality—for example payment timeouts remaining “no impact” while search pages Storefront.
6. Explain which material change would trigger review of the contract, not just the vendor’s status tile.

Do not copy the warehouse row and rename it. Catalog reads have different criticality, consumers, and freeze consequences from dispatch. Your durable output is the attribution and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 8 capability when you can explain why a provider status page is not a Northwind SLO, attribute payment to Storefront and warehouse to Fulfillment, keep email non-critical, refuse `no_user_impact`, name timeout and retry budget without claiming cascade is solved, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Dependencies are named and attributed. Northwind’s own services can still retry until the portfolio cascades. `order-worker` can retry payment without a shed. Storefront can accept work the worker cannot finish. Fulfillment can be pulled into Storefront’s retry storm.

Chapter 9 defines degradation and shedding so a payment slowness does not become every journey’s outage, and so degraded success is accounted as burn rather than hidden as a green graph.

9. Degrade Deliberately Before Failure Cascades

Chapter 8 put payment, warehouse, and email inside the reliability contract. Payment timeout is Storefront journey-burn. Warehouse timeout is Fulfillment journey-burn. Email stays skip-and-record. Those bounds still sit on the provider row. order-worker can retry payment without a shed. Storefront can accept orders the worker cannot finish. Fulfillment can be pulled into Storefront's retry storm. Graphs of "accepted orders" stay green.

The production question is now:

How does Northwind shed load, refuse work, or run in a degraded mode before a local failure becomes every journey's outage?

Without that design, "try harder" converts a payment slowness into a portfolio outage. Silent drop without a named degraded mode hides customer harm. Chapter 8's retry budget of 1 is not permission to retry until both **SLOs (Service Level Objectives)** are exhausted. Chapter 5 already pages `order_success_ratio` at Storefront and `dispatch_success_ratio` at Fulfillment. Chapter 9 must not page Fulfillment as the cause of Storefront payment slowness.

This chapter names degraded modes, shedding rules, and cascade denials that protect Chapter 3 journeys. Degraded success is accounted as Storefront burn, not as a good accept. Unbounded retries fail the check. The lab does not inject live overload. Tenant quota and unit-cost scaling remain Platform and DevOps; they are not this shed.

1. An unsafe retry storm

A weak record says:

```
worker: order-worker
retry_limit: unbounded
action: accept-anyway
accounting: success
page_cause: fulfillment-oncall
```

It keeps accepting orders while payment is slow. It treats those accepts as good events. It retries until shared capacity takes Fulfillment with it. It pages `fulfillment-oncall` as if dispatch were the cause. "The accept graph is green" may describe a counter. It cannot complete a degradation design.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 9 baseline:

```
make chapter-09-baseline
```

The command succeeds when it detects the intended unsafe design:

```
chapter 09 baseline: unbounded payment retries cascading into fulfillment
correctly detected
```

The fixture leaves retries unbounded, accepts work the worker cannot finish, counts those accepts as success, and pages Fulfillment as the payment cause. Availability theater continues: green accept graphs, exhausted budgets on both journeys, informal heroics on the wrong rotation.

That inversion drives the chapter.

2. The production model: named mode, shed, and cascade deny

Theory — Degradation before cascade

This model enables Northwind to refuse new accepts and bound order-worker retries when payment is overloaded, rather than amplifying retries until Fulfillment burns and the accept graph stays green.

A degraded mode is user-visible and accounted as burn

Degradation is a named state with a user-visible behavior. Checkout refuses new orders, or it shows that payment cannot complete. That is honest. A 200 on accept while payment never finishes is silent drop. Silent drop is availability theater wearing a success counter.

The mode must name the Chapter 1 journey, the Chapter 2 **SLI (Service Level Indicator)**, and how remaining budget accounts the events. Teaching mode: payment-overload-degraded on accept-and-complete-order / order_success_ratio, user-visible checkout-refuses-new-orders, accounting journey-burn. Accepted-but-unpaid work is a bad event. It is not a good accept. The mode file must not emit remaining_budget or accounting: success. Those would let a degraded path approve itself, the same self-approval Chapter 3 refused.

Best Practice: Name the degraded mode, the user-visible behavior, and the journey-burn accounting before shedding a single request.

Production Practice: Payment overload degrades Storefront. It does not page Fulfillment as the cause, and it does not count in-flight accepts as success.

Shedding consumes the Chapter 8 retry budget; it is not quota and not scaling

Chapter 8 already named payment retry_budget: 1 and fail-closed. Shedding must consume that number. retry_limit on order-worker equals 1. A larger limit is retry amplification. unbounded is not a limit.

When the budget is exhausted, Storefront refuses new accepts. That is backpressure at the edge so the worker pile cannot grow. It is not Platform tenant quota. Quota decides how much environment a tenant may consume. It is not DevOps unit-cost scaling. Adding workers so everyone can retry harder is cost policy, not cascade control. The shed record must name those two distinctions so later chapters cannot replace the rule with a quota ticket or a scale-out.

Warehouse keeps its own Chapter 8 contract. Copying the Storefront payment shed onto dispatch is the same class of copy Chapter 3 forbade for SLO targets. This chapter's teaching storm is payment. Do not invent a second identical shed to look complete.

Cascade deny: payment slowness is not a Fulfillment page

A retry storm that also calls warehouse, or that saturates shared capacity, can burn dispatch-fulfillment as a side effect. That side effect is still not “Fulfillment caused payment slowness.” Chapter 5 pages Fulfillment for `dispatch_success_ratio` burn. It must not gain a new page that says payment slowness is a dispatch incident.

The cascade contract denies three moves:

- amplifying retries past the Chapter 8 payment budget;
- paging fulfillment-oncall as the cause of payment overload;
- treating dispatch burn from that storm as a reason to keep accepting Storefront orders.

Email stays out of this storm. Chapter 8 already skip-and-records it. Shedding payment does not promote notification-service onto the critical path.

The lab does not inject live overload. Retry limits and denials are local fixtures, like Chapter 3's event counts.

3. Publish modes, shedding, and cascade denials

The completed Chapter 9 model uses three files:

```
degradation/modes.yaml
degradation/shedding.yaml
degradation/cascade.yaml
```

The separation is deliberate. Modes name user-visible degraded states and burn accounting. Shedding binds order-worker to the Chapter 8 retry budget and refuses new accepts. Cascade denials keep Fulfillment from being paged as the payment cause. None of the files may store remaining budget or stamp degraded work as success.

Practice — Name the degraded mode and account it as Storefront burn

Refuse silent drop. Do not emit remaining budget. Do not count accepted-but-unpaid as success.

Open `degradation/modes.yaml`. The teaching mode is `payment-overload-degraded`. Journey is `accept-and-complete-order`. **SLI** is `order_success_ratio`. Trigger is `payment-retry-budget-exhausted`. User-visible behavior is `checkout-refuses-new-orders`. Accounting is `journey-burn`. Worker is `order-worker`. Owner is `reliability-program`. If accounting: `success` or `remaining_budget` appears, the checkpoint fails.

Practice — Bound order-worker retries to the Chapter 8 payment budget and shed new accepts

Do not replace the shed with tenant quota or unit-cost scaling.

Open `degradation/shedding.yaml`. The rule `shed-storefront-payment-overload` consumes provider payment, worker order-worker, journey accept-and-complete-order, action refuse-new-accepts, and `retry_limit: 1`. That limit must equal the Chapter 8 payment `retry_budget`. `distinct_from` lists `platform-tenant-quota` and `devops-unit-cost-scaling`. If `retry_limit` is unbounded or the action is `accept-anyway`, the checkpoint fails.

Inspect each shed with three questions:

1. Does the retry limit match the Chapter 8 contract, or did “try harder” rewrite it?
2. What does the user see when the shed fires—and is that counted as burn?
3. Would Fulfillment still be paged as the cause if this payment row were deleted?

Practice — Deny paging Fulfillment as the payment cause

Dispatch pages remain for dispatch burn. They must not become the payment incident.

Open `degradation/cascade.yaml`. Denial `deny-payment-retry-into-fulfillment` names source provider payment, source journey accept-and-complete-order, `must_not_page: fulfillment-oncall`, and `must_not_burn: dispatch-fulfillment` as a reason to keep accepting. If `page_cause: fulfillment-oncall` appears, the storm has been blamed on the wrong rotation.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-09-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 09 checkpoint: shed, named burn accounting, and cascade denial
verified
```

The audit validates the three Chapter 9 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- a named degraded mode exists for payment overload on accept-and-complete-order;
- user-visible behavior is not silent drop;
- accounting is journey-burn, not success, and remaining budget is not emitted;
- order-worker retry limit equals Chapter 8 payment retry_budget;
- new accepts are refused when that budget is exhausted;
- the shed is distinct from Platform tenant quota and DevOps unit-cost scaling;
- Fulfillment is not paged as the cause of payment slowness; and
- unbounded retries and accept-anyway fail.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not inject live overload. It does not prove that one retry or refuse-new-accepts is the right commercial shed. Those are judgment claims. The review triggers exist so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 9 evidence
Mechanism evidence	Schemas and the degradation evaluator operated successfully.
Decision evidence	Named mode, shed action, retry limit, and cascade denials are explicit.
Outcome evidence	Degraded accepts are accounted as Storefront journey-burn, not as success.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 9 creates decision evidence about how Northwind fails small. Pretending that the local checkpoint proved a production retry storm would weaken every later incident and game day.

4. Test the design under failure

Cumulative reliability failure — Unbounded payment retries cascade into Fulfillment

Practice — Shed, name the mode, stop amplification, account Storefront burn

Drop unbounded. Stop paging Fulfillment as the cause. Stop counting accepts as success.

The baseline fixture contains this model:

```
worker: order-worker
retry_limit: unbounded
action: accept-anyway
accounting: success
page_cause: fulfillment-oncall
```

The problem is not merely a missing timeout. Chapter 8 already named 2s and retry budget 1. The worker ignores that budget, Storefront keeps accepting, the accept graph stays green, and Fulfillment is paged for a payment slowness it did not cause. Both remaining budgets can exhaust while leadership sees “orders accepted.” That is the book’s cumulative reliability failure: availability theater plus informal heroics, now as a retry loop.

Severity: high; one slow payment becomes two journey outages and a mis-paged rotation.

Plausible harm: checkout appears to work; payment never finishes; dispatch fails under shared load; Fulfillment on-call debugs the wrong cause.

Potential blast radius: every worker taught to retry harder; every green accept counter treated as an SLO.

Bounded by: later portfolio incident command. None repairs a shed that is “accept anyway.”

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** accept-and-complete-order is at risk on Storefront. dispatch-fulfillment may be harmed as a cascade, not as the cause.
- **Error budget:** Degraded accepts consume Storefront remaining budget. Hiding them as success forges outcome evidence. Not yet a new freeze rule.
- **Human system:** Applicable as load on Chapter 6 rotations. Paging Fulfillment for payment slowness is informal heroics under a provider name.
- **Portfolio recovery:** Not yet produced. A green accept graph cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling retries unbounded encourages three controls: keep accepting so the storefront looks live, count those accepts as good so the **SLI** stays green, and page whoever is downstream when shared capacity fails. Those moves may be sincere. They do not answer which journey the customer lost, which remaining budget burns, or which rotation Chapter 6 designed for payment.

Silent drop makes mechanism evidence (accept count) look like outcome evidence. `page_cause: fulfillment-oncall` makes Chapter 5's dispatch pages ornamental as a blame label. `accept-anyway` makes Chapter 8's fail-closed payment contract ornamental.

Correction

The completed model does not leave retries unbounded. It names `payment-overload-degraded`, shows checkout refusing new orders, accounts those events as Storefront journey-burn, sets order-worker retry limit to the Chapter 8 payment budget, refuses new accepts, and denies paging Fulfillment as the payment cause.

That correction changes later decisions:

- Chapter 10 must command a spanning incident without closing because accepts looked green, and without treating Fulfillment as the payment owner.
- Chapter 13 must exercise dependency loss with this shed in place, not with an unbounded retry drill that stands in for cascade control.
- Chapter 14 must not treat restored accept volume as **Evidence of portfolio recovery** if payment still cannot complete.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary load-generator command would not make it more practical.

5. Production reality

Common cascade errors

Retrying past the named budget

Chapter 8's `retry_budget: 1` is the limit. A larger number is amplification.

Accepting work the worker cannot finish

A green accept counter is not a completed order.

Counting degraded success as a good event

Accounting is journey-burn. A stored success flag is a slide.

Paging Fulfillment for Storefront payment slowness

Cause and cascade are different. Chapter 5 already split those pages.

Replacing the shed with tenant quota or scale-out

Quota and unit cost are inherited jobs. They do not refuse a retry storm.

Silent drop without a named mode

If the user cannot see the degradation, remaining budget cannot either.

Copying the payment shed onto warehouse to look complete

Dispatch has its own contract. This chapter's teaching failure is payment amplification.

6. What changed

Before	After
order-worker retries were unbounded.	Retry limit equals Chapter 8 payment retry_budget: 1.
Storefront accepted work it could not finish.	New accepts are refused when the budget is exhausted.
Accepted-but-unpaid counted as success.	The degraded mode accounts those events as Storefront journey-burn.
Fulfillment was paged as the payment cause.	Cascade denial forbids fulfillment-oncall as the payment page.
Quota or scale-out stood in for a shed.	The shed is distinct from tenant quota and unit-cost scaling.
A valid schema could appear to prove cascade control.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable degradation contract that later incidents and game days can consume without retrying payment into a portfolio outage or hiding harm behind a green accept graph.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Degradation-and-shedding policy	degradation/modes.yaml and degradation/shedding.yaml	They retain the named mode, user-visible behavior, burn accounting, and retry limit later incidents must not replace with “try harder.”
Cascade-failure contract	degradation/cascade.yaml	It retains the denial that payment slowness is not a Fulfillment page and not a reason to keep accepting.

These artifacts should change when Northwind’s workers, payment retry budget, or user-visible degraded behavior materially change—not whenever a dashboard adds an accept counter.

What You Learned

Northwind sheds and degrades before a local payment slowness becomes every journey’s outage. order-worker retries are bounded by the Chapter 8 contract. New accepts are refused. Degraded work is user-visible and accounted as Storefront burn, not as success. Fulfillment is not paged as the payment cause. Tenant quota and unit-cost scaling are not the shed. Schema checks can prove structural completeness within declared scope. They cannot inject live overload. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Degrade catalog-browse without copying the payment shed

A storefront engineer wants catalog search to retry unbounded “because browse already has tickets, and if search is slow we should try harder until Fulfillment is fine.”

Extend the Chapter 9 model without adding incident command yet:

1. Decide whether catalog-browse may retry unbounded, given Chapters 1–3 and the Chapter 8 search-contract practice.
2. Name a user-visible degraded behavior that is not a renamed checkout-refuses-new-orders.
3. Account any degraded browse events without emitting `remaining_budget` or `accounting: success`.
4. Do not page `fulfillment-oncall` as the cause of a Storefront search slowness.
5. Identify one observation that would falsify the shed—for example accepts still counted as success while retries are unbounded.
6. Explain which material change would trigger review of the mode, not just the worker replica count.

Do not copy the payment refuse-new-accepts row and rename it. Catalog reads have different criticality, workers, and freeze consequences from checkout. Your durable output is the mode and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 9 capability when you can explain why unbounded retries are not a contract, shed new accepts against the Chapter 8 payment budget, name a user-visible mode accounted as journey-burn, refuse to page Fulfillment as the payment cause, distinguish quota and scaling from shedding, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Cascade is bounded in the model. A multi-service incident is still commanded as one failed change on one path. Inherited DevOps Chapter 12 can coordinate one production change. A payment slowness that also harmed dispatch still gets one informal commander in Slack.

Chapter 10 defines portfolio incident command so Storefront green cannot close Fulfillment, and so the right rotations and Chapter 4 freeze join the trace.

10. Command Incidents Across Services and Tenants

Chapter 9 can shed Storefront when payment is overloaded and refuse to page Fulfillment as the cause. A payment slowness that also harmed dispatch still gets one informal commander in Slack. Inherited DevOps Chapter 12 can coordinate one failed production change. That is a single-path capability. It is not portfolio command.

The production question is now:

How is an incident that spans Storefront, Fulfillment, and the platform commanded, and what evidence is portfolio evidence rather than one-path recovery?

Without that design, single-path command mis-assigns owner, freeze, and recovery proof. Closing because Storefront `order_success_ratio` looks green leaves `dispatch-fulfillment` down. Treating a **platform-product** incident as a **tenant-application** incident pages Storefront for job-time. Chapter 4 already froze Storefront releases and fleet step `storage-1-0-to-2-0`. An incident that ignores those actions makes freeze ornamental. Chapter 6 already designed living primaries. Slack-as-commander is Slack-as-primary under a louder name.

This chapter defines multi-service command, distinct roles, and executed traces. It **consumes** the inherited DevOps recovery-evidence shape and Platform support kinds. It does not reteach one-change coordination or platform support. A trace cannot emit `status: recovered`. That language belongs to later **Evidence of portfolio recovery**, not to closing a ticket.

1. An unsafe one-path close

A weak record says:

```
commander: slack
close_on: order_success_ratio
status: closed
affected: [accept-and-complete-order]
```

It reuses DevOps one-change command. It closes on a Storefront **SLI (Service Level Indicator)** while dispatch is still failing. It never pages `platform-oncall` for a platform-product issue. “Orders look green” may describe one path. It cannot complete a portfolio incident.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 10 baseline:

```
make chapter-10-baseline
```

The command succeeds when it detects the intended unsafe command:

```
chapter 10 baseline: one-path close while dispatch fails correctly
detected
```

The fixture sets commander to Slack, closes on `order_success_ratio` and `terminal_order_outcomes`, omits `dispatch-fulfillment`, skips Chapter 4 freeze join, treats catalog contact `storefront-oncall` as the system, and routes a platform-product incident onto Storefront. One green graph has been treated as the portfolio. Informal heroics have been treated as command. That inversion drives the chapter.

2. The production model: commander, scope, freeze join, support kind

Theory — Portfolio incident command

This model enables Northwind to command a spanning incident with named roles, both journeys, the right on-call systems, and Chapter 4 freeze, rather than closing a Slack thread because Storefront looks green.

One-path recovery evidence cannot close the portfolio

DevOps Chapter 12 already named recovery evidence for one failed change: `desired_actual_agreement`, `healthy_consecutive_samples`, and `terminal_order_outcomes`. This chapter **consumes** those fields. It does not rewrite them into a new incident-management product. They remain insufficient to close a portfolio incident. So is `order_success_ratio` alone.

If dispatch is failing, Storefront green is availability theater. If only Storefront was listed, Fulfillment was never in the incident. Closing then is a one-path close.

Best Practice: Name every affected Chapter 1 journey and every Chapter 6 on-call system before anyone talks about close.

Production Practice: A spanning tenant-application incident lists `accept-and-complete-order` and `dispatch-fulfillment`, pages `storefront-oncall-system` and `fulfillment-oncall-system`, and stays open while dispatch is failing.

Commander is a role, not whoever answered Slack

Commander and operator are distinct. Commander owns the incident clock and whether close is even discussable. Operator executes. `slack`, `chat-history`, and `whoever-answered` cannot be commander. Those are the same forbidden primaries Chapter 6 already refused.

The commander for a tenant-application spanning incident is a living primary from Chapter 6, not a catalog contact. `storefront-oncall` is who the catalog says to call. `storefront-oncall-system` is the system. Treating the contact as the system reopens Chapter 6.

Cadence is a teaching value: 30m. It is not an industry constant. It exists so later learning and game days inherit a reviewable clock rather than “we were in Slack.”

Freeze joins Chapter 4; it is not invented in the war room

Storefront remaining budget is already exhausted. Chapter 4 already recorded `freeze-storefront-releases` and `freeze-fleet-storage-upgrade`. A spanning incident that includes `accept-and-complete-order` must **join** those action ids. It must not copy Platform freeze window, cohort, or rollback onto the trace. It must not invent `platform-upgrade-freeze` as the reason.

If `freeze_join` is empty while those actions exist, exhausted budget cannot stop change during the incident. Chapter 4 becomes a file nobody opened.

Platform-product and tenant-application are different incidents

Inherited Platform support kinds are platform-product and tenant-application. Consume them. Do not invent a third kind called “whatever is on fire.”

A platform-product incident about time-to-first-environment routes to platform-oncall-system. It is a **job-time budget** issue, not a portfolio **SLO (Service Level Objective)** close. It must not land on Storefront because “they are already up.” A tenant-application spanning incident must not be labeled platform-product to dodge freeze join.

The lab does not run a real incident-management tool. Traces are local fixtures.

3. Publish command, roles, and traces

The completed Chapter 10 model uses three files:

```
incidents/command.yaml
incidents/roles.yaml
incidents/traces.yaml
```

The separation is deliberate. Command names forbidden commanders, cadence, and which evidence cannot close the portfolio. Roles keep commander distinct from operator. Traces are executed incidents: spanning tenant-application and platform-product. None of the files may emit status: recovered.

Practice — Consume inherited one-path evidence as insufficient

Keep desired_actual_agreement, healthy_consecutive_samples, terminal_order_outcomes, and order_success_ratio off the close path.

Open incidents/command.yaml. Owner is reliability-program. Forbidden commanders include slack, chat-history, and whoever-answered. Cadence is 30m. insufficient_close_evidence lists the inherited DevOps recovery fields and order_success_ratio. If those inherited ids are missing, later traces will close on one-path evidence and call it portfolio recovery.

Practice — Keep commander distinct from operator; Slack cannot command

A single hero role is unofficial command.

Open incidents/roles.yaml. Role commander may discuss close. Role operator may not. If the only role is whoever-answered, the checkpoint fails.

Practice — Trace both journeys, both on-call systems, and Chapter 4 freeze

Storefront green cannot close Fulfillment. Platform-product cannot land on Storefront.

Open incidents/traces.yaml. Teaching traces:

1. spanning-payment-and-dispatch — support_kind: tenant-application, commander storefront-primary-a, journeys accept-and-complete-order and dispatch-fulfillment, systems storefront-oncall-system and fulfillment-oncall-system, freeze join freeze-storefront-releases and freeze-fleet-storage-upgrade, dispatch state failing, status open, as_of quoted as RFC 3339.
2. platform-product-job-time — support_kind: platform-product, commander platform-primary-a, job-time time-to-first-environment, system platform-oncall-system, status open.

Inspect each trace with three questions:

1. If Storefront's accept graph were deleted, would this incident still exist?
2. Which Chapter 6 system is paged—and is it a catalog contact wearing a system id?
3. Does Chapter 4 freeze still apply, or was freeze left in the policy file?

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-10-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 10 checkpoint: portfolio command, freeze join, and split support
kinds verified
```

The audit validates the three Chapter 10 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- inherited one-path recovery evidence is listed as insufficient to close;
- order_success_ratio alone cannot close;
- Slack cannot be commander;
- commander and operator are distinct roles;
- the spanning trace names both Chapter 1 journeys and both tenant on-call systems;
- dispatch failing keeps status open;
- Chapter 4 freeze actions are joined by id;
- catalog contacts are not treated as on-call systems;
- platform-product routes to platform-oncall-system; and
- status: recovered is refused.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not run a real incident-management tool. It does not prove that 30 minutes is the right commercial cadence. Those are judgment claims. The review trigger exists so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 10 evidence
Mechanism evidence	Schemas and the incident evaluator operated successfully.
Decision evidence	Command model, roles, support kinds, freeze join, and insufficient close evidence are explicit.
Outcome evidence	A spanning trace cannot close while dispatch fails; one-path SLI close is denied.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 10 creates decision evidence about how a spanning failure is commanded. Pretending that the local checkpoint proved a real war room would weaken later learning, game days, and fail-over.

4. Test the design under failure

Connected consequence — One-path close on order success while dispatch fails

Practice — Re-open under portfolio command

Page both tenant systems, join freeze, route platform-product to the platform rotation, and drop Slack as commander.

The baseline fixture contains this model:

```
commander: slack
close_on: [order_success_ratio, terminal_order_outcomes]
status: closed
affected: [accept-and-complete-order]
oncall: storefront-oncall
platform_incident_on: storefront-oncall-system
```

The problem is not merely a missing field. DevOps one-change evidence has been treated as portfolio close. Chapter 9's cascade can still harm dispatch while the ticket says done. Chapter 4 freeze was never joined. A platform-product wait-time incident pages Storefront. Informal heroics continue under an inherited recovery field.

Severity: high; Fulfillment stays down, freeze does not apply, and the wrong rotation debugs job-time.

Plausible harm: leadership sees “incident closed”; warehouse dispatch remains failing; Storefront is woken for environment wait time.

Potential blast radius: every one-path green graph used as a close; every Slack thread treated as command.

Bounded by: later learning-program verification. None repairs a close that ignored a journey.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** Both accept-and-complete-order and dispatch-fulfillment are at risk. Closing on the first does not restore the second.
- **Error budget:** Chapter 4 freeze still applies to Storefront releases and storage-1-0-to-2-0. The trace must join those actions.
- **Human system:** Applicable as Chapter 6 rotations. Slack-as-commander is informal heroics.
- **Portfolio recovery:** Not yet produced. `terminal_order_outcomes` on one path cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling Slack the commander encourages three controls: close when the first graph recovers, skip the journey that is still failing, and send platform-product work to whoever is already in the channel. Those moves may be sincere. They do not answer which journeys are still failed, which freeze still applies, or which on-call system Chapter 6 designed.

Closing on `order_success_ratio` makes outcome evidence a single **SLI**. Omitting freeze join makes Chapter 4 a brochure. Routing platform-product to Storefront reopens Chapter 2’s job-time confusion as an incident path.

Correction

The completed model does not close the spanning incident. It names a living primary as commander, lists both journeys and both tenant on-call systems, joins the Chapter 4 freeze actions, keeps status open while dispatch is failing, routes platform-product to `platform-oncall-system`, and treats inherited one-path evidence as insufficient to close.

That correction changes later decisions:

- Chapter 11 must require a learning record or expiring waiver for this complete command, not a blog post that says “be more careful.”
- Chapter 13 must exercise the on-call page path as these systems, not as a Slack channel discovered during the game day.
- Chapter 14 must not treat one-path `terminal_order_outcomes` as **Evidence of portfolio recovery**.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary chat-ops bot would not make it more practical.

5. Production reality

Common incident-command errors

Reusing DevOps one-change close as portfolio close

`terminal_order_outcomes` is inherited one-path evidence. It is not a portfolio close.

Closing because Storefront is green

Dispatch is a second journey. Green on one **SLI** is theater if the other is failing.

Slack as commander

A channel is not a role. Chapter 6 already refused it as primary.

Treating a catalog contact as the on-call system

`storefront-oncall` is a label. `storefront-oncall-system` is the system.

Skipping freeze join

If Chapter 4 already froze the digest and the fleet, the incident does not get to unfreeze by omission.

Routing platform-product to Storefront

Job-time is a **job-time budget**. It pages the platform rotation.

Emitting status: recovered on the trace

That forges later recovery evidence. This chapter may keep an incident open or contained. It may not declare the portfolio recovered.

Copying the spanning payment-and-dispatch trace onto every ticket

A platform-product incident is a different support kind. Copying journeys onto it recreates tenant-application coverage for job-time.

6. What changed

Before	After
Commander was Slack.	Commander is a living Chapter 6 primary; Slack is forbidden.
Close used <code>order_success_ratio</code> and <code>terminal_order_outcomes</code> .	Inherited one-path evidence is insufficient to close.
Only Storefront was listed.	The spanning trace names both journeys and both tenant on-call systems.
Dispatch was still failing and the ticket was closed.	Status stays open while dispatch is failing.
Freeze stayed in the policy file.	The trace joins freeze-storefront-releases and freeze-fleet-storage-upgrade.
Platform-product landed on Storefront.	Platform-product pages platform-oncall-system.

What changed was not merely three YAML files. Northwind now has a reviewable command contract that later learning, game days, and fail-over can consume without closing a Slack thread because one graph recovered.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Portfolio incident-command model	incidents/command.yaml and incidents/roles.yaml	They retain forbidden commanders, distinct roles, cadence, and insufficient close evidence later chapters must not replace with Slack.
Executed multi-service trace	incidents/traces.yaml	It retains spanning tenant-application command, freeze join, and a split platform-product trace.

These artifacts should change when Northwind's journeys, on-call systems, freeze actions, or support kinds materially change—not whenever a new chat channel is created.

What You Learned

A spanning incident is commanded as a portfolio, not as one failed change on one path. Inherited DevOps recovery evidence cannot close it. Storefront green cannot close Fulfillment. Chapter 4 freeze joins by id. Commander is a living primary, not Slack. Catalog contacts are not on-call systems. Platform-product pages the platform rotation. Schema checks can prove structural completeness within declared scope. They cannot run a real incident-management tool. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Command a catalog-browse incident without copying the spanning trace

A storefront engineer wants to close the payment-and-dispatch incident “because catalog browse recovered, and that is what customers see,” and to page Storefront for a catalog-freshness stall “because they are already on.”

Extend the Chapter 10 model without adding a learning program yet:

1. Decide whether catalog-browse is in the spanning tenant-application incident, a separate tenant-application incident, or out of scope given Chapters 1–3.
2. Decide whether catalog-freshness is platform-product job-time and which Chapter 6 system it pages.
3. Name commander and operator without using Slack or copying storefront-primary-a from the payment trace as if browse were checkout.
4. State what would make close invalid—for example dispatch still failing.
5. Identify one observation that would falsify the command—for example freeze join omitted while Storefront budget is exhausted.
6. Explain which material change would trigger review of the command model, not just the chat channel.

Do not copy the payment-and-dispatch trace and rename it. Catalog reads have different criticality, support kind, and freeze consequences from checkout plus dispatch. Your durable output is the command decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 10 capability when you can explain why one-path close is invalid, name commander and both tenant journeys for a spanning incident, join Chapter 4 freeze, route platform-product to the platform rotation, refuse Slack as commander, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Incidents can be commanded. Learning is still a document with no verified follow-through. Postmortems are optional. Action items have no owner. The same payment retry cascade can return.

Chapter 11 adopts a learning-program contract so every complete Chapter 10 incident has a record or an expiring waiver, and so actions have owner, due date, and verification that is not the record approving itself.

11. Turn Incidents into a Reliability Learning Program

Chapter 10 can command a spanning incident without closing because Storefront looks green. Command is not learning. Postmortems are optional. Action items have no owner. The same Chapter 9 payment retry cascade can return. A polished write-up that says “be more careful” is blameless theater.

The production question is now:

What learning program turns incidents into owned, verified change, rather than into blameless theater?

Without that decision, a template without verification is a blog post. Blame without a system produces silence. Action volume without a bound recreates Chapter 7 toil. Inherited DevSecOps already requires that verification be produced independently of the record that claims it. A learning file that stamps verified: true on itself is the same self-approval Chapter 3 refused for remaining budget. This chapter is not eradication, and it is not a second DevOps one-change retrospective.

This chapter adopts a learning-program contract: which Chapter 10 incidents require a record, how actions are owned and dated, what independent verification looks like, and when the program itself is failing. The lab cannot prove that an organization actually learned.

1. An unsafe learning write-up

A weak record says:

```
incident: spanning-payment-and-dispatch
actions:
  - be-more-careful
verified: true
```

It has a polished postmortem. It names no owner, no due date, and no verification independent of the write-up. The Chapter 9 cascade has no owned change. The platform-product trace has neither a record nor an expiring waiver. “We talked about it” may describe last week’s meeting. It cannot complete a learning program.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 11 baseline:

```
make chapter-11-baseline
```

The command succeeds when it detects the intended unsafe decision:

```
chapter 11 baseline: hortatory postmortem without verified action
correctly detected
```

The fixture stores a hortatory action, lets the record verify itself, omits owner and due date, leaves the Chapter 9 cascade without a verified action, and leaves platform-product-job-time uncovered. A document has been treated as a control. Informal heroics have been treated as follow-through. That inversion drives the chapter.

2. The production model: record or waiver, owned action, independent verification

Theory — Learning as a control

This model enables Northwind to turn commanded incidents into owned, dated, independently verified change, rather than into a blameless blog post that cannot stop a repeated cascade.

Every commanded incident is covered, or explicitly waived

Chapter 10 produced two traces: spanning-payment-and-dispatch and platform-product-job-time. Both are commanded incidents. Silence is not coverage. Each id needs a record or a waiver. A waiver is not a missing file. It names owner, expires_at, and a removal path, the same shape Chapter 4 already required for error-budget exceptions.

Teaching coverage: the spanning tenant-application incident gets a record with contributing factors. The platform-product job-time incident gets a waiver with expiry. That waiver is a reviewable choice, not “job-time does not count.” Copying the spanning record onto job-time would pretend checkout-plus-dispatch contributing factors apply to time-to-first-environment.

The record is blameless contributing-factor analysis, not a named human as the cause. A dashboard, alert, page, or postmortem is still only mechanism evidence. It is not proof that a journey kept its **SLO (Service Level Objective)**.

Best Practice: Require a record or an expiring waiver for every Chapter 10 incident id before calling the program complete.

Production Practice: The spanning cascade incident cannot be waived away while Chapter 9 shedding is the change that must be verified.

Actions have owner, due date, and verification the record cannot emit

“Be more careful” is hortatory. It is forbidden as an action id and as a change. An action names an owner, a quoted RFC 3339 due_at, and a verification producer that is not the record. Inherited DevSecOps independent_producer_required: true is consumed here. If verification.producer is the record id, the write-up has approved itself.

The teaching action verify-payment-retry-shed addresses chapter-09-cascade. Its producer is the Chapter 9 shed rule shed-storefront-payment-overload. That is independent of record-spanning-payment-and-dispatch. The action file must not stamp verified: true. Outcome evidence is that the producer exists and is distinct, not a boolean the action stores.

A repeated Chapter 9 cascade with no such verified action means the learning program has failed, even if the postmortem is excellent.

Learning is bounded by toil, and is not eradication

Unbounded action lists recreate Chapter 7 homework. The program **joins** northwind-toil-bound. It does not invent a second hour inventory. If the join is missing, later game days will attach infinite follow-up to every fixture.

This program is not DevSecOps eradication and not **Evidence of restored trust**. It is not a rewrite of DevOps Chapter 12's one-change retrospective. Those remain inherited. Substituting them for verified reliability change leaves the cascade free to return.

The lab cannot prove that an organization actually learned. Records, waivers, and action joins are local fixtures.

3. Adopt the program, records, and actions

The completed Chapter 11 model uses three files:

```
learning/program.yaml
learning/records.yaml
learning/actions.yaml
```

The separation is deliberate. The program names required incidents, forbidden hortatory labels, independent verification, the toil join, and when the program fails. Records and waivers cover Chapter 10 ids. Actions carry owner, due date, and a producer that is not the record. None of the files may emit `verified: true` or `learned: true`.

Practice — Forbid hortatory actions and self-verification

Keep be-more-careful off the action list. Do not let a record verify itself.

Open `learning/program.yaml`. Owner is `reliability-program`. `required_incidents` lists both Chapter 10 trace ids. `forbidden_actions` includes `be-more-careful`. `verification_independent` is `true`. `toil_join` is `northwind-toil-bound`. `program_fails_when` is `repeated-cascade-without-verified-action`. Forbidden substitutes include `devsecops-eradication` and `devops-one-change-retrospective`.

Practice — Cover every Chapter 10 incident with a record or an expiring waiver

Do not copy the spanning record onto job-time.

Open `learning/records.yaml`. `record-spanning-payment-and-dispatch` is a record for the spanning incident, with contributing factors that include the retry cascade and one-path command. `waiver-platform-product-job-time` is a waiver for the platform-product trace, owner `reliability-program`, quoted `expires_at`, and a removal path to write a record or a verified action. If the waiver has no expiry, it is silence with nicer YAML.

Practice — Own the cascade change and verify it from Chapter 9 shedding, not from the write-up

A missing cascade action fails the program even when the postmortem exists.

Open `learning/actions.yaml`. `verify-payment-retry-shed` names incident spanning-`payment-and-dispatch`, owner `reliability-program`, quoted `due_at`, change `consume-chapter-09-payment-shed`, addresses: `chapter-09-cascade`, and verification producer `shed-storefront-payment-overload` with `independent_of_record: true`. If `due_at` is missing, the action is a slogan. If the producer is the record id, the slogan has a checkbox.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-11-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 11 checkpoint: records or waivers, owned actions, and independent
verification verified
```

The audit validates the three Chapter 11 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- both Chapter 10 incident ids have a record or an expiring waiver;
- waivers name owner, expiry, and removal path;
- hortatory `be-more-careful` cannot be an action;
- every action has owner and `due_at`;
- verification is independent of the record, consuming DevSecOps `independent_producer_required`;
- a Chapter 9 cascade action exists and is independently verified;
- the program joins the Chapter 7 toil bound;
- `verified: true` and `learned: true` are not stored on the files; and
- eradication and one-change retrospective are not substitutes.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint cannot prove that an organization actually learned. It does not prove that 30 days is the right commercial review cadence. Those are judgment claims. The review trigger exists so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 11 evidence
Mechanism evidence	Schemas and the learning evaluator operated successfully.
Decision evidence	Program contract, coverage of Chapter 10 ids, forbidden hortatory labels, and toil join are explicit.
Outcome evidence	Cascade follow-through is joined to an independent Chapter 9 producer, not emitted as verified: true.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 11 creates decision evidence about how incidents become change. Pretending that the local checkpoint proved a culture learned would weaken later game days and fail-over.

4. Test the decision under failure

Independent control failure — Polished postmortem with hortatory actions

Practice — Replace hortatory actions with owned, dated, independently verified change

Cover the platform-product incident. Verify the cascade from the shed, not from the write-up.

The baseline fixture contains this model:

```
incident: spanning-payment-and-dispatch
actions:
  - be-more-careful
verified: true
```

The problem is not merely a missing field. A blameless document exists. The Chapter 9 cascade can repeat. The platform-product incident has no waiver. The record is the verifier. That is availability theater applied to learning: the slide is green, the control did not change production.

Severity: high; the same retry storm returns, and Chapter 10 command becomes a meeting that produces prose.

Plausible harm: payment overload cascades again; freeze is re-joined in Slack; job-time incidents vanish without expiry.

Potential blast radius: every polished template; every “be more careful” backlog.

Bounded by: later game-day join to these actions. None repairs a program that cannot fail.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** The spanning record is about accept-and-complete-order and dispatch-fulfillment. Hortatory text does not change those journeys.
- **Error budget:** Not a new freeze. Chapter-local implication: unverified cascade follow-through leaves Chapter 4 freeze as a recurring surprise.
- **Human system:** Applicable as load. Unbounded actions recreate Chapter 7 toil on the Chapter 6 rotations.
- **Portfolio recovery:** Not yet produced. A postmortem cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling the write-up complete encourages three controls: skip owners because the prose is good, skip dates because “we all know,” and tick verified because the record exists. Those moves may be sincere. They do not answer which Chapter 9 producer changed, which Chapter 10 id is uncovered, or what would falsify the program.

Self-verification makes outcome evidence a constant. Silence on platform-product-job-time makes Chapter 10’s second trace ornamental. Hortatory actions make blamelessness a tone rather than a system.

Correction

The completed model does not keep be-more-careful. It covers both Chapter 10 ids, owns a cascade action dated and verified from the Chapter 9 shed, joins the Chapter 7 toil bound, forbids the record as verifier, and treats a repeated cascade without that action as program failure.

That correction changes later decisions:

- Chapter 13 must feed game-day results into these actions, not into a new hortatory list.
- Chapter 14 must not treat a completed postmortem as **Evidence of portfolio recovery**.
- Regional architecture in Chapter 12 still has to be written; learning does not invent a second region.

The decision is practical because it changes the production contract across the rest of the book. Adding an arbitrary retrospective workshop would not make it more practical.

5. Production reality

Common learning errors

Hortatory actions

“Be more careful” cannot be verified. A shed rule can.

Letting the record verify itself

Independent producer is the inherited evidence-map rule. A checkbox on the write-up is self-approval.

Optional postmortems

Silence on a Chapter 10 id is not a waiver. A waiver expires.

Copying one record onto every incident

Job-time is not checkout-plus-dispatch. Coverage is per id.

Unbounded action volume

Without the Chapter 7 join, learning becomes the next toil inventory.

Substituting eradication or a one-change retrospective

Those are inherited programs with different promises. They do not verify a portfolio cascade action.

Storing learned: true

The program can fail. A stored success flag cannot.

6. What changed

Before	After
The action was be-more-careful.	The cascade action is owned, dated, and forbidden to be hortatory.
The record stamped verified: true.	Verification producer is the Chapter 9 shed, independent of the record.
Platform-product had no coverage.	It has an expiring waiver with owner and removal path.
The spanning incident had prose only.	It has a contributing-factor record plus a verified cascade action.
Action volume was unbounded.	The program joins northwind-toil-bound.
A valid schema could appear to prove learning.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable learning control that later game days and fail-over can join without treating a blog post as follow-through.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Learning-program contract	learning/program.yaml	It retains required incidents, forbidden hortatory labels, independent verification, toil join, and program-failure conditions later chapters must not replace with a template.
Action-verification register	learning/records.yaml and learning/actions.yaml	They retain coverage of Chapter 10 ids and the cascade action whose producer is not the record.

These artifacts should change when Northwind's incident ids, cascade controls, or toil bound materially change—not whenever a new retrospective template is adopted.

What You Learned

Learning is a reliability control: record or expiring waiver per commanded incident, owned dated actions, and verification independent of the write-up. Hortatory actions fail. A repeated Chapter 9 cascade without a verified action fails the program. The program joins the toil bound and does not substitute eradication or a one-change retrospective. Schema checks can prove structural completeness within declared scope. They cannot prove an organization actually learned. A decision earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Cover a catalog-browse incident without copying the spanning record

A storefront engineer wants a one-line action “be more careful about search” and to mark the spanning payment record as covering browse “because it is the same postmortem.”

Extend the Chapter 11 model without adding regional architecture yet:

1. Decide whether catalog-browse needs its own record, a waiver, or is out of scope given Chapters 1–3 and 10.
2. If you add an action, give it owner and due_at; do not use be-more-careful.
3. Name a verification producer that is not the spanning payment record.
4. Do not emit verified: true or learned: true.
5. Identify one observation that would falsify coverage—for example the Chapter 9 cascade repeating with no shed-joined action.
6. Explain which material change would trigger review of the program, not just the template.

Do not copy the spanning record and rename it. Catalog reads have different contributing factors and freeze consequences from checkout plus dispatch. Your durable output is the coverage decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 11 capability when you can explain why a polished postmortem is not a control, cover every Chapter 10 id with a record or expiring waiver, own a cascade action with independent verification, refuse hortatory text, join the toil bound, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Learning can change the portfolio. The architecture still assumes one region. Platform Chapter 14 restored a control plane inside one region and recorded `not-regional-loss` and `not-portfolio-rto`. DevOps Chapter 13 reconstructed one environment. “We have backups” is not multi-region.

Chapter 12 writes a regional-loss architecture with numeric portfolio **RTO (Recovery Time Objective)** and **RPO (Recovery Point Objective)**, and lists those inherited restores as insufficient.

12. Design for the Loss of a Region

Chapter 11 can turn a commanded incident into owned, verified change. The architecture still assumes one region. Platform Chapter 14 restored a control plane inside one region and recorded not-regional-loss and not-portfolio-rto. DevOps Chapter 13 reconstructed one environment. “We have backups” is treated as multi-region. **RTO (Recovery Time Objective)** is “as fast as possible.” Payment and warehouse regionality are hopes.

The production question is now:

What happens when a region is lost, what portfolio RTO and **RPO (Recovery Point Objective)** are defensible, and which data cannot silently move?

Without that model, fail-over and restore collapse. Reconstructing one environment is filed as regional recovery. Restoring plane last known good 1.0 is filed as the same thing as contract last known good tenant-storage-1.0. Storefront and Fulfillment isolation does not survive the story. This chapter writes the architecture. It does not run a game day. It does not execute fail-over. It does not produce **Evidence of portfolio recovery**.

1. An unsafe regional claim

A weak record says:

```
rto: as-fast-as-possible
recovery:
  - devops-reconstruction
  - platform-plane-restore
```

It points at inherited restores and calls them regional recovery. It has no second region, no fail-over order, no numeric objectives, and no isolation constraint. “We have backups” may describe one environment’s durable data. It cannot complete a regional-loss architecture.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 12 baseline:

```
make chapter-12-baseline
```

The command succeeds when it detects the intended unsafe model:

```
chapter 12 baseline: inherited restore claimed as regional recovery
correctly detected
```

The fixture stores RTO as as-fast-as-possible, claims regional recovery from DevOps reconstruction and Platform plane restore, omits tenant isolation, omits payment and warehouse regionality, and collapses plane last known good with the storage contract identity. One-region restore has been treated as a portfolio region plan. Unmeasured speed has been treated as an objective.

That inversion drives the chapter.

2. The production model: regions, numeric objectives, and named insufficiency

Theory — Regional-loss architecture

This model enables Northwind to describe what happens when a region is lost, with numeric portfolio objectives and constraints, rather than by pointing at one-environment reconstruction or a control-plane restore.

Fail-over is not restore

Restore rebuilds something in place: one environment from DevOps Chapter 13 roots, or a control plane to last known good 1.0 inside the same region. Fail-over moves serving to another region under a reviewed order. Those are different production questions. Platform already recorded not-regional-loss and not-portfolio-rto. This chapter **consumes** those limitations. It does not rewrite Platform recovery, and it does not pretend they were never written.

Plane last known good 1.0 and contract last known good tenant-storage-1.0 stay distinct identities. Neither is a region. Listing them as one restore id collapses two inherited recoveries into a slogan.

Best Practice: Name at least two regions, a fail-over order, and the inherited restores that are insufficient.

Production Practice: Teaching topology is active-passive: region-primary serves, region-standby is the fail-over target. Active-active without an order is a hope. This chapter does not execute the shift.

Portfolio RTO and RPO are numeric and owned

RTO is how long the portfolio may take to serve the Chapter 1 journeys from the surviving region. RPO is how much journey state may be lost. Both are numbers with an owner. Teaching values: RTO 14400 seconds (four hours), RPO 900 seconds (fifteen minutes), owner reliability-program. Those are local fixtures, not industry constants.

as-fast-as-possible is not a number. Unmeasured RTO cannot later prove a miss. The objectives file must not emit status: recovered. Architecture is not **Evidence of portfolio recovery**.

Isolation and provider regionality are constraints, not hopes

Storefront and Fulfillment are inherited tenants. Isolation must survive fail-over. A mixed-tenant shift is a later Chapter 14 rejection, but Chapter 12 must already name the constraint or Chapter 14 inherits “put them in the same restore.”

Payment is a critical Storefront dependency. Warehouse is a critical Fulfillment dependency. Chapter 8 already placed them inside journey **SLOs (Service Level Objectives)**. Their regionality is a constraint: they do not silently follow Northwind’s traffic. Email remains non-critical; it is not a third region story.

Data that cannot silently cross regions includes tenant order state and warehouse dispatch state. The lab cannot discover unknown real data gravity. Named constraints are the architecture. Missing gravity stays missing; it is not filled with a backup slogan.

3. Publish architecture, objectives, and constraints

The completed Chapter 12 model uses three files:

```
regions/architecture.yaml
regions/objectives.yaml
regions/constraints.yaml
```

The separation is deliberate. Architecture names regions, mode, and fail-over order. Objectives name numeric RTO and RPO with owner. Constraints name isolation, provider regionality, data that cannot silently move, and insufficient inherited restores. None of the files may claim the portfolio recovered.

Practice — Name two regions and an order; do not call restore fail-over

Keep the topology active-passive until a later chapter executes a shift.

Open `regions/architecture.yaml`. Regions are `region-primary` and `region-standby`. Mode is `active-passive`. Fail-over order is from `region-primary` to `region-standby` when `region-primary` is lost. Owner is `reliability-program`. If only one region appears, later fail-over has nowhere to go.

Practice — Publish numeric owned RTO and RPO

Refuse `as-fast-as-possible`. Do not emit `recovered`.

Open `regions/objectives.yaml`. `rto_seconds` is 14400. `rpo_seconds` is 900. Owner is `reliability-program`. Review trigger is `material change to journeys, regions, or provider regionality`. If RTO is a slogan, the checkpoint fails.

Inspect the objectives with three questions:

1. Could a later fail-over miss this number, or is the target unfalsifiable?
2. Is the owner `reliability-program`, or whoever restored the plane last time?
3. Would this still be true if DevOps reconstruction succeeded in the lost region?

Practice — List inherited restores as insufficient; keep isolation and provider regionality

Consume `not-regional-loss` and `not-portfolio-rto`. Do not collapse plane `1.0` with `tenant-storage-1.0`.

Open `regions/constraints.yaml`. Isolation requires `Storefront` and `Fulfillment` to remain distinct through fail-over. Provider regionality marks `payment` and `warehouse` as `region-scoped`. `no_silent_cross_region` names `tenant order state` and `warehouse dispatch state`.

`insufficient_restores` lists DevOps one-environment reconstruction and Platform plane restore. `insufficient_identities` lists plane last known good 1.0 and contract last known good tenant-storage-1.0 as separate rows. `limitations_consumed` lists not-regional-loss and not-portfolio-rto.

If those inherited limitation ids are missing, later chapters will treat Platform recovery as the regional plan.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-12-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 12 checkpoint: numeric rto rpo, isolation, and insufficient
inherited restores verified
```

The audit validates the three Chapter 12 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- two regions and an active-passive fail-over order exist;
- RTO and RPO are positive numbers owned by `reliability-program`;
- RTO is not as-fast-as-possible;
- Storefront and Fulfillment isolation survives fail-over;
- payment and warehouse regionality are constraints;
- DevOps reconstruction and Platform plane restore are insufficient;
- plane last known good 1.0 and tenant-storage-1.0 stay distinct insufficient identities;
- Platform limitations not-regional-loss and not-portfolio-rto are consumed; and
- `status: recovered` is not emitted.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint cannot discover unknown real data gravity. It does not fail over a live region. It does not prove that four hours or fifteen minutes is the right commercial pair. Those are judgment claims. The review trigger exists so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 12 evidence
Mechanism evidence	Schemas and the regional-architecture evaluator operated successfully.
Decision evidence	Regions, fail-over order, numeric objectives, isolation, and insufficient inherited restores are explicit.
Outcome evidence	Not yet produced as a fail-over; objectives are falsifiable numbers, not slogans.
Recovery evidence	Not yet produced; later chapters must prove Evidence of portfolio recovery in the model.

Chapter 12 creates decision evidence about regional loss. Pretending that the local checkpoint proved a multi-region estate would weaken game days and executed fail-over.

4. Test the model under failure

Independent control failure — Inherited restore claimed as regional recovery

Practice — Separate fail-over from restore; assign numeric objectives; list inherited recoveries as insufficient

Drop as-fast-as-possible. Keep plane 1.0 distinct from tenant-storage-1.0.

The baseline fixture contains this model:

```
rto: as-fast-as-possible
recovery:
  - devops-reconstruction
  - platform-plane-restore
claims: regional-recovery
```

The problem is not merely a missing second region. The model treats one-environment reconstruction and in-region plane restore as the regional architecture. Unmeasured RTO cannot later miss. Isolation and provider regionality are absent, so a mixed-tenant hope is the plan.

Severity: high; a real region loss will invent the plan during the event, the book's cumulative informal heroics.

Plausible harm: Storefront is restored in the lost region; Fulfillment isolation collapses; payment does not exist in the “standby”; RTO is whatever someone typed in chat.

Potential blast radius: every backup labeled multi-region; every plane restore labeled portfolio RTO.

Bounded by: later game days and executed fail-over. None repairs an architecture that is two inherited runbooks.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** Both accept-and-complete-order and dispatch-fulfillment are at risk when a region is lost. Reconstructing one Storefront environment does not recover dispatch.
- **Error budget:** Not a new freeze rule. Chapter-local implication: unmeasured RTO cannot later authorize continue or freeze during fail-over.
- **Human system:** Applicable as later game-day and fail-over load on Chapter 6 rotations. An unmeasured objective makes the commander guess.
- **Portfolio recovery:** Not yet produced. Pointing at plane last known good cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling inherited restore “regional recovery” encourages three controls: skip a second region because backups exist, skip numbers because speed is obvious, and skip isolation because tenants already share a platform. Those moves may be sincere. They do not answer where traffic goes, how much state may be lost, or which provider will not follow.

Collapsing plane 1.0 with tenant-storage-1.0 makes two Platform identities one restore. Omitting not-regional-loss makes Platform’s own limitation ornamental. as-fast-as-possible makes outcome evidence a mood.

Correction

The completed model does not claim inherited restore is regional fail-over. It names two regions and an active-passive order, publishes numeric owned RTO and RPO, keeps tenant isolation, marks payment and warehouse as region-scoped, lists DevOps reconstruction and plane restore as insufficient, keeps the two last-known-good identities distinct, and consumes Platform’s regional limitations.

That correction changes later decisions:

- Chapter 13 must tabletop or simulate this regional loss as one scenario among others, not as a mixed-backup fixture that stands in for fail-over.
- Chapter 14 must execute fail-over against these numbers and constraints, and must not treat reconstruction or plane restore as **Evidence of portfolio recovery**.

The model is practical because it changes the production contract across the rest of the book. Adding an arbitrary cloud-region diagram would not make it more practical.

5. Production reality

Common regional-architecture errors

Treating backups as a second region

Durable data in one region is restore. Fail-over needs another region.

Unmeasured RTO

“As fast as possible” cannot miss. A second count can.

Collapsing plane last known good with the storage contract

They are distinct inherited identities. Neither is a region.

Ignoring tenant isolation

Storefront and Fulfillment must remain distinct through the story, or Chapter 14 inherits mixed-tenant fail-over.

Hoping payment and warehouse follow traffic

Chapter 8 already made them critical. Regionality is a constraint.

Emitting recovered on the architecture file

Architecture is not **Evidence of portfolio recovery**.

Copying Storefront’s region row onto Fulfillment as if dispatch were checkout

Different journey, dependency, and data gravity.

6. What changed

Before	After
RTO was as-fast-as-possible.	RTO is 14400 seconds, RPO is 900 seconds, owned by reliability-program.
Recovery pointed at DevOps reconstruction and plane restore.	Those inherited restores are listed as insufficient.
Plane 1.0 and tenant-storage-1.0 were one identity.	They remain distinct insufficient identities.
Isolation was assumed.	Storefront and Fulfillment isolation survives fail-over.
Payment and warehouse would “just work.”	Both are region-scoped constraints.
A valid schema could appear to prove multi-region.	Structural, decision, outcome, and recovery evidence remain distinct.

What changed was not merely three YAML files. Northwind now has a reviewable regional-loss architecture that later game days and fail-over can consume without inventing the plan during the outage.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Regional-loss architecture	regions/architecture.yaml	It retains regions, active-passive mode, and fail-over order later execution must not replace with a single restore.
Portfolio RTO/RPO decision record	regions/objectives.yaml and regions/constraints.yaml	They retain numeric objectives, isolation, provider regionality, and insufficient inherited restores.

These artifacts should change when Northwind's regions, journeys, or provider regionality materially change—not whenever a backup product is renamed.

What You Learned

Regional loss needs two regions, a fail-over order, and numeric owned RTO and RPO. Fail-over is not restore. DevOps one-environment reconstruction and Platform plane restore are insufficient. Plane last known good and contract last known good stay distinct. Tenant isolation and payment/warehouse regionality are constraints. Schema checks can prove structural completeness within declared scope. They cannot discover unknown real data gravity or fail over a live estate. A model earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Place catalog-browse data without copying the payment region row

A storefront engineer wants catalog search “in both regions because backups already exist, RTO as fast as possible, same as payment.”

Extend the Chapter 12 model without adding a game-day program yet:

1. Decide whether catalog-browse state may silently cross regions, given Chapters 1–3 and 8.
2. Give it a constraint that is not a renamed payment region-scoped row, or record it out of scope.
3. Do not use as-fast-as-possible as its objective.
4. Do not list DevOps reconstruction as sufficient for that data.
5. Identify one observation that would falsify the architecture—for example plane restore still claimed as regional recovery.
6. Explain which material change would trigger review of the objectives, not just the backup job name.

Do not copy the warehouse constraint and rename it. Catalog reads have different gravity and freeze consequences from dispatch. Your durable output is the constraint and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 12 capability when you can explain why inherited restore is not fail-over, name two regions and numeric RTO/RPO, keep isolation and provider regionality, list plane 1.0 and tenant-storage-1.0 as distinct insufficient identities, consume not-regional-loss and not-portfolio-rto, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

The architecture exists on paper. It has not been exercised as a program. The only practiced recovery nearby is Platform's mixed-backup isolation test. There is no cadence, blast-radius contract, or learning join.

Chapter 13 publishes a game-day program so freeze, on-call page path, dependency loss, and regional-loss tabletop or simulated fail-over are exercised on a cadence—not as a rehearsal of Chapter 14 fail-over, and not as a single mixed-backup fixture.

13. Run Game Days as a Recurring Program

Chapter 12 named two regions, numeric portfolio objectives, and inherited restores as insufficient. That architecture still sits on paper. The only practiced recovery nearby is Platform's mixed-backup isolation test. There is no cadence, no blast-radius contract, and no join to Chapter 11 learning. A single fixture becomes "we do game days."

The production question is now:

How does Northwind exercise reliability controls on a cadence, including failures that are not one mixed-backup fixture, without taking the portfolio down for real?

Without that program, unbounded chaos becomes an outage. A game day that cannot fail is theater. This chapter must exercise error-budget freeze, on-call page path, dependency loss, and regional-loss tabletop or simulated fail-over. It is not a rehearsal of Chapter 14 executed fail-over. It does not chaos-test a live fleet. It does not produce **Evidence of portfolio recovery**.

1. An unsafe annual mixed-backup

A weak record says:

```
scenario: mixed-backup
cadence: annual
status: complete
```

It files Platform Chapter 14's mixed-backup fixture as the year's game day and marks the program complete. It never exercises a Chapter 4 freeze, a Chapter 6 page path, a Chapter 8 dependency loss, or a Chapter 12 tabletop. "We restored a backup once" may describe an isolation test. It cannot complete a reliability game-day program.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 13 baseline:

```
make chapter-13-baseline
```

The command succeeds when it detects the intended unsafe design:

```
chapter 13 baseline: single mixed-backup game day marked complete
correctly detected
```

The fixture stores one mixed-backup result, stamps the program complete, omits cadence as a recurrence contract, records no abort, and never joins learning. A Platform isolation fixture has been treated as the SRE program. Informal heroics have been scheduled once a year.

That inversion drives the chapter.

2. The production model: cadence, coverage, abort, learning join

Theory — Recurring game-day program

This model enables Northwind to exercise freeze, page path, dependency loss, and regional-loss tabletop on a cadence, rather than by marking a single mixed-backup fixture complete.

A game day is not chaos and not Chapter 14 fail-over

A game day is a scoped, abortable exercise of a named control. Chaos without a blast-radius contract is an outage with a calendar invite. A **DR (disaster recovery)** restore of one backup is not a portfolio game-day program. Chapter 14 will execute regional fail-over against Chapter 12 objectives. This chapter **tabletops or simulates** regional loss. If a scenario's kind is chapter-14-failover, the program has skipped ahead and left freeze, paging, and dependency unexercised.

Best Practice: Require four kinds before anyone talks about complete: error-budget-freeze, on-call-page-path, dependency-loss, and regional-loss-tabletop.

Production Practice: mixed-backup may exist as a fixture. It is insufficient-alone. It cannot complete the program.

Cadence and abort keep the blast radius declared

Cadence is a teaching value: 90d. It is not an industry constant. Annual-once is not recurrence; it is a holiday. Abort when blast radius would exceed the contract: blast-radius-exceeds-contract. An unbounded-chaos attempt that would take the live portfolio down must record disposition: abort. A program with no abort path cannot fail safely.

The lab does not chaos-test a live fleet. Results are local fixtures, like Chapter 3's event counts.

Results feed Chapter 11; they do not emit recovered

Game-day results join the learning register. The dependency-loss result feeds verify-payment-retry-shed. That is already an owned, independently verified action. The program does not invent a hortatory "be more careful next game day." It does not stamp status: recovered. A successful tabletop is mechanism and decision evidence. It is not **Evidence of portfolio recovery**.

Joins are explicit:

Kind	Joins
error-budget-freeze	Chapter 4 freeze-storefront-releases
on-call-page-path	Chapter 6 storefront-oncall-system
dependency-loss	Chapter 8 payment, feeds Chapter 11 cascade action
regional-loss-tabletop	Chapter 12 region-primary → region-standby, mode tabletop

Paging Slack, paging a catalog contact, or treating email as the dependency drill reopens earlier chapters under a game-day name.

3. Publish program, scenarios, and results

The completed Chapter 13 model uses three files:

```
gamedays/program.yaml
gamedays/scenarios.yaml
gamedays/results.yaml
```

The separation is deliberate. The program names cadence, required kinds, abort rule, learning join, and that mixed-backup cannot complete it. Scenarios bind kinds to earlier artifact ids. Results record in-bounds runs, an abort, and mixed-backup as insufficient-alone. None of the files may emit recovered.

Practice — Forbid completing the program on mixed-backup alone

Keep mixed-backup insufficient-alone. Do not skip the four required kinds.

Open `gamedays/program.yaml`. Owner is `reliability-program`. Cadence is `90d`. `required_kinds` lists the four kinds. `abort_when` is `blast-radius-exceeds-contract`. `learning_join` is `northwind-learning-actions`. `forbidden_complete_on` is `mixed-backup`. `not_chapter_14_failover` is `true`. If `complete: true` appears because mixed-backup ran, the checkpoint fails.

Practice — Bind each required kind to an earlier control

Freeze joins a Chapter 4 action id. Page path joins a Chapter 6 system. Dependency loss is `payment`, not `email`. Regional loss is `tabletop`.

Open `gamedays/scenarios.yaml`. Teaching rows include `freeze`, `page path`, `payment dependency loss`, `regional-loss tabletop`, `mixed-backup` marked `insufficient_alone`, and an `unbounded-chaos` row whose only legal result is `abort`. The regional row mode is `tabletop`. If that mode is `chapter-14-failover`, the program rehearsed the next chapter.

Inspect each scenario with three questions:

1. Which earlier chapter's artifact does this exercise—or is it only a backup job?
2. What blast radius is declared, and what abort fires if it would be exceeded?
3. Would this still be a game day if mixed-backup were deleted?

Practice — Record four in-bounds results, one abort, and mixed-backup as insufficient-alone

Feed the dependency result into the Chapter 11 cascade action. Do not stamp `recovered`.

Open `gamedays/results.yaml`. Four in-bounds results cover the required kinds. One abort records `blast-radius-exceeds-contract`. The mixed-backup result is `insufficient-alone`. The dependency result `feeds_action` is `verify-payment-retry-shed`. Quoted `as_of` values are RFC 3339. If only mixed-backup is complete, the program has not run.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-13-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 13 checkpoint: freeze, page path, dependency, and regional
tabletop coverage verified
```

The audit validates the three Chapter 13 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- cadence is a recurrence, not annual as the only event;
- four required kinds have in-bounds results;
- mixed-backup cannot complete the program;
- an abort is recorded for blast radius that would exceed the contract;
- freeze joins a Chapter 4 freeze action;
- page path joins a Chapter 6 on-call system, not Slack or a catalog contact;
- dependency loss is payment (or warehouse), not email as a critical drill;
- regional loss is tabletop, not Chapter 14 fail-over;
- results feed a Chapter 11 action; and
- status: recovered is not emitted.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not chaos-test a live fleet. It does not prove that 90 days is the right commercial cadence. Those are judgment claims. The review trigger exists so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 13 evidence
Mechanism evidence	Schemas and the game-day evaluator operated successfully.
Decision evidence	Cadence, required kinds, abort, learning join, and mixed-backup insufficiency are explicit.
Outcome evidence	Completeness is computed from four in-bounds kinds plus abort, not emitted from mixed-backup.
Recovery evidence	Not yet produced; Chapter 14 must prove Evidence of portfolio recovery in the model.

Chapter 13 creates decision evidence about how controls are exercised. Pretending that the local checkpoint proved a production game day would weaken executed fail-over.

4. Test the design under failure

Independent control failure — Mixed-backup filed as the annual game day and marked complete

Practice — Fail completeness; add freeze, page path, dependency, and regional tabletop; require recurrence

Drop complete on mixed-backup alone. Record an abort path.

The baseline fixture contains this model:

```
scenario: mixed-backup
cadence: annual
status: complete
```

The problem is not merely a missing calendar. Platform’s isolation fixture has been treated as the SRE program. Freeze, paging, payment loss, and regional tabletop never ran. There is no abort. Learning is not joined. Chapter 12’s architecture remains unexercised. Chapter 14 is either skipped or silently rehearsed as “we already restored a backup.”

Severity: high; the first real region loss still invents the plan, and exhausted budget still has never been frozen in an exercise.

Plausible harm: mixed-backup succeeds; freeze is unpracticed; Slack is the page path; payment loss is a surprise; regional tabletop never happened.

Potential blast radius: every annual “game day” that is one fixture; every unbounded chaos drill without abort.

Bounded by: later executed fail-over. None repairs a program that cannot fail.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** Freeze and payment-loss drills protect accept-and-complete-order. Regional tabletop protects both journeys. Mixed-backup alone protects neither as a portfolio exercise.
- **Error budget:** Applicable as the freeze scenario joining Chapter 4. Skipping it leaves freeze unpracticed.
- **Human system:** Applicable as the page-path scenario joining Chapter 6. Mixed-backup does not page a living primary.
- **Portfolio recovery:** Not yet produced. A mixed-backup complete flag cannot become **Evidence of portfolio recovery**.

Diagnosis

Calling mixed-backup complete encourages three controls: skip the other kinds because a restore ran, skip abort because the fixture is safe, and skip learning because the slide is green. Those moves may be sincere. They do not answer whether freeze still joins, whether the page hit a Chapter 6 system, or whether regional loss was tabletopped against Chapter 12.

A chapter-14-failover scenario would make the same mistake in the other direction: rehearsing execution while freeze and paging stay theoretical.

Correction

The completed model does not mark mixed-backup complete. It requires four kinds, records in-bounds results for each, aborts unbounded blast radius, joins freeze, on-call, payment, and regional tabletop, feeds Chapter 11, and forbids treating this program as Chapter 14 fail-over.

That correction changes later decisions:

- Chapter 14 must execute fail-over against Chapter 12, not copy these tabletop results as **Evidence of portfolio recovery**.
- Mixed-backup remains an isolation fixture. It still cannot close the reliability program.

The design is practical because it changes the production contract across the rest of the book. Adding an arbitrary chaos product would not make it more practical.

5. Production reality

Common game-day errors

Completing on one mixed-backup result

Insufficient-alone means the program is not done.

Rehearsing Chapter 14 as the only scenario

Tabletop regional loss. Do not skip freeze, paging, and dependency.

Annual-once cadence

A holiday is not a program.

No abort path

If blast radius cannot fail the exercise, the exercise cannot be safe.

Paging Slack or a catalog contact

Chapter 6 already refused that as the system.

Email as the dependency drill

Chapter 8 already kept notification non-critical.

Hortatory follow-up instead of a Chapter 11 action

Results feed owned verification, not “be more careful.”

Storing recovered on the result

A tabletop is not portfolio recovery.

6. What changed

Before	After
Mixed-backup was marked complete.	Mixed-backup is insufficient-alone; four kinds have in-bounds results.
Cadence was annual-once.	Cadence is 90d.
There was no abort.	Unbounded blast radius records abort.

Before	After
Freeze, page path, and payment were unexercised.	Each kind joins a Chapter 4, 6, or 8 artifact.
Regional loss was either skipped or Chapter 14.	Regional loss is tabletop against Chapter 12.
Learning was not joined.	The dependency result feeds verify-payment-retry-shed.

What changed was not merely three YAML files. Northwind now has a reviewable game-day program that later fail-over can follow without pretending a backup fixture was the reliability program.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Game-day program contract	gamedays/program.yaml	It retains cadence, required kinds, abort, learning join, and mixed-backup insufficiency later execution must not replace with one fixture.
Scenario-and-result register	gamedays/scenarios.yaml and gamedays/results.yaml	They retain the four exercised kinds, the abort, and tabletop regional loss.

These artifacts should change when Northwind's freeze actions, on-call systems, dependencies, or regions materially change—not whenever a backup job is renamed.

What You Learned

A game-day program recurs, declares blast radius, and can abort. Completeness requires freeze, page path, dependency loss, and regional-loss tabletop—not a single mixed-backup result, and not a rehearsal of Chapter 14. Results feed Chapter 11. Schema checks can prove structural completeness within declared scope. They cannot chaos-test a live fleet. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Exercise catalog-browse without copying mixed-backup or the payment drill

A storefront engineer wants to mark the program complete “because mixed-backup already ran, and browse can reuse the payment dependency scenario.”

Extend the Chapter 13 model without adding executed fail-over yet:

1. Decide whether catalog-browse needs its own scenario kind or is out of scope given Chapters 1–3 and 8.
2. If you add a result, keep mixed-backup insufficient-alone.
3. Do not page Slack, and do not treat the result as Chapter 14 fail-over.
4. Join or refuse a Chapter 11 action without hortatory text.
5. Identify one observation that would falsify completeness—for example only mixed-backup marked complete.
6. Explain which material change would trigger review of the program, not just the backup fixture name.

Do not copy the payment dependency row and rename it. Catalog reads have different criticality and freeze consequences from checkout. Your durable output is the coverage decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 13 capability when you can explain why mixed-backup cannot complete the program, name the four required kinds, record an abort, join freeze, on-call, payment, and regional tabletop, refuse Chapter 14 rehearsal, feed Chapter 11, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

The program can exercise controls. An actual regional fail-over has not been proven. A region is lost. The temptation is to reconstruct one environment, restore the plane from newest, or invent the order during the outage.

Chapter 14 executes fail-over against Chapter 12, keeps tenant isolation, and produces **Evidence of portfolio recovery** that cannot hide a missed **RTO (Recovery Time Objective)**, a missed **RPO (Recovery Point Objective)**, or collapsed isolation.

14. Fail Over a Region Without Taking the Portfolio Down

Chapter 13 can exercise freeze, page path, dependency loss, and regional-loss tabletop on a cadence. A region is still lost. The temptation is to reconstruct one environment, restore the plane from newest, or invent the order during the outage. Mixed-tenant fail-over recreates Platform's shared blast radius. A green reconstruction job is treated as the portfolio.

The production question is now:

How does Northwind fail over a lost region, keep tenant isolation, meet portfolio **RTO (Recovery Time Objective)** and **RPO (Recovery Point Objective)**, and prove the portfolio recovered?

Without that execution, newest mixed-region state is applied, Fulfillment intent lands in Storefront, and recovery is declared because one environment's reconstruction succeeded. This chapter follows the Chapter 12 order, keeps Storefront and Fulfillment isolated, and produces **Evidence of portfolio recovery** in the model. It cannot prove a live multi-region estate. It is not DevSecOps restored trust, not Platform restored isolation, and not a Chapter 13 tabletop copied as recovery.

1. An unsafe mixed-region recovered stamp

A weak record says:

```
replay: newest-mixed-region
fulfillment_intent: storefront
status: recovered
recovery: devops-reconstruction
```

It applies newest state across tenants and regions, puts Fulfillment intent into Storefront, and stamps recovered because DevOps reconstruction ran. Plane last known good 1.0 and tenant-storage-1.0 collapse again. Isolation does not survive. **SLOs (Service Level Objectives)** for accept-and-complete-order and dispatch-fulfillment are not checked. Informal heroics have been given a restore job.

Work from the lab working tree using the How to Use This Book procedure. From the SRE lab root, run the Chapter 14 baseline:

```
make chapter-14-baseline
```

The command succeeds when it detects the intended unsafe design:

```
chapter 14 baseline: mixed-region replay declared recovered correctly
detected
```

The fixture replays mixed-region newest state, lands Fulfillment in Storefront, claims reconstruction as recovery, misses numeric RTO and RPO, collapses isolation, and emits `status: recovered` to hide those misses. A Platform mixed-backup failure and a DevOps one-environment reconstruction have been treated as the SRE fail-over. Availability theater has been given a recovered flag.

That inversion drives the chapter.

2. The production model: order, isolation, explicit continue or freeze, computed recovery

Theory — Executed regional fail-over

This model enables Northwind to fail over a lost region from reviewed evidence, keep tenant isolation, and prove portfolio recovery against numeric objectives, rather than by applying newest mixed-region state or reconstructing one environment.

Fail-over follows Chapter 12; it does not restore in place

The plan consumes northwind-regional-architecture. Lost region-primary is isolated. Serving moves to region-standby when region-primary-lost. Mode stays active-passive. Restore-in-place to the lost region is DevOps reconstruction under a fail-over name. Chapter 13's gameday-regional-loss-tabletop and Platform's mixed-backup are insufficient evidence. They may have been practiced. They are not this execution.

Last known good is per region and per tenant. Plane 1.0 and contract tenant-storage-1.0 remain distinct insufficient identities. Newest is not last known good. A snapshot that holds every inherited root can still replay one tenant into another.

Best Practice: Isolate the lost region, follow the reviewed order, and reject mixed-tenant and mixed-region replay before anyone talks about recovered.

Production Practice: Teaching observations: elapsed 3600 seconds against RTO 14400, data loss 300 seconds against RPO 900. Those are local fixtures, not industry constants.

Continue or freeze is explicit; recovered is computed

Platform already taught continue versus freeze for tenants during plane restore. This chapter applies that idea to a region. Storefront **serves** from standby (continue) and **change** stays freeze on freeze-storefront-releases. Fulfillment **serves** (continue) and **change** stays slow on slow-fulfillment-releases. Missing decisions are mixed-backup with a regional story. The commander is storefront-primary-a, a living Chapter 6 primary, not Slack.

Verification records journeys, elapsed seconds, RPO loss, isolation, mixed-replay rejection, insufficient inherited restores, and consumed Platform limitations not-regional-loss and not-portfolio-rto. It must not emit status: recovered. Completeness is computed. If RTO is missed, RPO is missed, or isolation collapsed, a recovered stamp is a hide, not a proof.

Job-time proofs such as time-to-first-environment stay a **job-time budget**. They cannot become portfolio recovery. Notification remaining non-critical cannot stand in for dispatch.

The lab cannot prove a real multi-region fail-over. Results are local fixtures, like Chapter 3's event counts. Modeled **Evidence of portfolio recovery** is the strongest claim this chapter may make.

3. Publish plan, trace, isolation, and verification

The completed Chapter 14 model uses four files:

```
failover/plan.yaml
failover/trace.yaml
failover/isolation.yaml
failover/verification.yaml
```

The separation is deliberate. The plan binds Chapter 12 order, commander, last known good, and inherited insufficiency. The trace records the shift and elapsed/RPO observations. Isolation names tenant continue/freeze and rejected replays. Verification names the journeys and isolation claims the evaluator uses. Independent fail-over observations live in `fixtures/observations/chapter-14.yaml`; they are not emitted by verification. None of the files may emit `recovered` or `slo_met`.

Practice — Follow the Chapter 12 order and refuse inherited restores as recovery

Isolate `region-primary`. Serve from `region-standby`. Keep `plane 1.0` distinct from `tenant-storage-1.0`.

Open `failover/plan.yaml`. Owner is `reliability-program`. follows is `northwind-regional-architecture`. Lost is `region-primary`, target `region-standby`, when `region-primary-lost`. Commander is `storefront-primary-a`. `insufficient_restores` lists `devops-one-environment-reconstruction` and `platform-plane-restore`. `insufficient_identities` lists quoted `"1.0"` and `tenant-storage-1.0`. `insufficient_evidence` lists `gameday-regional-loss-tabletop` and `mixed-backup`. If target is the lost region, or recovery points at reconstruction, the checkpoint fails.

Practice — Record elapsed and RPO loss as observations, not as a recovered flag

Keep the trace and verification numbers equal. Do not stamp `recovered`.

Open `failover/trace.yaml`. from / to / when match the plan. `lost_isolated` is true. Quoted `started_at` and `completed_at` are RFC 3339. `elapsed_seconds` is 3600. `rpo_lost_seconds` is 300. If the lost region is not isolated, fail-over has not started.

Practice — Keep Storefront and Fulfillment distinct with explicit continue or freeze

Reject mixed-tenant and mixed-region replay. Do not land Fulfillment intent in Storefront.

Open `failover/isolation.yaml`. Tenants are `storefront` and `fulfillment`. source is `explicit-tenant-decision`. `survives_failover` is true. `rejected_replays` includes `mixed-tenant` and `mixed-region`. Each tenant has serving, change, a Chapter 4 `policy_join`, and a standby last known good that is not `"1.0"` and not `tenant-storage-1.0`. If `fulfillment_intent` is `storefront`, Platform's `mixed-backup` has returned as regional recovery.

Inspect each tenant with three questions:

1. Which region now serves this tenant—or is newest mixed state being applied?
2. Is change continue, slow, or freeze by a Chapter 4 action id?
3. Would isolation still hold if Fulfillment's last known good were replayed into Storefront?

Practice — Compute journey SLO outcomes from independent observations

Do not emit `slo_met`. Listing a journey id is not proof the SLO held.

Open `failover/verification.yaml`. Journeys are `accept-and-complete-order` and `dispatch-fulfillment`. Elapsed and RPO loss match the trace. `isolation_holds` is true. `mixed_replay` is rejected. `inherited_restores_insufficient` is true. `limitations_consumed` lists `not-regional-loss` and `not-portfolio-rto`. There is no `status` field and no `slo_met` field. Open `fixtures/observations/chapter-14.yaml`. Teaching counts: order success 997/1000 against target 0.995, order latency 995/1000 against 0.99, dispatch 992/1000 against 0.99. The evaluator compares those ratios to the Chapter 3 catalog. If verification lists the journeys while the observations miss the targets, or if it stamps `slo_met: true`, the hide is the finding.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-14-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 14 checkpoint: isolation, rto rpo, and evidence of portfolio
recovery verified
```

The audit validates the four Chapter 14 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- the plan follows Chapter 12 active-passive order;
- the lost region is isolated;
- mixed-tenant and mixed-region replay are rejected;
- Storefront and Fulfillment continue or freeze by explicit decision joined to Chapter 4;
- last known good is not newest, not plane 1.0, and not tenant-storage-1.0;
- inherited restores and game-day tabletop are insufficient;
- elapsed meets RTO and data loss meets RPO;
- both protected journeys are listed **and** their catalog SLOs are met from independent fail-over observations;
- `slo_met` is not emitted;
- job-time is not claimed as recovery;
- Platform limitations are consumed; and
- `status: recovered` is not emitted.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint cannot prove a real multi-region fail-over. It does not prove that 3600 seconds is the right commercial elapsed time. Those are judgment claims. The review trigger exists so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 14 evidence
Mechanism evidence	Schemas and the fail-over evaluator operated successfully.
Decision evidence	Order, isolation, continue/freeze, and inherited insufficiency are explicit.
Outcome evidence	Elapsed $\leq$ RTO, data loss $\leq$ RPO, and both journeys' catalog SLOs are met from independent fail-over observations—not from a listed id.
Recovery evidence	Produced in the model: mixed replay rejected, isolation holds, inherited restores insufficient, recovered not self-emitted. That is Evidence of portfolio recovery . It is not DevSecOps restored trust, not Platform restored isolation, and not a live-estate fail-over.

Chapter 14 creates modeled recovery evidence. Pretending that the local checkpoint proved a production multi-region estate would reopen Platform's not-regional-loss as if it had never been written.

4. Test the design under failure

Cumulative reliability failure — Newest mixed-region state applied, Fulfillment intent in Storefront, recovered because reconstruction succeeded

Practice — Reject mixed replay; fail over from reviewed regional evidence; verify isolation and journey SLOs; record inherited restores as not portfolio recovery

Drop status: recovered. Keep tenants distinct.

The baseline fixture contains this model:

```
replay: newest-mixed-region
fulfillment_intent: storefront
status: recovered
recovery: devops-reconstruction
```

The problem is not merely a slow fail-over. Newest mixed-region state recreates Platform's shared blast radius across a second region. Fulfillment intent in Storefront collapses tenancy the platform product already forbade. Reconstruction of one environment is filed as portfolio recovery. Unmeasured miss is hidden by a recovered stamp. Informal heroics have a job name.

Severity: high; the book's cumulative availability theater and informal heroics now have a recovered flag.

Plausible harm: Storefront serves Fulfillment orders; dispatch state is gone; payment and warehouse regionality are ignored; RTO and RPO cannot later miss.

Potential blast radius: every newest snapshot labeled fail-over; every reconstruction labeled portfolio recovery; every tabletop copied as recovered.

Bounded by: none later in this book. The conclusion cannot repair a recovered stamp that hid mixed replay.

Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Reliability questions

- **Journey:** Both accept-and-complete-order and dispatch-fulfillment are at risk. Reconstructing one Storefront environment does not recover dispatch, and mixed replay can destroy both.
- **Error budget:** Applicable as continue/freeze joined to Chapter 4. Mixed newest apply ignores freeze. A recovered stamp cannot un-exhaust budget.
- **Human system:** Applicable as the commander storefront-primary-a. Slack as commander returns Chapter 6 heroics during the region loss.
- **Portfolio recovery:** This is the chapter that must produce it. Mixed replay plus reconstruction plus status: recovered is the opposite of **Evidence of portfolio recovery**.

Diagnosis

Calling mixed-region newest recovered encourages three controls: skip isolation because a restore ran, skip Chapter 12 numbers because traffic returned, and skip journey SLOs because one reconstruction job succeeded. Those moves may be sincere. They do not answer whether Fulfillment stayed Fulfillment, whether elapsed met 14400 seconds, or whether plane 1.0 was treated as a region.

Copying Chapter 13 tabletop results into status: recovered, or listing journey ids while fail-over observations miss the SLO targets, makes the same mistake with a practiced fixture: mechanism evidence treated as recovery evidence.

Correction

The completed model isolates the lost region, follows active-passive order, rejects mixed-tenant and mixed-region replay, names continue/freeze per tenant, lists inherited restores and game-day tabletop as insufficient, records elapsed and RPO loss against Chapter 12, verifies both journeys from independent observations against catalog targets, consumes Platform limitations, and refuses to emit recovered or slo_met.

Recovery

When those observations hold, the evaluator computes **Evidence of portfolio recovery** in the model. That is the durable outcome. It is not a live fail-over certificate. It is not restored trust. It is not restored isolation of a control plane. It is the portfolio claim this book was remaining owner to make.

That correction changes the conclusion:

- The reliability program can name journeys, freeze change, page a system, learn, exercise game days, and recover a region without inventing the plan during the outage.
- Reconstruction, plane restore, mixed-backup, and tabletop remain what they were. They still cannot close this chapter.

The design is practical because it changes the production contract the rest of the series handed off. Adding an arbitrary multi-region product would not make it more practical.

5. Production reality

Common fail-over errors

Applying newest mixed-region state

Newest is not last known good. Mixed-region is mixed-tenant with geography.

Landing Fulfillment intent in Storefront

Isolation that does not survive fail-over is Platform Chapter 14's denied case, repeated.

Stamping recovered on reconstruction or plane restore

Inherited restores stay insufficient. Completeness is computed.

Copying Chapter 13 tabletop as recovered

A game day is not fail-over execution.

Missing continue or freeze

An implicit "bring them all back" is mixed-backup.

Paging Slack as commander

Chapter 6 already refused that as the system.

Promoting job-time to portfolio recovery

time-to-first-environment remains a job-time budget.

Hiding a missed RTO or RPO behind recovered

If the number can miss, a flag cannot un-miss it.

Hiding a missed journey SLO behind a listed id or s1o_met

A journey name in verification is not outcome evidence. The ratio is computed.

6. What changed

Before	After
Newest mixed-region state was applied.	Mixed-tenant and mixed-region replay are rejected.
Fulfillment intent landed in Storefront.	Each tenant has explicit continue/freeze and its own last known good.
Reconstruction was stamped recovered.	Inherited restores are insufficient; recovered is computed, not emitted.
Fail-over was restore-in-place.	Lost region-primary is isolated; region-standby serves.
RTO and RPO could not miss.	Elapsed $3600 \leq 14400$; data loss $300 \leq 900$.
Journey ids stood in for SLO outcomes.	Fail-over observations are compared to catalog targets; slo_met is not emitted.
Game-day tabletop stood in for execution.	Tabletop is listed as insufficient evidence.

What changed was not merely four YAML files. Northwind now has modeled **Evidence of portfolio recovery** that later readers can audit without pretending a reconstruction job was the reliability program.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Regional fail-over plan	failover/plan.yaml	It retains Chapter 12 order, commander, last known good, and inherited insufficiency later slogans must not replace with newest.
Evidence of portfolio recovery	failover/trace.yaml, failover/isolation.yaml, and failover/verification.yaml	They retain isolation, continue/freeze, numeric observations, and computed recovery without a self-emitted recovered stamp.

These artifacts should change when Northwind's regions, journeys, isolation, or Chapter 12 objectives materially change—not whenever a restore job is renamed.

What You Learned

Regional fail-over isolates the lost region, follows a reviewed order, keeps tenants distinct, and meets numeric RTO and RPO. Mixed replay is rejected. Inherited restores and game-day tabletop are insufficient. Journey SLO outcomes are computed from independent observations, not from a listed id. Verification must not emit recovered or `slo_met` to hide a miss. Schema checks can prove structural completeness within declared scope. They cannot prove a live multi-region estate. Modeled **Evidence of portfolio recovery** is the claim. A design earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

Independent Practice — Place catalog-browse fail-over without copying mixed-tenant or the order-path row

A storefront engineer wants catalog search “failed over with Storefront, newest snapshot, recovered because reconstruction succeeded, same continue as checkout.”

Extend the Chapter 14 model without adding a live region:

1. Decide whether catalog-browse is in the fail-over verification, given Chapters 1–3, 8, and 12.
2. If you add a tenant decision, do not copy Storefront’s `freeze-storefront-releases` row and rename it.
3. Do not apply newest mixed-region state, and do not emit `status: recovered` or `slo_met`.
4. Do not list DevOps reconstruction as sufficient for that data.
5. Identify one observation that would falsify recovery—for example Fulfillment intent in Storefront, elapsed above RTO, or fail-over counts below the catalog target.
6. Explain which material change would trigger review of the plan, not just the restore job name.

Do not copy the mixed-tenant denial and rename it. Catalog reads have different freeze and gravity consequences from checkout. Your durable output is the coverage decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 14 capability when you can explain why mixed-region newest is not fail-over, isolate the lost region, keep tenant continue/freeze explicit, meet RTO and RPO, list inherited restores as insufficient, refuse a recovered stamp that hides a miss, distinguish structural validation from decision, outcome, and recovery evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Journeys, **SLOs**, error-budget policy, on-call, learning, game days, and regional fail-over now exist as one governed reliability portfolio. The remaining owner recorded in Platform Chapter 15 and Decision 004 has a modeled answer.

Chapter 15 restates that outcome, the five principles as a portfolio-control loop, and what this book does not claim. It has no lab.

15. Conclusion — A Governed Reliability Portfolio

Northwind began with a working delivery path, a working security path, and an owned internal platform. Storefront could promote a digest and bind workload identity. Fulfillment was a second tenant with catalog owners and escalation contacts. Platform Chapter 14 could restore a control plane without taking tenants with it, and still recorded not-regional-loss and not-portfolio-rto. Remaining owner reliability-program held portfolio **SLO (Service Level Objective)** governance that did not yet exist.

A named journey was useful only when an **SLI (Service Level Indicator)** users could feel was accepted. An accepted indicator still could not freeze change. A freeze still could not page a living primary. A page still landed on Slack until on-call was a system. A rotation still drowned in toil until the work was bounded. Bounded time still could not attribute payment or warehouse loss. Attribution still cascaded until Northwind shed on purpose. A shed still could not command both tenants. A commanded incident still became a hortatory postmortem. Learning still assumed one region. Architecture still sat on paper until game days recurred. A tabletop still was not fail-over.

The book's central argument is therefore broader than a dashboard of nines:

Production **SRE (Site Reliability Engineering)** is the governance of reliability across a service portfolio: user-visible SLOs, error-budget policy that can freeze change, an on-call system rather than informal heroics, recurring game days, and recovery from regional loss with **Evidence of portfolio recovery**.

What Northwind can now do

Northwind can now:

- name protected journeys, refusals, and reliability-program as owner, and reject cluster uptime as success;
- accept Storefront order_success_ratio and Fulfillment dispatch_success_ratio, keep platform job-time adjacent, and reject component uptime;
- publish portfolio SLOs and windows, compute remaining error budget, and refuse **SLA (Service Level Agreement)** text as a target;
- continue, slow, or freeze change—including fleet step storage-1-0-to-2-0 by reference—when budget is exhausted;
- page on journey burn and ticket CPU, replica Ready, and job-time;
- page storefront-oncall-system, fulfillment-oncall-system, and platform-oncall-system rather than catalog contacts or Slack;
- classify toil, bound it numerically, and block a new critical SLO when the bound is breached;
- put payment and warehouse inside the journey contract and keep email non-critical;
- shed payment overload, account degraded success as burn, and deny a cascade into Fulfillment;

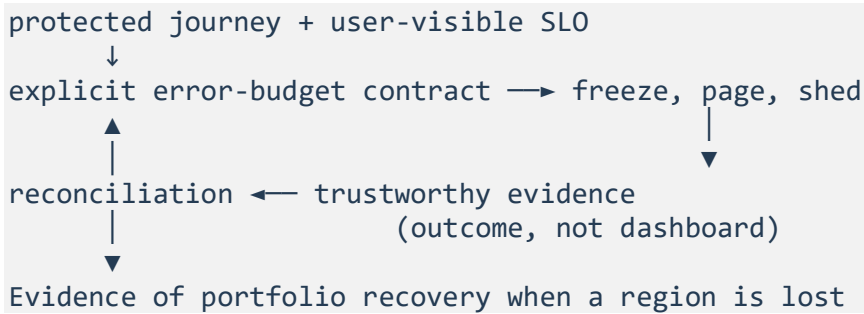
- command spanning incidents with a living primary, both journeys, and a freeze join, and refuse one-path close;
- record or waive incidents, own independently verified actions, and refuse hortatory follow-up;
- name two regions, numeric portfolio **RTO (Recovery Time Objective)** and **RPO (Recovery Point Objective)**, isolation that survives fail-over, and inherited restores as insufficient;
- run game days on a cadence that can abort, covering freeze, page path, dependency loss, and regional-loss tabletop, without completing on mixed-backup;
- fail over region-primary to region-standby, reject mixed-tenant and mixed-region replay, and compute **Evidence of portfolio recovery** without emitting status : recovered.

These capabilities form one program. They are not fourteen independent checklists. The promise they keep is the Chapter 1 contract, now executed:

Storefront and Fulfillment keep user-visible SLOs; exhausted error budget can freeze change, including fleet change; on-call and learning are systems rather than heroics; a region can be lost and the portfolio recovered from a reviewed plan, with evidence that this is not reconstruction of one environment or restore of one control plane.

The five principles as a portfolio-control loop

The recurring principles now connect as one loop around the reliability portfolio, not around a single team's pipeline or a single platform product:



Blast-radius control keeps Fulfillment's dispatch, warehouse loss, incident, game day, and fail-over from becoming Storefront's outage—and keeps mixed-tenant newest from becoming both.

Explicit contracts make journeys, SLI treatments, SLO windows, freeze actions, on-call systems, dependency criticality, degradation modes, incident command, learning actions, regional order, game-day kinds, and fail-over continue-or-freeze reviewable.

Trustworthy evidence separates observation from expectation. A catalog, page map, postmortem, game-day result, or verification cannot emit its own passing grade. Remaining budget is computed. Completeness is computed. status: recovered is not a hide for a missed RTO, a missed RPO, or collapsed isolation.

Reconciliation compares desired journeys, recorded last known good per region, inherited restores, and actual fail-over observations. Disagreement is owned. Newest is not last known good. Plane 1.0 is not tenant-storage-1.0. Neither is a region. A Chapter 13 tabletop is not Chapter 14 execution.

Recovery requires **Evidence of portfolio recovery**—not merely that one service returned, one environment was reconstructed, or one control plane was restored. Chapter 14 is the first chapter that produces that recovery evidence, and it is bounded: mixed replay rejected, isolation held, elapsed and data loss inside Chapter 12 objectives, inherited restores insufficient, recovered not self-emitted. It is not DevSecOps restored trust, not Platform restored isolation, and not proof a live multi-region estate failed over.

The four reliability questions remain the reading contract:

1. What user-visible journey is at risk, and for which services?
2. What error budget remains, and what change does it authorize or freeze?
3. What human system absorbs the failure without informal heroics?
4. What evidence proves the portfolio recovered—not only one service, one environment, or one control plane?

A later change that cannot answer those four questions is not an improvement. It is availability theater with a new graph, or informal heroics with a new chat channel.

What this book does not claim

The companion lab is intentionally deterministic. It proves reliability decisions, SLO and error-budget policy, on-call, toil, dependencies, degradation, incident command, learning, game days, and bounded fail-over without requiring a live telemetry backend, paging vendor, identity provider, multi-region fleet, or incident-management product. It does not prove that a particular organization's Prometheus, PagerDuty, identity provider, or public-cloud region behaves as the fixture does.

Production adoption requires replacing each simulated boundary with observed evidence from the real implementation. Values such as 99.5 percent, a 30-day window, a 90-day game-day cadence, RTO 14400, and RPO 900 must be measured under the actual journeys and failure modes.

The scope is also deliberately bounded:

- *Practical DevOps Engineering* remains the delivery path for one team: fast feedback, verifiable artifacts, reconciled infrastructure, bounded runtime, workload identity, observable outcomes, progressive delivery, compatible data change, safe asynchronous processing, **GitOps (Git-based operations)**, cost of one workload, incident coordination, and reconstruction from durable evidence. This book governs a portfolio that already has that path. It does not reteach it. One-environment reconstruction remains insufficient for regional loss.
- *Practical DevSecOps Engineering* remains the security of that path: threat and risk, attributable identity, privileged delegation, supply chain, vulnerability treatment, secrets, data protection, policy, runtime confinement, detection, investigation, eradication, bounded restored trust, and operational governance. This book may keep break-glass on-call attributable. It does not rebuild those controls. Reliability recovery is not **Evidence of restored trust**.

- *Practical Platform Engineering* remains the owned internal product: users, jobs, tenancy, catalog, paved road, environments, contracts, control plane, guardrails, developer-experience measurement, quota, fleet, support, and bounded plane restore. Jobs obtain-bounded-environment, ship-on-paved-road, and publish-owned-path stay named. Time-to-first-environment, paved-road-completion, and catalog-freshness remain a **job-time budget**. Catalog escalations remain contacts. Plane last known good 1.0 and contract last known good tenant-storage-1.0 stay distinct. This book may freeze a fleet because an error budget is exhausted; it does not redesign fleet upgrade or tenant isolation. Recovery here is not **Evidence of restored isolation** and not **Evidence of bounded platform-product recovery**.
- Governance may consume SLO reports, freeze decisions, incident records, game-day results, and fail-over verification. This book does not certify Northwind against a legal or industry framework, and it does not write customer SLAs as a legal product.
- A dedicated observability-stack textbook, chaos-engineering product manual, public-cloud multi-region reference, queueing-theory course, incident-command handbook, and AI-ops chapter are out of scope.

Chapter 14 proves that inventoried regional evidence can be failed over and tenant isolation can hold in the model. It does not claim that Northwind has completed an enterprise disaster-recovery program or a live multi-region fail-over.

What this book closes

Platform Chapter 15 handed off remaining owner reliability-program and named the unfinished work: portfolio SLOs for Storefront and Fulfillment, error-budget policy that can freeze a fleet for a reason Platform did not own, an on-call system rather than catalog contacts, regional loss Chapter 14 explicitly refused, and game days that exercise more than one mixed-backup fixture. Decision 004 recorded *Practical SRE Engineering* as the fourth book and required checksum-identified inheritance from DevOps, DevSecOps, and Platform.

Those remaining questions now have modeled answers. The jobs, tenancy, catalog contacts, job-time budget, and two last-known-good identities were inherited, not rediscovered. The lab never read earlier working trees at runtime. The published DevOps, DevSecOps, and Platform manuscripts were not rewritten.

There is no fifth series book in Decision 004. A later book would need a new series decision, its own planning set, and checksum-identified inheritance from the books it consumes.

The production question to keep asking

When evaluating a new SLO, freeze, page, rotation, game day, or fail-over tool, ask:

What user-visible journey is at risk, what error budget remains and what change it freezes, who is on-call with authority rather than heroics, and what evidence proves the portfolio recovered—not only one service, one environment, or one control plane?

That question keeps concepts subordinate to production work. It also prevents “best practice” from becoming a golden dashboard copied without a journey, a freeze, a living primary, or a fail-over that still isolates tenants.

The Northwind model and companion implementation are complete for the promise of this book. The lasting skill is not reproducing its YAML, SLO percentages, or RTO integers. It is being able to redesign the same reliability program when the organization, journeys, error budgets, on-call load, dependencies, regions, and constraints are different.

Glossary and Abbreviations

Abbreviations

Abbreviation and full form	Use in this book
CI/CD (Continuous Integration and Continuous Delivery)	Inherited delivery surface. This book does not teach it from first principles.
DR (disaster recovery)	A restore of one backup or one environment. It is not a portfolio game-day program and not regional fail-over.
GitOps (Git-based operations)	Inherited reconciliation from DevOps. Reconstruction of one environment remains insufficient for regional loss.
RPO (Recovery Point Objective)	How much journey state may be lost in a regional fail-over. Teaching value: 900 seconds.
RTO (Recovery Time Objective)	How long the portfolio may take to serve Chapter 1 journeys from the surviving region. Teaching value: 14400 seconds. Platform recorded not-portfolio-rto.
SLA (Service Level Agreement)	A customer or legal promise. It is not an SLO target.
SLI (Service Level Indicator)	A user-visible measurement of a protected journey, or an adjacent platform-product job-time proof. Component uptime is rejected.
SLO (Service Level Objective)	The internal reliability contract for a journey over a window. Error budget is computed from it.
SRE (Site Reliability Engineering)	Reliability engineering across a service portfolio: SLOs, error budgets, on-call, learning, game days, and regional fail-over.

How to Use This Book writes **SRE (Site Reliability Engineering)**, **CI/CD (Continuous Integration and Continuous Delivery)**, **SLOs (Service Level Objectives)**, and **SLIs (Service Level Indicators)** on first use. Later reader-facing chapters expand an abbreviation again on first use in that document.

Production terms

Availability theater

A green graph that does not represent a user-visible journey. Cluster uptime, replica Ready, and portal availability are theater unless they are refused with a remaining owner.

Error budget

Remaining unreliability for a portfolio SLO. Reserve the phrase for SRE governance. A Platform fleet freeze for an upgrade is not an error-budget freeze.

Evidence of portfolio recovery

Proof that a lost region was failed over, tenant isolation held, journey SLOs were met from independent observations, and inherited restores were not treated as recovery. It is not DevSecOps restored trust and not Platform restored isolation.

Job-time budget

Platform-product unreliability for jobs such as time-to-first-environment. It is never a portfolio SLO.

On-call system

Rotation, load, handoff, and authority bound to a living primary. Catalog contacts storefront-oncall, fulfillment-oncall, and platform-oncall are inputs, not the system.

Toil

Operational work that scales with production and does not permanently improve the system. A numeric bound protects engineering time; a breach blocks new critical SLO scope.

References

The chapter text cites sources at the decision they support. This consolidated list is organized for review and maintenance. Product behavior and documentation change; verify version-specific details before applying an example to production.

This book's production decisions are Northwind's. Chapters that make no product or standards claim beyond inherited contracts have no external vendor or framework entries.

Series books

- *Practical DevOps Engineering* v1.0 — one team's production delivery path: fast feedback, verifiable artifacts, reconciled infrastructure, bounded runtime, workload identity, observable outcomes, progressive delivery, compatible data change, safe asynchronous processing, **GitOps (Git-based operations)**, cost of one workload, incident coordination, and reconstruction from durable evidence.
- *Practical DevSecOps Engineering* v1.0 — security of that same path: assets and harms, threat and risk, attributable identity, privileged delegation, supply-chain trust, vulnerability treatment, secrets, data protection, policy, runtime confinement, detection, investigation, eradication, bounded restored trust, and operational governance.
- *Practical Platform Engineering* v1.0 — owned internal platform product: users, jobs, tenancy, catalog, paved road, environments, contracts, control plane, guardrails, developer-experience measurement, quota, fleet, support, and bounded plane restore. Remaining owner reliability-program holds portfolio **SLO (Service Level Objective)** governance. Chapter 14 recorded not-regional-loss and not-portfolio-rto.

Inherited contracts this book consumes

The SRE lab does not read earlier labs' working trees at runtime. It carries reduced, checksum-identified interface fixtures under `inherited/devops-v1.1/`, `inherited/devsecops-v1.0/`, and `inherited/platform-v1.0/`.

- DevOps provisional Storefront **SLIs (Service Level Indicators)** and burn policy — candidate input to Chapters 2 and 5, not a chapter to rewrite.
- DevOps one-change incident evidence — role interface for Chapter 10; insufficient to close a portfolio incident.
- DevOps one-environment reconstruction — insufficient for Chapters 12–14.
- DevSecOps `self_approval_forbidden` — named authorization field consumed by Chapter 6 on-call authority.
- DevSecOps independent-producer requirement — consumed by Chapter 11 learning verification.
- Platform job proofs `time-to-first-environment`, `paved-road-completion`, and `catalog-freshness` — a **job-time budget**, never a portfolio SLO.
- Platform catalog escalations `storefront-oncall`, `fulfillment-oncall`, and `platform-oncall` — contacts, not an on-call system.
- Platform fleet step `storage-1-0-to-2-0` — referenced by Chapter 4 error-budget freeze; upgrade freeze window, cohort, and rollback are not copied.
- Platform plane last known good `1.0` and contract last known good `tenant-storage-1.0` — distinct insufficient identities for regional loss.
- Platform limitations `not-regional-loss` and `not-portfolio-rto` — the gap Chapters 12–14 close.

Chapter 14 — Regional fail-over

No public-cloud multi-region reference or enterprise disaster-recovery program is cited. Modeled **Evidence of portfolio recovery** is computed from inventoried regional evidence, tenant isolation, numeric **RTO (Recovery Time Objective)** and **RPO (Recovery Point Objective)**, and independent journey observations. It does not prove a live multi-region estate.